
Package javax.ce.install

The Install package manages applications and standard extensions present in a Java application environment.

See:

Description

Class Summary	
<u>Installer</u>	An Installer manages applications and standard extensions present in a Java application environment.
<u>InstallInfo</u>	The installer keeps track of the information about each installed jar file in an InstallInfo object.

Package javax.ce.install Description

The Install package manages applications and standard extensions present in a Java application environment.

Package Features

The Installer API provides the following features:

- Convenient application deployment with no additional installation procedures needed
- This extension includes an interface to install and remove applications.
- Access to application identification during installation
- Management of application upgrades by providing access to GUI visibility procedures

Design Goals

To support application development and deployment , the API supports the following installation design goals:

- Convenient application packaging for development and distribution including the following components:
 - Jar files that contain application classes
 - Identifying the main entry point class for an application
 - Identifying icons to represent the application in a user interface
- Installation mechanisms for installing an application into the environment
- Versioning - identifying what version of an application is present

Open Issues

- The package name of the Install API proposed here may be changed to better reflect its appropriateness for smaller devices.

Install Code Sample

The Install package implements the Installer class which returns the unique name of the installed package based on a URL passed using the `getInstaller()` method, and installs the JAR file, for example, as illustrated in the following code sample.

```
/*
 * Copyright 1999 Sun Microsystems Inc, All rights reserved.
 */

import java.util.Date;
import javax.install.Installer;
import java.io.IOException;
import java.net.*;

/**
 * This example program shows how to install a new extension.
 */
public class SampleInstall1 {

    Installer installer;    // the installer

    /**
     * Install a new jar file.
     */
    public static void    main(String[] args) {
        SampleInstall1 sample = new
            SampleInstall1();

        if (sample.open()) {
            sample.install(args[0]);

            sample.close();
        }
    }

    /**
     * Install the specified jar file on the platform.
     */
    void install(String location) {
        try {
            URL url = new URL(location);
            String name    = installer.install(url);
            System.out.println("Installed" + name);
        } catch (IOException ex) {
            System.out.println("Installed failed: " + ex.getMessage());
        }
    }
}
```

```
/*
 * Open the installer.
 *
 * @return true if open was successful
 */
boolean open()
{
    installer = Installer.getInstaller();

    return (installer != null);
}

/*
 * Close the installer database.
 *
 * @return true if close was successful
 */
boolean close()
{
    installer = null;
    return true;
}
}
```

Hierarchy For Package javax.ce.install

Package Hierarchies:

All Packages

Class Hierarchy

- class java.lang.Object
 - class javax.ce.install.**Installer**
 - class javax.ce.install.**InstallInfo**
-
-

javax.ce.install **Class Installer**

```
java.lang.Object
|
+-- javax.ce.install.Installer
```

public abstract class **Installer**
extends java.lang.Object

An Installer manages applications and standard extensions present in a Java application environment. An application or extension library is distributed in a JAR file. The manifest describes the contents. Specific manifest header attributes specify how the contents will be installed. The Installer implementation is specific to the platform and provides access to procedures that make an application visible to users. During installation the Installer invokes methods in the installed software to allow the software to perform additional actions during install and uninstall.

Each installed package is uniquely identified by a symbolic name constructed from the combination of the values of the Implementation-Title and Implementation-Vendor attributes. The syntax and content of the strings used to identify installed packages are implementation dependent. Only packages installed or upgraded using this API appear in the list of known packages.

So that the API can be used by automated and software management tools, implementations *must not* utilize a user interface to prompt for confirmation or perform any action that would block.

Notes: Not all components of a system can be managed through this interface. For example, the base system might be burned into ROM or may require special procedures for native code, or device drivers.

The JAR file is the unit of installation as defined and used in the JDK 1.2 JRE specification with the addition of an attribute defining the icon(s) to be presented in a user interface. The table below refers to the manifest attributes of JDK 1.2. The attributes are defined in `InstallInfo`.

Description	Manifest attribute	Comments
Service name	Implementation-Title	non-parsable, human readable string for display in informational dialogs (SurfWriter)
Service version	Implementation-Version	i.e., SurfWriter 2.0.7
Service manufacturer	Implementation-Vendor	i.e., SurfWriter Technologies, Inc.
Title of the specification	Specification-Title	i.e., SurfWriter 2.0.7 Specification
Version number	Specification-Version	i.e., 1.0
Vendor's company or organization	Specification-Vendor	i.e., SurfWriter Technologies, Inc.
Main Class	Main-Class	Name of Java Class containing main() method
Main Icon(s)	Main-Icon	GIF file(s) within JAR containing icons representing the application
Bundled extensions	Class-Path	Relative location of other packages upon which this depends

See Also:

[InstallInfo](#)

Constructor Summary

Installer()

Method Summary

abstract <u>InstallInfo</u>	<u>getAttributes</u> (java.lang.String symbolicName) Get the attributes of the named installed package.
static <u>Installer</u>	<u>getInstaller()</u> Get the current installer for this Java Runtime Environment (JRE).
abstract java.lang.String	<u>install</u> (java.lang.String jarfilelocation) Install a software package from the JAR file at the URL.
abstract java.lang.String[]	<u>list()</u> List all installed software packages by symbolic name
abstract void	<u>remove</u> (java.lang.String symbolicName) Remove a software package given its symbolic name (e.g., com.acme.software)
static void	<u>setInstaller</u> (<u>Installer</u> installer) Set the current Installer for this Java Runtime Environment.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail*Installer*

```
public Installer()
```

Method Detail*getInstaller*

```
public static Installer getInstaller()  
                        throws java.lang.IllegalStateException
```

Get the current installer for this Java Runtime Environment (JRE).

Returns:

the current installer for this JRE.

Throws:

java.lang.IllegalStateException - if the installer has not been set.

setInstaller

```
public static void setInstaller(Installer installer)  
                        throws java.lang.SecurityException
```

Set the current Installer for this Java Runtime Environment. This overrides an installer configured with the *[TBD]* system property.

Parameters:

`installer` - the new installer for this JRE.

install

```
public abstract java.lang.String install(java.lang.String jarfilelocation)  
                        throws java.io.IOException,  
                        java.lang.SecurityException
```

Install a software package from the JAR file at the URL. The URL may be a file:URL or an http URL. Some platforms might only support installing from local storage.

If the component to be installed is the same as an existing component (by comparing the IMPLEMENTATION_TITLE and IMPLEMENTATION_VENDOR) then this install is an upgrade. and replaces in its entirety the current version. It is implementation dependent when the upgraded component is made available for use. The current version must continue to be available to any other components that are using it. The newer version may be made available immediately to other components

The implementation of install must be robust in the presence of failures such as running out of memory. If this method throws an exception then the package must *not* be installed and any previous version of the component must be left intact and operational.

Parameters:

`jarfilelocation` - the URL from which the jar file can be retrieved.

Returns:

the unique name of the installed package. The name is based on URL passed in and may be modified to be unique within the platform.

Throws:

`java.io.IOException` - is thrown if the jar file cannot be accessed.

`java.lang.SecurityException` - if the caller does not have permission to install software.

remove

```
public abstract void remove(java.lang.String symbolicName)
    throws java.lang.IllegalStateException
```

Remove a software package given its symbolic name (e.g., com.acme.software)

If the component is in use it must continue to be available to the other components that are using it. The resources it consumes must not be released until it is not in use.

Parameters:

`the` - Symbolic name for the installed package.

Throws:

`java.lang.SecurityException` - if the caller does not have permission to remove the installed package.

`java.lang.IllegalStateException` - if the symbolic name does not match any installed jar file.

`java.lang.SecurityException` - if the caller does not have permission to remove software.

list

```
public abstract java.lang.String[] list()
```

List all installed software packages by symbolic name

Returns:

an Array of the symbolic names for the installed packages.

Throws:

java.lang.SecurityException - if the caller does not have permission to see what software is installed.

getAttributes

public abstract InstallInfo **getAttributes**(java.lang.String symbolicName)

Get the attributes of the named installed package. The attributes include its title, version, for example. The predefined property names define the standard attributes for each application.

Parameters:

the - symbolic name for the installed package.

Returns:

a Attributes object containing the attributes of the installed package. `null` is returned if there is no package installed with the symbolic name.

Throws:

java.lang.SecurityException - if the caller does not have permission to see the attributes of installed software.

See Also:

InstallInfo

javax.ce.install

Class InstallInfo

```
java.lang.Object
|
+-- javax.ce.install.InstallInfo
```

public abstract class **InstallInfo**
 extends java.lang.Object

The installer keeps track of the information about each installed jar file in an InstallInfo object. An InstallInfo instance holds information from the Jar manifest which can be access via the `getAttribute` method.

The information held in an InstallInfo object is immutable; it will not change over time.

See Also:
[Installer](#)

Field Summary	
static java.lang.String	<u>IMPLEMENTATION_TITLE</u> Implementation title of the installed application.
static java.lang.String	<u>IMPLEMENTATION_VENDOR</u> Implementation vendor of the installed application.
static java.lang.String	<u>IMPLEMENTATION_VERSION</u> Implementation version of the installed application.
static java.lang.String	<u>MAIN_CLASS</u> The class name property identifying the class containing the <code>main</code> method of the installed application.
static java.lang.String	<u>MAIN_ICON</u> Icon(s) for the installed application.
static java.lang.String	<u>SPECIFICATION_TITLE</u> Specification title of the installed application.
static java.lang.String	<u>SPECIFICATION_VENDOR</u> Specification owner of the installed application.
static java.lang.String	<u>SPECIFICATION_VERSION</u> Specification version of the installed application.

Method Summary

abstract java.lang.String	<u>getAttribute</u> (java.lang.String attribute) Get an attribute associated with this installed application, service, or library.
abstract java.lang.String	<u>getName</u> () Return the symbolic name of this jar file.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

MAIN_CLASS

public static final java.lang.String **MAIN_CLASS**

The class name property identifying the class containing the `main` method of the installed application. When launched the `main` method of this class should be invoked.

MAIN_ICON

public static final java.lang.String **MAIN_ICON**

Icon(s) for the installed application. The value of the attribute is a comma separated list of paths within the JAR file containing GIF images. All of the images are used comparably to represent the application in a user interface. Multiple sizes of images may be provided to allow for an appropriate image for the user interface.

IMPLEMENTATION_TITLE

public static final java.lang.String **IMPLEMENTATION_TITLE**

Implementation title of the installed application. This string should be formatted for presentation to users as an unique identification of the application or extension library.

IMPLEMENTATION_VERSION

```
public static final java.lang.String IMPLEMENTATION_VERSION
```

Implementation version of the installed application.

IMPLEMENTATION_VENDOR

```
public static final java.lang.String IMPLEMENTATION_VENDOR
```

Implementation vendor of the installed application.

SPECIFICATION_TITLE

```
public static final java.lang.String SPECIFICATION_TITLE
```

Specification title of the installed application.

SPECIFICATION_VERSION

```
public static final java.lang.String SPECIFICATION_VERSION
```

Specification version of the installed application.

SPECIFICATION_VENDOR

```
public static final java.lang.String SPECIFICATION_VENDOR
```

Specification owner of the installed application.

Method Detail

getName

```
public abstract java.lang.String getName()
```

Return the symbolic name of this jar file.

getAttribute

public abstract java.lang.String **getAttribute**(java.lang.String attribute)

Get an attribute associated with this installed application, service, or library.

Parameters:

attribute - the name of one of the predefined attributes.

Returns:

the value of the attribute; `null` is returned if it has no value.

Package javax.net.datagram

The Network Datagram API allows program to program communication by means of sending and receiving datagrams.

See:

Description

Interface Summary

<u>Address</u>	Addresses for datagrams implement the Address interface.
-----------------------	--

Class Summary

<u>Datagram</u>	A Datagram holds the data and address of a single message received from or to be sent on a DatagramService.
<u>DatagramNameService</u>	A DatagramNameService is used to resolve server name and service names to transport specific addresses.
<u>DatagramService</u>	A DatagramService provides the ability to send and receive datagrams.
<u>DatagramServiceFactory</u>	DatagramServiceFactory is used to register and unregister DatagramServices that support network transports and bearers on the device.

Exception Summary

<u>AddressNotSupportedException</u>	AddressNotSupportedException reports that an address or address type is not supported.
<u>IntermittentNetworkException</u>	IntermittentNetworkExceptions report a network function could not be completed due to a transient network condition.
<u>MalformedAddressException</u>	MalformedAddressException reports that address information is incorrect or in an invalid format.

Package javax.net.datagram Description

The Network Datagram API allows program to program communication by means of sending and receiving datagrams. The API allows developers only to be concerned with the addressing and data protocol while hiding the details of the transport from the application.

Design Goals

The primary design goal for the Network Datagram API is to provide applications with smooth program to program communication of datagrams that are independent of the underlying transport or bearer. The API was designed to:

- Be independent of the transport or bearer
- Encapsulate transport dependent address information in a name service
- Applications should not have to be re-written to run on various phone or networks
- Support non-IP transports such as SMS, GSM data, or Pagers
- Support client and server applications
- Support multiple applications concurrently
- Support abstract and concrete addressing
- Support unicast or multicast addressing

Network Datagram API Sample Code

The following sample program shows how to use the DatagramService to send and receive datagrams. It uses the server and service names to locate the address of the service. The message to be sent is encapsulated in a Datagram and send to the appropriate DatagramService for that address.

```
/*
 * Copyright 1999 Sun Microsystems Inc, All rights reserved.
 */

import javax.net.datagram.*;
import java.io.IOException;

/**
 * Send a series of Datagrams to ourselves.
 * Receive and print them.
 */
public class DatagramSample1 implements Runnable {
    DatagramService service;

    public static void main(String[] args) {
        DatagramSample1 m = new DatagramSample1();
        m.test();
    }

    private void test() {
        try {
            Address addr = DatagramNameService.lookup("localhost", "message");
```

```

        service = DatagramService.getService(addr);

        // Start receiving thread
        Thread t = new Thread(this, "Receiver");
        t.start();

        for (int i = 0 ; i < 10; i++) {
            String message = "Message " + i;
            Datagram d = new Datagram(message.getBytes(), addr);
            service.send(d);

            sleep(1000);
        }

        sleep(5);          // wait for last few to be received

        service.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

/*
 * Thread to receive datagrams and print them.
 */
public void run() {
    byte[] buffer = new byte[128];
    Datagram datagram = new Datagram(buffer);
    while (true) {
        try {
            datagram.setLength(buffer.length);
            service.receive(datagram);
            String message = new String(datagram.getData(),
                                         0, datagram.getLength());
            System.out.println("Received " + message);
        } catch (IOException e) {
            System.out.println("Error receiving: " + e.getMessage());
            break;
        }
    }
}

/*
 * Simple method to sleep for some number of milliseconds.
 */
private void sleep(int n) {
    try {
        Thread.sleep(n);
    } catch (InterruptedException e) {
    }
}
}

```


Abstract Addressing

Abstract addressing simplifies the application by providing a mechanism for mapping server names and service names to the address of the service. The mapping is performed on the client device possibly using a local directory service or a network directory service.

`DatagramNameService` supports abstract addressing. It provides methods to look up the address of a particular server and service by name. It returns an `Address` that is used to select the `DatagramService`.

Concrete Addressing

Concrete addressing is used when the application is aware of the service address and does not need the indirection provided by abstract addressing. The external form of concrete addresses uses string formats similar to URLs to select the transport and provide arguments that identify the transport specific address information.

`DatagramService` provides the method to parse the external form of addresses and return `Address` objects that are used to select the `DatagramService`.

Several concrete address types are defined following the familiar patterns established for URLs by [RFC 1738](#).

Protocol	Comment	Parameters
sms	raw text-only GSM sms message	GSM phone number
udp	UDP datagram via IP	[IP host or address] and port
wdpsms	WAP datagram via GSM SMS message(s)	[GSM phone number] and port - defaults to configured WAP server

SMS Address Type

The SMS address type supports using the Datagram API to send and receive SMS messages. For raw text messages the address is the phone number of the destination phone.

; SMS Datagram

```
smsaddr      = "sms://" phonenum
phonenum     = [ "+" ] 1*phonechar
phonechar    = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" |
               "8" | "9" | "#" | "*"

```

For example:

```
sms://+1234567890
    SMS to the GSM phone number

```

UDP Datagram Address Type

The UDP address type supports using the Datagram API to send and receive IP datagrams addressed to a host and port.

; UDP Datagram

```
udpaddr      = "udp://" hostport
hostport     = host [ ":" port ]
host         = hostname | hostnumber
hostname     = *[ domainlabel "." ] toplabel
domainlabel  = alphanumerical | alphanumerical *[ alphanumerical | "-" ] alphanumerical
toplabel     = alpha | alpha *[ alphanumerical | "-" ] alphanumerical
alphanumerical = alpha | digit
hostnumber   = digits "." digits "." digits "." digits
port         = digits

```

For example:

```
udp://joes.pizzas.com:7654
    UDP to IP address and to a port
udp://12.34.56.78:7654
    UDP to a port

```

WDP over SMS Datagram Address Type

The WDP over SMS datagram type supports using the extended addressing features of WDP on top of SMS. Datagrams are addressed to a phone number plus a port number.

; WDP over SMS Datagram

```
wdpsmsaddr   = "wdpsms://" phoneport
phoneport    = phonenum [ ":" port ]

```

For example:

```
wdpsms://+1234567890:1234
    WDP via SMS to GSM phone number and to a port

```

Notes for implementors of DatagramService

An implementation of DatagramService is needed for each transport supported by the device. Each device also needs to implement a DatagramNameService.

When implementing a DatagramService a number of considerations need to be kept in mind. For each transport subclasses are used to implement Address, DatagramService, and DatagramServiceFactory. The DatagramServiceFactory needs to be registered with the type of Address it handles.

The implementation of DatagramNameService needs to use local (or possibly remote) information to map server and service names to transport specific Address instances. The names are resolved by a name service to address, bearer, port number etc. For example, for a device with GSM SMS support the following steps would create the necessary datagram address. If the server is "The PIZA Hut", and the service="orders" the device will look up the service name, and resolve "The PIZA Hut:orders" to bearer SMS, phone number +3585551212, protocol WDP/SMS, and port 1234. It would create an appropriate typed Address instance for WDP/SMS and initialize it with the phone number and port number.

The name service (daemon or implementation detail of the API) shall at least reside on the phone, but it may optionally delegate certain name lookups to a remote name server. e.g. if/when this remote name service is implemented, then a client application may get connected to a different host from the same service provider when the device roams to a new network. The remote name service is not being defined or standardised.

Classes implementing Address

This datagram API does not define any Address classes. The implementation will need to define some classes implementing the Address interface. The application must never use these classes explicitly, because that breaks the write-once-run-anywhere requirements.

Concrete addressing is supported by each address class and by the DatagramServiceFactory that supports that protocol. The Address class must implement its `toString` method to produce a string formatted according the formats above as appropriate. The DatagramServiceFactory must be able to parse the format produced by the corresponding address class.

This API purposely omits classes implementing Address. This is because the applications do not need to be aware of the addressing details, and should not be aware of them.

Receiving datagrams should be as bearer agnostic as possible... i.e. if the AddressBook contains multiple bearer/transport/protocol options for a particular server and service, then the implementation can select one that is working.

Using the local Addressbook for a DatagramNameService

The Datagram API may resolve server/service names using a name resolution service. For devices that include an addressbook it is recommended that the implementation include a name service that resolves names using contact information from the addressbook. The server name is used to identify the ContactCard for the server and selected ItemFields are used to select the service and the transport

selection and address information as described below.

The addressbook entries should be searched for items whose name matches the server/service name by the Datagram API implementation to resolve names to actual addresses. It should look for an item of type "FN" with a value equal to the server name.

Then it should search for fields named by the service name concatenated to "X-DATAGRAM-".

The value of each X-DATAGRAM- field should be the address for the service using the syntax described above for Concrete Addressing. If one of the fields has the "PREF" Parameter then its address should be used if it is available before other available addresses.

For example, the code fragment below will provide an idea of what is needed for a ContactCard with ItemFields (corresponding to server "The PIZZA Hut", and service "orders").

```
ItemField("FN", "The PIZZA Hut")
ItemField("TEL", "+1-555-1212")
ItemField("X-DATAGRAM-ORDERS", "sms://+1888-555-1212")
ItemField("X-DATAGRAM-ORDERS", "udp://147.23.120.2:543")
```

These extensions ("X-DATAGRAM*" etc) should not be visible to the user unless explicitly turned on in the AddressBook application configuration. Adhering to this advice will soften the problem of maintaining the indirection information.

The indirection provided by DatagramNameService decouples the addressing but introduces the problem of maintaining the indirection information. In the case of the AddressBook information may be either typed in by the user or is copied in from a vCard.

Hierarchy For Package javax.net.datagram

Package Hierarchies:

All Packages

Class Hierarchy

- class java.lang.Object
 - class javax.net.datagram.**Datagram**
 - class javax.net.datagram.**DatagramNameService**
 - class javax.net.datagram.**DatagramService**
 - class javax.net.datagram.**DatagramServiceFactory**
 - class java.lang.Throwable (implements java.io.Serializable)
 - class java.lang.Exception
 - class java.io.IOException
 - class javax.net.datagram.**AddressNotSupportedException**
 - class javax.net.datagram.**IntermittentNetworkException**
 - class javax.net.datagram.**MalformedAddressException**

Interface Hierarchy

- interface javax.net.datagram.**Address**
-

javax.net.datagram **Interface Address**

public interface **Address**

Addresses for datagrams implement the Address interface. Each type of Address is a transport specific address object that contains the protocol and addressing information. For example, a `UDPAAddress` would contain a hostname and port number or a `SMSAddress` would contain a phone number. When sending and receiving datagrams the type of address is used to select the `DatagramService` which in turn selects the transport.

Named service based addressing is supported by `DatagramNameService`. The application only needs the service name and service address. The implementation maps the service name and address onto a transport and transport specific address. This gives the application a great deal of independence from the details of the transport.

Address and its subtypes and implementations must be immutable. They must not have any methods that change the address. Only constructors are allowed to create and initialize the values of Address fields.

See Also:

[Datagram](#), [DatagramService](#), [DatagramNameService](#)

Method Summary	
<code>java.lang.String</code>	<code>toString()</code> Get the external form of this Address.

Method Detail

toString

public java.lang.String **toString()**

Get the external form of this Address. The encoded address must be a format required by the `DatagramService` `parseAddress` method to reconstruct an Address.

The external form of the address is similar to url formats where the scheme identifies the physical transport and the rest of the string is transport specific. See the `javax.net.Datagram` package documentation for details.

Overrides:

toString in class java.lang.Object

Returns:

the String containing an external parsable representation of this address.

*javax.net.datagram****Class MalformedAddressException***

```

java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--java.io.IOException
            |
            +--javax.net.datagram.MalformedAddressException

```

```

public class MalformedAddressException
extends java.io.IOException

```

MalformedAddressException reports that address information is incorrect or in an invalid format.

See Also:

[Serialized Form](#)

Constructor Summary

<u>MalformedAddressException</u> (java.lang.String message)	
Create a new exception with the specified message.	

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

MalformedAddressException

```
public MalformedAddressException(java.lang.String message)
```

Create a new exception with the specified message.

*javax.net.datagram***Class AddressNotSupportedException**

```

java.lang.Object
|
+-- java.lang.Throwable
    |
    +-- java.lang.Exception
        |
        +-- java.io.IOException
            |
            +-- javax.net.datagram.AddressNotSupportedException

```

```

public class AddressNotSupportedException
    extends java.io.IOException

```

AddressNotSupportedException reports that an address or address type is not supported. For example, it is used to report that there is no UDP protocol support.

See Also:

[Serialized Form](#)

Constructor Summary

AddressNotSupportedException (java.lang.String message)
Create a new exception with the specified message.

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

AddressNotSupportedException

```
public AddressNotSupportedException(java.lang.String message)
```

Create a new exception with the specified message.

javax.net.datagram***Class IntermittentNetworkException***

```

java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--java.io.IOException
            |
            +--javax.net.datagram.IntermittentNetworkException

```

public class **IntermittentNetworkException**
 extends java.io.IOException

IntermittentNetworkExceptions report a network function could not be completed due to a transient network condition. The function can be retried at a later time.

See Also:

[Serialized Form](#)

Constructor Summary

IntermittentNetworkException(java.lang.String message)

Create a new exception with the specified message.

IntermittentNetworkException(java.lang.String message,
 java.io.IOException cause)

Create a new exception with the specified message and underlying cause of this intermittent condition.

Method Summary

java.io.IOException **getCause**()

Get the underlying cause of this intermittent network condition.

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace,
 printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail*IntermittentNetworkException*

```
public IntermittentNetworkException(java.lang.String message)
```

Create a new exception with the specified message.

Parameters:

message - the cause of the exception.

IntermittentNetworkException

```
public IntermittentNetworkException(java.lang.String message,  
                                     java.io.IOException cause)
```

Create a new exception with the specified message and underlying cause of this intermittent condition.

Parameters:

message - the description of the cause.

cause - the exception that caused the intermittent condition .

Method Detail*getCause*

```
public java.io.IOException getCause()
```

Get the underlying cause of this intermittent network condition.

Returns:

the IOException that caused this intermittent exception; may be null if there was no exception to report.

javax.net.datagram **Class Datagram**

```
java.lang.Object
|
+-- javax.net.datagram.Datagram
```

```
public class Datagram
extends java.lang.Object
```

A Datagram holds the data and address of a single message received from or to be sent on a DatagramService. It consists of an array of bytes to be sent or received and the address that sent or is to receive the data.

All of the methods of Datagram are unsynchronized. If an application needs to share the datagram between its own threads it can perform the synchronization. While a DatagramService is sending or receiving a datagram the application must not access any of its methods.

See Also:

[DatagramService](#), [Address](#)

Constructor Summary

Datagram(byte[] buf)

Construct a datagram with the buffer.

Datagram(byte[] buf, [Address](#) addr)

Construct a new datagram with the buffer and address.

Datagram(byte[] buf, int offset, int len, [Address](#) addr)

Construct a new datagram with the buffer, offset, length and address.

Method Summary

<u>Address</u>	<u>getAddress</u> () The address this datagram was sent from or will be sent to.
byte[]	<u>getData</u> () Get the data buffer of this datagram.
int	<u>getLength</u> () Get the length of the data in this datagram.
int	<u>getOffset</u> () Get the offset of the first byte of data in this datagram.
void	<u>setAddress</u> (<u>Address</u> addr) Set the address this was sent from or will be sent to.
void	<u>setData</u> (byte[] buffer, int offset, int len) Set the data, offset, and length of the data of this datagram.
void	<u>setLength</u> (int len) Set the length of the data in this datagram.
void	<u>setOffset</u> (int offset) Set the offset of the first byte of data in this datagram.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

Datagram

```
public Datagram(byte[] buf,
                 int offset,
                 int len,
                 Address addr)
```

Construct a new datagram with the buffer, offset, length and address.

Parameters:

data - the array of bytes in this datagram

offset - into the buffer of the first byte to be sent or received.

length - the length of the datagram in the byte array. The length plus offset must be less than or equal to buf.length.

`addr` - the Address the datagram was sent from or should be sent to.

Throws:

`java.lang.IllegalArgumentException` - if the offset or length is less than zero or the length plus offset is greater than the length of the byte array.

Datagram

```
public Datagram(byte[] buf,  
                Address addr)
```

Construct a new datagram with the buffer and address. The length is set to the length of the buffer. The offset is set to zero.

Parameters:

`buf` - the array of bytes to buffer this datagram.

`addr` - the Address of the datagram.

Datagram

```
public Datagram(byte[] buf)
```

Construct a datagram with the buffer. This datagram can be used to receive a datagram or to send one after the address is set. The length is set to the length of the buffer. The offset is set to zero. The address is set to `null`.

Parameters:

`buffer` - the array of bytes to buffer this datagram.

Method Detail

getData

```
public byte[] getData()
```

Get the data buffer of this datagram. The contents of the buffer are not copied. The valid data in the buffer is described by the offset and length. The application should use `getOffset` and `getLength` to operate only on the valid data in the buffer. The offset indicates the first data byte in the buffer. The length indicates the number of bytes after the offset of valid data.

Returns:

the buffer for this datagram.

setData

```
public void setData(byte[] buffer,  
                    int offset,  
                    int len)
```

Set the data, offset, and length of the data of this datagram.

Parameters:

`buffer` - the array of bytes containing the data.

`offset` - the offset in the buffer of the first byte of the data.

`len` - the length of the data in the buffer.

Throws:

`java.lang.IllegalArgumentException` - is thrown if the offset or length is less than zero or the offset plus length is greater than the length of the buffer.

getLength

```
public int getLength()
```

Get the length of the data in this datagram.

Returns:

the number of data bytes in the datagram.

setLength

```
public void setLength(int len)
```

Set the length of the data in this datagram. The length plus offset must be less than or equal to `getData().length`.

Parameters:

`len` - the number of data bytes in the datagram.

Throws:

`java.lang.IllegalArgumentException` - is thrown if length is less than zero or the offset plus length is greater than the length of the buffer.

getOffset

```
public int getOffset()
```

Get the offset of the first byte of data in this datagram.

Returns:

the offset of the first byte of data in the datagram.

setOffset

public void **setOffset**(int offset)

Set the offset of the first byte of data in this datagram. The offset must be greater than or equal to zero and offset plus length must be less than or equal to `getData().length`.

Parameters:

offset - of the first byte of the data in the datagram.

Throws:

`java.lang.IllegalArgumentException` - is thrown if the offset is less than zero or the offset plus length is greater than the length of the buffer.

getAddress

public Address **getAddress**()

The address this datagram was sent from or will be sent to.

Returns:

the address this datagram was sent from or will be sent to.

setAddress

public void **setAddress**(Address addr)

Set the address this was sent from or will be sent to.

Parameters:

addr - the address this datagram was sent from or will be sent to.

*javax.net.datagram****Class DatagramNameService***`java.lang.Object`

|

`+-- javax.net.datagram.DatagramNameService`

public abstract class **DatagramNameService**

extends `java.lang.Object`

A `DatagramNameService` is used to resolve server name and service names to transport specific addresses. Multiple name services are supported and used in sequence to lookup names. Registration methods provide for the addition and removal of name services.

To resolve a server and service name the `lookup` method is called. It invokes the `lookupService` method of each service. If no exception is thrown the returned `Address` object is the address for the service.

The mapping of server and service names is system dependent and the management of the mapping information is outside the scope this specification. The name service must be able to map the server and service names to the transport specific `Address`. The transport specific information must be sufficient to create the appropriate subclass of `Address` object needed by the transport. For example, if the name service includes a mapping to UDP datagrams then it must be able to supply a hostname or IP address and port number suitable for creating a hypothetical `UDPAddress` object with that information. Each `DatagramNameService` installed must be able to extract the information and know how to create the address object from it.

See Also:

[DatagramService](#), [Address](#)

Method Summary

static void	<u>addNameService</u> (DatagramNameService service) Add a name service used to map server and service names to an Address.
static <u>Address</u>	<u>lookup</u> (java.lang.String server, java.lang.String service) Lookup the named server and service name and return an Address for it.
static <u>Address</u> []	<u>lookupAll</u> (java.lang.String server, java.lang.String service) Lookup all of the Addresses for the named server and service.
static void	<u>removeNameService</u> (DatagramNameService service) Remove the DatagramNameService registered to map server and service names to an Address.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

lookup

```
public static Address lookup(java.lang.String server,
                               java.lang.String service)
    throws AddressNotSupportedException,
           MalformedAddressException,
           java.lang.IllegalStateException
```

Lookup the named server and service name and return an Address for it.

Each registered DatagramNameService is requested to lookup the server name and service name using its `lookupService` method. The search terminates when an Address is returned or there are no more DatagramNameServices. If a MalformedAddressException, or AddressNotSupported exception occurs in `lookupService` the last such exception is saved and the search continued. If no DatagramNameService returns an address the saved exception is thrown if any.

Parameters:

server - the name of the server providing service
service - the name of the service requested

Returns:

the Address object for the service. `null` is returned if the service has no mapping for the server and service.

Throws:

[AddressNotSupportedException](#) - is thrown if the address mapping refers to an unavailable address type.

[MalformedAddressException](#) - is thrown if the mapping for the requested service cannot be decoded.

`java.lang.IllegalStateException` - is thrown if there are no DatagramNameServices registered. This is an error in the configuration.

lookupAll

```
public static Address[] lookupAll(java.lang.String server,
                                java.lang.String service)
                                throws AddressNotSupportedException,
                                MalformedAddressException,
                                java.lang.IllegalStateException
```

Lookup all of the Addresses for the named server and service. The server name and service name are mapped by each registered DatagramNameService to an address type and encoded address information.

Each registered DatagramNameService is requested to lookup the server name and service name using its `lookupAllServices` method. All of the addresses found from all services are returned in an array. If a `MalformedAddressException`, or `AddressNotSupported` exception occurs in `lookupService` the last such exception is saved and the search continued. If no DatagramNameService returns any Addresses the saved exception is thrown if any.

Parameters:

`server` - the name of the server providing service

`service` - the name of the service requested

Returns:

an array of the address objects for the service; `null` is returned if no mappings were found for the server and service name.

Throws:

[AddressNotSupportedException](#) - is thrown if an address mapping refers to an unavailable address type.

[MalformedAddressException](#) - is thrown if the mapping for the the requested service cannot be decoded.

`java.lang.IllegalStateException` - is thrown if there are no DatagramNameServices registered. This is an error in the configuration.

addNameService

```
public static void addNameService(DatagramNameService service)
```

Add a name service used to map server and service names to an Address. The new NameService will be searched after previously added NameServices.

Parameters:

`service` - a DatagramNameService to be used to map server and service names

Throws:

java.lang.SecurityException - is thrown if the caller does not have permission to add a service.

removeNameService

```
public static void removeNameService(DatagramNameService service)
```

Remove the DatagramNameService registered to map server and service names to an Address. Silently ignores trying to remove a service that is not registered.

Parameters:

`service` - a DatagramNameService to be removed.

Throws:

java.lang.SecurityException - is thrown if the caller does not have permission to remove the service.

javax.net.datagram

Class DatagramService

```
java.lang.Object
|
+-- javax.net.datagram.DatagramService
```

public abstract class **DatagramService**
extends java.lang.Object

A DatagramService provides the ability to send and receive datagrams. DatagramServices act as client or as a server depending upon how the DatagramService is being used. Two types of addressing are supported, abstract and concrete. DatagramServices support abstract addressing based on the Address types with server name and service name addressing supported by the **DatagramNameService**.

DatagramServices support concrete addressing by using URL based formats for the external form of the address. Each format is specific to a transport protocol. Each DatagramService can act as both a source of datagrams and a destination for datagrams. Each DatagramService has an address which is the source address for the datagrams it sends.

To send a datagram the application must get the destination address of the service. The **DatagramNameService.lookup** method is used to find the Address of a service. The specific address returned will indicate the transport or bearer type and contain the transport or bearer specific address. Addresses may also be created directly using **DatagramService.parseAddress**.

Each Datagram holds the address to which the data should be sent and the offset, length and buffer holding the data.

The DatagramService that will be used to send the datagram is selected based on the usage and address. If the application is acting only as a client of another service then **DatagramService.getService** is used to find the DatagramService. The destination address's type is used to select the DatagramService appropriate for the addresses transport or bearer and an source address is assigned. This address is sent with datagrams so that that replies to datagrams can be returned to the datagram source.

If the application is acting as a server then the service name and current host should be passed to **DatagramNameService.lookup** to get the well known address of the local service. That address should be passed to **DatagramService.getServerService** to select the appropriate DatagramService and set the source address. Individual DatagramServices register themselves with the **DatagramServiceFactory**.

Implementations of DatagramService must be multi-thread safe. That is, their methods must be re-entrant and protect any data structures from concurrent modification.

DatagramService Examples

To send a message, an application might write code as follows:

```
Address address = DatagramNameService.lookup("The PIZA Hut", "orders");
String msg = "family size super supreme, 2 x garlic bread, 2 litres pepsi";
DatagramService serv = DatagramService.getService(address);
Datagram d = new Datagram(msg.getBytes(), address);
serv.send(d);
// Send other datagrams to other Addresses
serv.close();
```

To send a message and receive a reply, an application might write code as follows:

```
Address address = DatagramNameService.lookup("The PIZA Hut", "orders");
String msg = "family size super supreme, 2 x garlic bread, 2 litres pepsi";
DatagramService serv = DatagramService.getService(address);
Datagram d = new Datagram(msg.getBytes(), address);
serv.send(d);

Datagram r = new Datagram(new byte[serv.getMaximumLength()]);
serv.receive(r);

serv.close();
```

To setup a server to accept Datagrams to this service, an application may write code as follows:

```
Address address = DatagramNameService.lookup("The PIZA Hut", "orders");
DatagramService serv = DatagramService.getServerService(address);

byte[] buf = new byte[serv.getMaximumLength()];
Datagram d = new Datagram(buf);
while (true) {
    d.setLength(buf.length);
    serv.receive(d);
    // Handle datagram and send a reply
    String msg = "Will be ready in 15 minutes";
    Datagram reply = new Datagram(msg.getBytes(), d.getAddress());
    serv.send(reply);
}
```

See Also:

[DatagramServiceFactory](#), [Datagram](#), [DatagramNameService](#), [Address](#)

Method Summary	
abstract void	<u>close()</u> Close the service.
abstract <u>Address</u>	<u>getAddress()</u> Get the address supported by this DatagramService.
abstract int	<u>getMaximumLength()</u> Get the maximum datagram size for this DatagramService.
abstract int	<u>getNominalLength()</u> Get the nominal datagram size for this DatagramService.
static <u>DatagramService</u>	<u>getServerService(Address addr)</u> Return the DatagramService that will act as a server for the specified Address.
static <u>DatagramService</u>	<u>getService(Address addr)</u> Return the DatagramService for the Address.
static <u>Address</u>	<u>parseAddress(java.lang.String address)</u> Parse the string address into an Address object.
abstract void	<u>receive(Datagram datagram)</u> Receive a datagram from this service.
abstract void	<u>receive(Datagram datagram, int milliseconds)</u> Receive a datagram from this service but return anyway if a datagram is not received within the timeout.
abstract void	<u>send(Datagram datagram)</u> Send the specified datagram to its destination address.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

getService

```
public static DatagramService getService(Address addr)
                                   throws IntermittentNetworkException,
                                           AddressNotSupportedException,
                                           java.io.IOException
```

Return the DatagramService for the Address. Specific address types identify the transport and physical address. The address is used to locate the DatagramService that can send and receive datagrams for the address type. Each of the registered DatagramServiceFactories is requested to open the service using its `open` method. The first DatagramService returned is used. The service can then be used to send and receive datagrams. The `getService` method should be used when the application needs to send datagrams and may expect a reply back from the service. A source address is assigned to this service and will not be re-used until the service is closed. Datagrams sent on this service include the source address so replies can be returned to this service.

Parameters:

addr - the Address to open as a client.

Returns:

the DatagramService used to send and receive messages.

Throws:

AddressNotSupportedException - is thrown if no transport is found for the address type.

IntermittentNetworkException - is thrown when an intermittent network problem has occurred.

It is up to the application to retry.

java.io.IOException - is thrown for errors that occur opening the service.

getServerService

```
public static DatagramService getServerService(Address addr)
                                   throws IntermittentNetworkException,
                                           AddressNotSupportedException,
                                           java.net.BindException,
                                           java.io.IOException
```

Return the DatagramService that will act as a server for the specified Address. Each of the registered DatagramServiceFactories is requested to open the service using its `openServer` method. The first DatagramService returned is used. The DatagramService will use the address specified to determine the source address for datagrams to be sent so that replies will return to this service. Depending on the address and the type of transport the address may not be available to another application until the service is closed.

Parameters:

addr - the Address to open as a server.

Returns:

the datagram service used to send and receive messages.

Throws:

AddressNotSupportedException - is thrown if no transport is found for the address type.

IntermittentNetworkException - is thrown when an intermittent network problem has occurred.

It is up to the application to retry.

java.net.BindException - is thrown if the specified address cannot be allocated.
java.io.IOException - is thrown for errors that occur opening the service.

parseAddress

```
public static Address parseAddress(java.lang.String address)
                                throws MalformedURLException,
                                AddressNotSupportedException
```

Parse the string address into an Address object. Each of the DatagramServiceFactories is requested to parse the address. The first returned Address is used. If a AddressNotSupportedException or MalformedURLException is thrown by any factory the last such exception is saved. If no Address is available the saved exception is thrown. If there are no factories registered then an AddressNotSupportedException is thrown.

The external form of the address is similar to url formats where the scheme identifies the physical transport and the rest of the string is transport specific. See the javax.net.Datagram package documentation for details.

Parameters:

address - the String containing an address in external format.

Returns:

an Address created by parsing the string.

Throws:

AddressNotSupportedException - is thrown if the address is not supported by any of the registered factories.

MalformedURLException - is thrown if the syntax of address is incorrect.

send

```
public abstract void send(Datagram datagram)
                        throws AddressNotSupportedException,
                        IntermittentNetworkException,
                        java.io.InterruptedIOException,
                        java.io.IOException
```

Send the specified datagram to its destination address. Send returns when the datagram has been sent. The Datagram can subsequently be modified without affecting what is actually transmitted. The specific DatagramService would need to copy the datagram if it will be queued for later transmission.

Parameters:

datagram - to be sent.

Throws:

AddressNotSupportedException - is thrown if the destination address is not supported by this service. and error occurs during sending.

IntermittentNetworkException - is thrown if an intermittent network problem has occurred. It is up to the application to retry.

java.io.InterruptedIOException - is thrown if close is called during the send.

java.io.IOException - or one of its subclasses is thrown if the service has been closed, if the

length of the datagram is greater than `getMaximumLength` allows or other transport specific errors occur.

receive

```
public abstract void receive(Datagram datagram)
    throws AddressNotSupportedException,
           IntermittentNetworkException,
           java.io.IOException
```

Receive a datagram from this service. The method will block until a datagram is received. The datagram will have the address of the sending service. The Address is transport specific and replies are usually only sent on the same transport. The offset is unchanged by `receive`. The data will be placed in buffer starting at the offset and will not exceed the number of bytes specified by length. The length will be set to the actual number of bytes received. Bytes in a datagram that do not fit in the buffer are discarded.

Parameters:

`datagram` - to be filled in with data and the source address from the received datagram.

Throws:

IntermittentNetworkException - is thrown if an intermittent network problem has occurred. It is up to the application to retry.

AddressNotSupportedException - is thrown if the DatagramService cannot receive datagrams.

`java.io.IOException` - or one of its subclasses is thrown if the service has been closed or other transport specific errors occur.

receive

```
public abstract void receive(Datagram datagram,
    int milliseconds)
    throws IntermittentNetworkException,
           AddressNotSupportedException,
           java.io.InterruptedIOException,
           java.io.IOException
```

Receive a datagram from this service but return anyway if a datagram is not received within the timeout. The method will block until a datagram is received or the timeout expires. The datagram will have the address of the sending service. The Address is transport specific and replies are usually only sent on the same transport. The offset is unchanged by `receive`. The data will be placed in buffer starting at the offset and will not exceed the number of bytes specified by length. The length will be set to the actual number of bytes received. Bytes in a datagram that do not fit in the buffer are discarded.

Parameters:

`datagram` - to be filled in with data and the source address from the received datagram.

`milliseconds` - the maximum number of milliseconds to wait for a datagram. If less than or equal to zero, `receive` will return immediately or as soon as possible. If there is a Datagram available it will be returned.

Throws:

AddressNotSupportedException - is thrown if the DatagramService cannot receive datagrams.

IntermittentNetworkException - is thrown if an intermittent network problem has occurred. It is up to the application to retry.

java.io.InterruptedIOException - is thrown if the timeout expires.

java.io.IOException - or one of its subclasses is thrown if the service has been closed or other transport specific errors occur.

getNominalLength

```
public abstract int getNominalLength()  
                    throws java.io.IOException
```

Get the nominal datagram size for this DatagramService. The nominal size indicates the size of datagram that the transport or bearer can convey without fragmenting. In other words, it fits in the physical packet size.

Returns:

the number of bytes that fit in a single message on this DatagramService.

Throws:

java.io.IOException - or one of its subclasses is thrown if any error occurs getting the nominal length.

getMaximumLength

```
public abstract int getMaximumLength()  
                    throws java.io.IOException
```

Get the maximum datagram size for this DatagramService. The maximum size indicates the size of datagram that the transport or bearer can convey to the destination as a single datagram. This may be a conservative estimate based on normal data and taking into account any overhead that may be needed for segmentation and reassembly.

Returns:

the number of bytes that can be sent and delivered as a single datagram at the destination by this DatagramService.

Throws:

java.io.IOException - or one of its subclasses is thrown if any error occurs getting the nominal length.

close

```
public abstract void close()  
                    throws java.io.IOException
```

Close the service. No more datagrams can be received or sent. Closing the service will force invocations of `receive` and `send` to return with an `IOException`. The `Address` can then be reopened using `getService(address)` by this or another application.

Throws:

`java.io.IOException` - or one of its subclasses is thrown if any error occurs during close.

getAddress

```
public abstract Address getAddress()  
                        throws java.io.IOException
```

Get the address supported by this `DatagramService`. This will be the source address assigned for the service.

Returns:

the `Address` for this service.

Throws:

`IntermittentNetworkException` - is thrown if an intermittent network problem has occurred. This error may be the result of a network or configuration error. It is up to the application to retry.

`java.io.IOException` - or one of its subclasses is thrown if any error occurs getting the address.

*javax.net.datagram***Class *DatagramServiceFactory***

```

java.lang.Object
|
+-- javax.net.datagram.DatagramServiceFactory

```

public abstract class **DatagramServiceFactory**
 extends java.lang.Object

DatagramServiceFactory is used to register and unregister DatagramServices that support network transports and bearers on the device. Each registered DatagramServiceFactory supports the decoding of address information that matches the DatagramService for which it is the factory.

Static methods in DatagramServiceFactory register and unregister the subclasses of DatagramServiceFactory that implement each network transport.

Abstract methods define the required methods of all DatagramServiceFactory subclasses. Each DatagramServiceFactory must be able to open DatagramServices for each Address type it supports.

DatagramServices are located by trying each factory in turn. If the factory supports the Address it returns the DatagramService and the application uses it to send and receive Datagrams to that address. Factories must be able to produce DatagramServices that act as servers for the specified address (see `openService`) or as clients of a service based on this address type (see `open`)

Implementations of DatagramServiceFactory must be implemented to be multi-thread safe and protect its data structures from concurrent modification.

See Also:

[DatagramService](#), [Address](#)

Method Summary	
static void	<u>addService</u>(<u>DatagramServiceFactory</u> factory) Register a factory for a DatagramService that can be used for sending and receiving datagrams on this transport or bearer.
static void	<u>removeService</u>(<u>DatagramServiceFactory</u> factory) Remove the factory of DatagramServices.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail*addService*

```
public static void addService(DatagramServiceFactory factory)
```

Register a factory for a DatagramService that can be used for sending and receiving datagrams on this transport or bearer. The service is added after the previously registered services. It will be searched after previously added services.

Parameters:

factory - to add to the list of registered factories.

removeService

```
public static void removeService(DatagramServiceFactory factory)
```

Remove the factory of DatagramServices. No new services can be opened from it. Any existing DatagramServices opened will remain open until they are closed by the application and will continue to receive and can be used to send datagrams. If the factory is not registered the call completes without an error.

Parameters:

factory - to remove from the list of registered factories.

Package javax.pim.addressbook

The AddressBook package defines the objects used to access, add, delete and update the contact information in a contact database.

See:

Description

Class Summary	
<u>ContactCard</u>	The ContactCard class is used to store a single entity's (person or place etc.) contact information.
<u>ContactDatabase</u>	The ContactDatabase models a single database of contact information.
<u>ContactGroup</u>	The ContactGroup class implements a group of ContactCards which themselves can also be groups.
<u>ContactTemplate</u>	The ContactTemplate class implements a template to be used when creating new contact cards.

Package javax.pim.addressbook Description

The AddressBook package defines the objects used to access, add, delete and update the contact information in a contact database.

An application using the AddressBook package locates and updates contact information such as phone numbers, postal addresses, email addresses, and other information about individuals, groups, and organizations. In GSM phones, for example, the AddressBook API provides access to name and address information stored on the SIM card.

The AddressBook package extends the database classes to hold ContactCards, ContactTemplates, and ContactGroups. Schema strings used in querying and storing contact information are those specified in vCard (see RFC 2426). (vCard personal data interchange format has been accepted as a standard method to transfer contact information.)

Package Features

The AddressBook API provides the following features:

- Supports multiple address books
- Allows multiple phone numbers for each contact card
- Allows more than one entry for a contact card such as additional addresses for a user
- Supports standard schema strings as specified in vCard personal data interchange format

Design Goals

The primary design goal of the AddressBook API is to be simple and thin so that it can fit in resource-limited devices. Because the AddressBook API is an extensible API that leverages a standard schema, application developers can easily find and use contact information for writing programs.

Contact information such as phone numbers can be retrieved from the ContactDatabase using the getPhoneNumbers method. The ContactCard class holds contact information and ContactTemplates can be used when new contact information is added to the contact database.

AddressBook Code Sample

If a person's work phone number needed to be accessed in an application using the AddressBook API, the name of the person, which is stored and implemented by the ContactCard class is queried and the respective work phone number is located using the getPreferredPhoneNumbers method, which filters the phone numbers stored in the contact database for the contact card, and returns only the preferred numbers. Sample code follows to further illustrate this concept.

```
/*
 * Copyright 1999 Sun Microsystems Inc, All rights reserved.
 */

import javax.pim.addressbook.*;
import javax.pim.database.*;

/**
 * This example program shows how to lookup the phone number for
 * one or more persons.
 */
public class ContactSample1 {

    ContactDatabase addressbook; // The open addressbook

    /**
     * The arguments to this program are the names of the people to
     * lookup. For each person, print the preferred telephone number.
     *
     * @param args the array of Strings containing each persons name.
     */
    public static void main(String[] args) {
        ContactSample1 sample = new ContactSample1();

        if (sample.open()) {
            for (int i = 0; i < args.length; i++) {
                sample.lookup(args[i]);
            }
            sample.close();
        }
    }

    /*
```

```

    * Lookup a name in the database and print the preferred voice phone
    * numbers available for it.
    *
    * @param name the name to look up
    */
void lookup(String name) {
    try {
        Iterator it = addressbook.items();
        while(it.hasNext()) {
            ContactCard card = (ContactCard)it.next();
            if (card.getName().indexOf(name) != -1) {
                String[] tel =
                    card.getPreferredPhoneNumbers(ContactDatabase.VOICE);
                for (int i = 0; i < tel.length; i++) {
                    System.out.println("Name: " + card.getName() +
                                      ": " + tel[i]);
                }
            }
        }
    } catch (DatabaseException ex) {
        System.out.println("Error getting contact information for " +
                           name + ": " +
                           ex.getMessage());
    }
}

/*
 * Open the database.
 *
 * @return true if open was successful
 */
boolean open() {
    try {
        // Open the contact database.
        addressbook = ContactDatabase.openDatabase();
    } catch (DatabaseException ex) {
        System.out.println("Unable to open ContactDatabase: " +
                           ex.getMessage());
        return false;
    }
    return true; // Addressbook opened successfully
}

/*
 * Close the contact database.
 *
 * @return true if close was successful
 */
boolean close() {
    try {
        // Close the contact database.
        if (addressbook != null )
            addressbook.close();
    } catch (DatabaseException ex) {
        System.out.println("Unable to close the ContactDatabase: " +
                           ex.getMessage());
        return false;
    }
}

```

```
        }  
        return true;  
    }  
}
```

Hierarchy For Package javax.pim.addressbook

Package Hierarchies:

All Packages

Class Hierarchy

- class java.lang.Object
 - class javax.pim.addressbook.**ContactDatabase** (implements javax.pim.database.**Database**)
 - class javax.pim.database.**Item**
 - class javax.pim.addressbook.**ContactCard**
 - class javax.pim.addressbook.**ContactGroup**
 - class javax.pim.addressbook.**ContactTemplate**
-
-

javax.pim.addressbook

Class ContactCard

```

java.lang.Object
|
+-- javax.pim.database.Item
    |
    +-- javax.pim.addressbook.ContactCard

```

Direct Known Subclasses:

ContactGroup, ContactTemplate

```

public class ContactCard
extends Item

```

The ContactCard class is used to store a single entity's (person or place etc.) contact information. Contact information is stored in a ContactCard's fields. A ContactCard can belong to zero or more ContactGroups.

See Also:

ContactTemplate, ContactGroup, Item, ItemField

Constructor Summary	
<u>ContactCard()</u>	
Creates a new empty ContactCard.	

Method Summary

<u>ContactGroup</u> []	<u>getGroups</u> () Returns an array of all the ContactGroups to which the contact card belongs or <code>null</code> if the card doesn't belong to any group.
java.lang.String	<u>getName</u> () This is a utility method for retrieving the value of this card's "FN" or "N" name fields.
java.lang.String	<u>getOrganization</u> () Returns the contents of an arbitrary "ORG" field of this contact card.
java.lang.String[]	<u>getPhoneNumbers</u> (java.lang.String type) This method retrieves values of this cards "TEL" telephony fields that have specified parameter with name "TYPE" and specified value.
java.lang.String[]	<u>getPreferredPhoneNumbers</u> (java.lang.String type) This method returns the preferred phone numbers.
void	<u>setFormattedName</u> (java.lang.String name) This method changes the "FN" field of this card.

Methods inherited from class javax.pim.database.Item

addField, getField, getFields, getFields, isSameItem, removeField, size

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

ContactCard

public **ContactCard**()

Creates a new empty ContactCard.

Method Detail

getGroups

```
public ContactGroup[] getGroups()
```

Returns an array of all the ContactGroups to which the contact card belongs or `null` if the card doesn't belong to any group.

Returns:

an array of ContactGroup objects to which this card belongs.

getName

```
public java.lang.String getName()
```

This is a utility method for retrieving the value of this card's "FN" or "N" name fields. If a card contains a "FN" field this method returns the content as a String. If the card doesn't have a "FN" field or the field has a `null` value, then existence of a "N" field is tested. If the card does not contain any "FN" or "N" fields, then `null` is returned.

When the "N" field is used this method converts the field to a single String and returns it. String conversion results in a String that has all the fields of this AggregateField concatenated to a single String (Family Part first, Name suffix last), the parts are separated with a space character.

If this card has several "FN" or "N" fields then the name returned is arbitrary.

This method should only be used to display the name of the entity this card represents.

Returns:

the name for this contact.

setFormattedName

```
public void setFormattedName(java.lang.String name)
```

This method changes the "FN" field of this card. If the card does not contain a "FN" field, then an "FN" field is added.

Parameters:

name - the name String

Throws:

NullPointerException - is thrown if name is `null`.

getPhoneNumbers

```
public java.lang.String[] getPhoneNumbers(java.lang.String type)
```

This method retrieves values of this cards "TEL" telephony fields that have specified parameter with name "TYPE" and specified value. If contact card doesn't have any matching "TEL" fields then an empty array is returned. This method takes into account the default parameter values of telephony fields. So every field that doesn't specify parameter "TYPE" any other contrarious parameter are assumed to be "VOICE" fields.

Parameters:

`type` - the type of phone number, selected from the types defined in the ContactDatabase documentation. For example, ContactDatabase.WORK.

Returns:

an array of Strings containing the phone numbers of the type requested.

getPreferredPhoneNumbers

```
public java.lang.String[] getPreferredPhoneNumbers(java.lang.String type)
```

This method returns the preferred phone numbers. If the card doesn't have preferred numbers of the specified type then the non-preferred phone number(s) of the specified type are returned.

Parameters:

`type` - the type of phone number, selected from the types defined in the ContactDatabase documentation. For example, ContactDatabase.WORK.

Returns:

an array of Strings containing the preferred phone numbers of the type requested.

See Also:

[`getPhoneNumbers\(java.lang.String\)`](#)

getOrganization

```
public java.lang.String getOrganization()
```

Returns the contents of an arbitrary "ORG" field of this contact card. Returns `null` if no organization field is set. Because "ORG" is an AggregateField, this method extracts the contents of each of the contained field's values and concatenated with a space (" ") character for a separator.

Returns:

the values of the "ORG" field concatenated together with " ".

javax.pim.addressbook **Class ContactDatabase**

```
java.lang.Object
|
+-- javax.pim.addressbook.ContactDatabase
```

public abstract class **ContactDatabase**
 extends java.lang.Object
 implements [Database](#)

The ContactDatabase models a single database of contact information. A ContactDatabase contains ContactCards, ContactTemplates, and ContactGroups. These objects contain fields which contain the actual data. Each object is an instance or an instances of subclasses of Database.Item class. ContactDatabase contains methods to traverse, add, delete and update the objects. A ContactDatabase can notify of changes made in database by sending events to listeners.

ContactDatabases are created and opened with static methods of this class. The default ContactDatabase can be opened with `openDatabase`.

The static methods delegate to the corresponding methods of the registered ContactDatabase factory. If the ContactDatabase factory is not set, the implementation may try to instantiate a class specified in system property `javax.pim.addressbook.factory` and set it as a factory.

Field names, field values, parameter names and parameter values of the AddressBook API are conceptually the same as those defined in the vCard specification (see RFC 2426). The vCard property names are used as naming conventions for AddressBook API's field names and parameter strings. So for example, vCard property name "N" has ContactCard equivalent static String field `ContactDatabase.N`.

The vCard specifications allows property parameter values as a parameter list. This is not the case in this API. If contact card must have several values of the same parameter, then the parameter must be stored as several Parameter objects. For example vCard property name "TEL" can have property parameter "TYPE". In textual vCard object the value for property parameter "TYPE" can be defined as a list (eg. TYPE=WORK,VOICE). In this API different TYPE parameters must be stored separately, ie. two parameter objects with same name "TYPE" but different values "WORK" and "VOICE" are stored as separate parameter objects.

In the table below are field name, parameter name and parameter value strings for Addressbook databases conforming to this API. The ContactDatabase class has static final fields of all these strings and field part values defined here. Aggregate fields have a sequence of values specified with part numbers. Fields are named similar to the vCard strings (eg. String "FN" is equivalent to field `ContactDatabase.FN`) except that if the string contains a dash character ('-') it is converted to an underscore ('_') in the field name. Attribute name and parameter strings are treated case sensitively to improve performance. It's preferred to write both field name and parameter strings in uppercase as in vCard specification. For example, "fn" and "FN" are different strings.

In the table below the default value strings for parameter named "TYPE" are in italic. So if the field has no parameters with the name "TYPE" set, then applications should treat the field as if it had those parameters marked as default. The default can be overridden with some other set of values by specifying one or more alternate values.

Field name, parameter name and parameter value strings can be extended with private names using the non-standard, private values mechanism defined in RFC 2045. Non-standard, private strings with a name starting "X-" may be used bilaterally between two cooperating agents.

Name	Field datatype	Allowed TYPE parameter values (defaults in italics)	Description
N	Aggregate (cont. 5 String fields)	-	Components of the name of the contact item. The structured type value corresponds, in sequence, to Family Name, Given Name, Additional Names, Honorific Prefixes, and Honorific Suffixes. Each component is an ItemField storing a String object.
FN	String	-	Formatted name, based on the semantics of the X.520 Common Name attribute.
NICKNAME	String	-	Specify the text corresponding to the nickname of the contact.
ORG	Aggregate (cont. 2 or more String fields)	-	Organization name and units. The first field of this aggregate field is an organization name field, the rest of the fields are organizational units (one or more).
CATEGORIES	String	-	To specify application category information about the contact.
TITLE	String	-	Job title, functional position or function of the object the contact represents etc.
ADR	Aggregate (cont. 7 String fields)	DOM, <i>INTL</i> , <i>POSTAL</i> , <i>PARCEL</i> , <i>WORK</i> , HOME, PREF	Structured delivery address.
LABEL	String	DOM, <i>INTL</i> , <i>POSTAL</i> , <i>PARCEL</i> , <i>WORK</i> , HOME, PREF	Formatted addressing label.
BDAY	Date	-	Birth date of the object the contact represents.

TEL	String	VOICE, PREF, WORK, HOME, FAX, MSG, CELL, PAGER, BBS, MODEM, CAR, ISDN, VIDEO, PCS	Specifies telephone number for telephony communication with the object the contact represents.
EMAIL	String	INTERNET, X400, PREF	Specifies electronic mail address for communication with the object the contact represents.
MAILER	String	-	Specifies the type of electronic mail software that is used by the individual associated with the contact. This field is based on the "X-Mailer" header field of internet mail messages.
TZ	String	-	To specify information related to the time zone of the object the contact represents.
GEO	String	-	To specify information related to the global positioning of the object the contact represents.
ROLE	String	-	Role, occupation or business category.
NOTE	String	-	To specify supplemental information or a comment that is associated with the contact.
URL	String	-	To specify a uniform resource locator associated with the object the contact refers to.
KEY	byte[]	IANA registered public key or authentication certificate format	To specify a public key or authentication certificate associated with the object that the contact represents.
LOGO	byte[]	-	To specify a graphic image of a logo associated with the object the contact represents.
PHOTO	byte[]	MIME graphic image format	Image or photograph information that annotates some aspect of the object the contact represents.
AGENT	byte[]	-	UID of this contact's agent.
REV	String	-	Revision information about the current contact.

SORT-STRING	String	-	To specify the family name or given name to be used for national-language-specific sorting of the FN and N types.
SOUND	byte[]	MIME audio format	Digital sound annotation.
UID	byte[]	-	Globally unique identifier as defined using the vCard specification rules for uids.
CLASS	String	-	An access classification.

In table below are defined the value strings for parameter name "TYPE":

TYPE parameter value	Description
DOM	ADR and LABEL field TYPE parameter value. Indicates domestic delivery label.
INTL	ADR and LABEL field TYPE parameter value. International delivery address.
POSTAL	ADR and LABEL field TYPE parameter value. Postal delivery address.
PARCEL	ADR and LABEL field TYPE parameter value. Parcel delivery address.
HOME	TEL, ADR and LABEL field TYPE parameter value. Indicates that field refers to a residence.
WORK	TEL, ADR and LABEL field TYPE parameter value. Indicates that field refers to a place of work.
PREF	TEL, EMAIL, ADR and LABEL field TYPE parameter value. Indicates that this field is preferred to be used when more than one field of this type is present in the contact item.
VOICE	TEL field TYPE parameter value. Indicates a voice telephone number.
FAX	TEL field TYPE parameter value. Indicates a facsimile telephone number.
MSG	TEL field TYPE parameter value. Indicates a messaging service on the telephone number.
CELL	TEL field TYPE parameter value. Indicates a cellular telephone number.
PAGER	TEL field TYPE parameter value. Indicates a paging device telephone number.
BBS	TEL field TYPE parameter value. Indicates a bulletin board system telephone number.
MODEM	TEL field TYPE parameter value. Indicates a MODEM connected telephone number.
CAR	TEL field TYPE parameter value. Indicates a car-phone telephone number.
ISDN	TEL field TYPE parameter value. Indicates an ISDN telephone number.
VIDEO	TEL field TYPE parameter value. Indicates a video conferencing telephone number.
PCS	TEL field TYPE parameter value. Indicates a personal communication services number (Digital cellular or GSM).
INTERNET	EMAIL field TYPE parameter value. Indicates an Internet addressing type.
X400	EMAIL field TYPE parameter value. Indicates a X.400 addressing type.

vCard specification separates "N" field (which is used to store name of an entity) to five parts. This field should be presented in the Addressbook API with AggregateField.

Here are static members for part numbers of N field:

Part	Field name
0	FAMILY_NAME
1	GIVEN_NAME
2	ADDITIONAL_NAMES
3	PREFIXES
4	SUFFIXES

ADR type field is separated to seven parts. Here are static members for part numbers of ADR field:

Part	Field name
0	PO_BOX
1	EXT_ADR
2	STREET_ADR
3	LOCALITY
4	REGION
5	POSTAL_CODE
6	COUNTRY

This class Addressbook models also SIM stored phone number information. SIM database consists also of ContactCards, each SIM stored contact card has three fields (see the table below).

Here is how SIM database should be mapped. These are static strings for querying SIM card slots:

Type	Allowed "TYPE" parameter values	Field datatype	Description
ContactCard.SIMSLOT	-	int	SIM slots number
ContactCard.FN	-	String	Name
ContactCard.TEL	-	String	Telephone number

Field Summary

static int	<u>ADDITIONAL NAMES</u> The index in the "N" field of the additional name.
static java.lang.String	<u>ADR</u> Structured delivery address.
static java.lang.String	<u>AGENT</u> UID this contact items agent.
static java.lang.String	<u>BBS</u> TEL field TYPE parameter value.
static java.lang.String	<u>BDAY</u> Birth date of the object the contact item represents.
static java.lang.String	<u>CAR</u> TEL field TYPE parameter value.
static java.lang.String	<u>CATEGORIES</u> To specify application category information about the item.
static java.lang.String	<u>CELL</u> TEL field TYPE parameter value.
static java.lang.String	<u>CLASS</u> An access classification.
static int	<u>COUNTRY</u> The index in the ADR field of the' country.
static java.lang.String	<u>DEFAULT_ADDRESSBOOK</u> String for retrieving address book.
static java.lang.String	<u>DEFAULT_SIM</u> String for retrieving default SIM address book.
static java.lang.String	<u>DOM</u> ADR and LABEL field TYPE parameter value.
static java.lang.String	<u>EMAIL</u> Specifies electronic mail address for communication with the object the contact item represents.

static int	<u>EXT_ADR</u> The index in the ADR field of the external address.
static int	<u>FAMILY_NAME</u> The index in the "N" field of the family name.
static java.lang.String	<u>FAX</u> TEL field TYPE parameter value.
static java.lang.String	<u>FN</u> Formatted name, based on semantics of the X.520 Common Name attribute.
static java.lang.String	<u>GEO</u> To specify information related to the global positioning of the object the contact item represents.
static int	<u>GIVEN_NAME</u> The index in the "N" field of the given name.
static java.lang.String	<u>HOME</u> TEL, ADR and LABEL field TYPE parameter value.
static java.lang.String	<u>INTERNET</u> EMAIL field TYPE parameter value.
static java.lang.String	<u>INTL</u> ADR and LABEL field TYPE parameter value.
static java.lang.String	<u>ISDN</u> TEL field TYPE parameter value.
static java.lang.String	<u>KEY</u> To specify public key or authentication certificate associated with the object that the contact item represents.
static java.lang.String	<u>LABEL</u> Formatted addressing label.
static int	<u>LOCALITY</u> The index in the ADR field of the locality.
static java.lang.String	<u>LOGO</u> To specify a graphic image of a logo associated with the object the contact item represents.
static java.lang.String	<u>MAILER</u> Specifies the type of electronic mail software that is used by the individual associated with the contact item.
static java.lang.String	<u>MODEM</u> TEL field TYPE parameter value.

static java.lang.String	<u>MSG</u> TEL field TYPE parameter value.
static java.lang.String	<u>N</u> Components of the name of the contact item.
static java.lang.String	<u>NICKNAME</u> Specify the text corresponding to the nickname of the item.
static java.lang.String	<u>NOTE</u> To specify supplemental information or a comment that is associated with the contact item.
static java.lang.String	<u>ORG</u> Organization name and units.
static java.lang.String	<u>PAGER</u> TEL field TYPE parameter value.
static java.lang.String	<u>PARCEL</u> ADR and LABEL field TYPE parameter value.
static java.lang.String	<u>PCS</u> TEL field TYPE parameter value.
static java.lang.String	<u>PHOTO</u> Image or photograph information that annotates some aspect of the object the contact item represents.
static int	<u>PO BOX</u> The index in the ADR field of the post office box.
static java.lang.String	<u>POSTAL</u> ADR and LABEL field TYPE parameter value.
static int	<u>POSTAL CODE</u> The index in the ADR field of the postal code.
static java.lang.String	<u>PREF</u> TEL, EMAIL, ADR and LABEL field TYPE parameter value.
static int	<u>PREFIXES</u> The index in the "N" field of the honorific prefixes.
static int	<u>REGION</u> The index in the ADR field of the region.
static java.lang.String	<u>REV</u> Revision information about the current contact item.
static java.lang.String	<u>ROLE</u> Role, occupation or business category.
static java.lang.String	<u>SIMSLLOT</u> This is the value string for slot numbers of SIM card contacts.

static java.lang.String	<u>SORT_STRING</u> To specify the family name or given name text to be used for national-language-specific sorting of the FN and N types.
static java.lang.String	<u>SOUND</u> Digital sound annotation.
static int	<u>STREET_ADR</u> The index in the ADR field of the street address.
static int	<u>SUFFIXES</u> The index in the "N" field of the honorific suffixes.
static java.lang.String	<u>TEL</u> Specifies telephone number for telephony communication with the object the contact item represents.
static java.lang.String	<u>TITLE</u> Job title, functional position or function of the object the contact item represents etc.
static java.lang.String	<u>TYPE</u> Parameter name TYPE String.
static java.lang.String	<u>TZ</u> To specify information related to the time zone of the object the contact item represents.
static java.lang.String	<u>UID</u> Globally unique identifier.
static java.lang.String	<u>URL</u> To specify a uniform resource locator associated with the object the contact item refers to.
static java.lang.String	<u>VIDEO</u> TEL field TYPE parameter value.
static java.lang.String	<u>VOICE</u> TEL field TYPE parameter value.
static java.lang.String	<u>WORK</u> TEL, ADR and LABEL field TYPE parameter value.
static java.lang.String	<u>X400</u> EMAIL field TYPE parameter value.

Method Summary	
abstract <u>Item</u>	<u>addItem</u> (<u>Item</u> item) Adds new item to the database.
abstract <u>Iterator</u>	<u>cardItems</u> () Returns iterator of ContactCard items.
static <u>ContactDatabase</u>	<u>createDatabase</u> (java.lang.String name) Create the specified ContactDatabase.
static void	<u>deleteDatabase</u> (java.lang.String name) Delete the specified ContactDatabase.
static <u>Iterator</u>	<u>getDatabaseNames</u> () Returns the names of available contact databases.
abstract <u>Iterator</u>	<u>groupItems</u> () Returns iterator of ContactGroup items.
static <u>ContactDatabase</u>	<u>openDatabase</u> () Opens the default ContactDatabase.
static <u>ContactDatabase</u>	<u>openDatabase</u> (java.lang.String name) Opens the specified ContactDatabase.
abstract <u>Iterator</u>	<u>templateItems</u> () Returns iterator of ContactTemplate items.
abstract void	<u>updateItem</u> (<u>Item</u> item) Updates item in this database.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

DEFAULT_ADDRESSBOOK

```
public static final java.lang.String DEFAULT_ADDRESSBOOK
```

String for retrieving address book.

DEFAULT_SIM

public static final java.lang.String **DEFAULT_SIM**

String for retrieving default SIM address book.

SIMSLLOT

public static final java.lang.String **SIMSLLOT**

This is the value string for slot numbers of SIM card contacts.

FAMILY_NAME

public static final int **FAMILY_NAME**

The index in the "N" field of the family name.

GIVEN_NAME

public static final int **GIVEN_NAME**

The index in the "N" field of the given name.

ADDITIONAL_NAMES

public static final int **ADDITIONAL_NAMES**

The index in the "N" field of the additional name.

PREFIXES

public static final int **PREFIXES**

The index in the "N" field of the honorific prefixes. part number

SUFFIXES

public static final int **SUFFIXES**

The index in the "N" field of the honorific suffixes.

PO_BOX

public static final int **PO_BOX**

The index in the ADR field of the post office box.

EXT_ADR

public static final int **EXT_ADR**

The index in the ADR field of the external address.

STREET_ADR

public static final int **STREET_ADR**

The index in the ADR field of the street address.

LOCALITY

public static final int **LOCALITY**

The index in the ADR field of the locality.

REGION

public static final int **REGION**

The index in the ADR field of the region.

POSTAL_CODE

public static final int **POSTAL_CODE**

The index in the ADR field of the postal code.

COUNTRY

```
public static final int COUNTRY
```

The index in the ADR field of the' country.

TYPE

```
public static final java.lang.String TYPE
```

Parameter name TYPE String.

N

```
public static final java.lang.String N
```

Components of the name of the contact item. The structured type value corresponds, in sequence, to Family Name, Given Name, Additional Names, Honorific Prefixes, and Honorific Suffixes. Each component is ItemField storing an String object.

Datatype of field: Aggregate (cont. 5 String fields).

Allowed TYPE parameter values: none.

FN

```
public static final java.lang.String FN
```

Formatted name, based on semantics of the X.520 Common Name attribute.

Datatype of field: String.

Allowed TYPE parameter values: none.

NICKNAME

```
public static final java.lang.String NICKNAME
```

Specify the text corresponding to the nickname of the item.

Datatype of field: String.

Allowed TYPE parameter values: none.

ORG

`public static final java.lang.String ORG`

Organization name and units. The first field of this aggregate field is an organizational name field, rest of the fields are organizational units (one or more).

Datatype of field: Aggregate (cont. 2 or more String fields).

Allowed TYPE parameter values: none.

CATEGORIES

`public static final java.lang.String CATEGORIES`

To specify application category information about the item.

Datatype of field: String.

Allowed TYPE parameter values: none.

TITLE

`public static final java.lang.String TITLE`

Job title, functional position or function of the object the contact item represents etc.

Datatype of field: String.

Allowed TYPE parameter values: none.

ADR

`public static final java.lang.String ADR`

Structured delivery address.

Datatype of field: Aggregate (cont. 7 String fields).

Allowed TYPE parameter values: DOM, *INTL*, *POSTAL*, *PARCEL*, *WORK*, HOME, PREF.

LABEL

`public static final java.lang.String LABEL`

Formatted addressing label.

Datatype of field: String.

Allowed TYPE parameter values: DOM, *INTL*, *POSTAL*, *PARCEL*, *WORK*, HOME, PREF.

BDAY

`public static final java.lang.String BDAY`

Birth date of the object the contact item represents.

Datatype of field: Date.

Allowed TYPE parameter values: none.

TEL

`public static final java.lang.String TEL`

Specifies telephone number for telephony communication with the object the contact item represents.

Datatype of field: String.

Allowed TYPE parameter values: *VOICE*, PREF, WORK, HOME, FAX, MSG, CELL, PAGER, BBS, MODEM, CAR, ISDN, VIDEO, PCS.

EMAIL

`public static final java.lang.String EMAIL`

Specifies electronic mail address for communication with the object the contact item represents.

Datatype of field: String.

Allowed TYPE parameter values: *INTERNET*, X400, PREF.

MAILER

`public static final java.lang.String MAILER`

Specifies the type of electronic mail software that is used by the individual associated with the contact item. This field is based on "X-Mailer" header field of internet mail messages.

Datatype of field: String.

Allowed TYPE parameter values: none.

TZ

`public static final java.lang.String TZ`

To specify information related to the time zone of the object the contact item represents.

Datatype of field: String.

Allowed TYPE parameter values: none.

GEO

`public static final java.lang.String GEO`

To specify information related to the global positioning of the object the contact item represents.

Datatype of field: String.

Allowed TYPE parameter values: none.

ROLE

`public static final java.lang.String ROLE`

Role, occupation or business category.

Datatype of field: String.

Allowed TYPE parameter values: none.

NOTE

`public static final java.lang.String NOTE`

To specify supplemental information or a comment that is associated with the contact item.

Datatype of field: String.

Allowed TYPE parameter values: none.

URL

`public static final java.lang.String URL`

To specify a uniform resource locator associated with the object the contact item refers to.

Datatype of field: String.

Allowed TYPE parameter values: none.

KEY

`public static final java.lang.String KEY`

To specify public key or authentication certificate associated with the object that the contact item represents.

Datatype of field: byte[].

Allowed TYPE parameter values: none.

LOGO

`public static final java.lang.String LOGO`

To specify a graphic image of a logo associated with the object the contact item represents.

Datatype of field: byte[].

Allowed TYPE parameter values: none.

PHOTO

`public static final java.lang.String PHOTO`

Image or photograph information that annotates some aspect of the object the contact item represents.

Datatype of field: `byte[]`.

Allowed TYPE parameter values: none.

AGENT

`public static final java.lang.String AGENT`

UID this contact items agent.

Datatype of field: `byte[]`.

Allowed TYPE parameter values: none.

REV

`public static final java.lang.String REV`

Revision information about the current contact item.

Datatype of field: `String`.

Allowed TYPE parameter values: none.

SORT_STRING

`public static final java.lang.String SORT_STRING`

To specify the family name or given name text to be used for national-language-specific sorting of the FN and N types.

Datatype of field: `String`.

Allowed TYPE parameter values: none.

SOUND

`public static final java.lang.String SOUND`

Digital sound annotation.

Datatype of field: `byte[]`.

Allowed TYPE parameter values: none.

UID

`public static final java.lang.String UID`

Globally unique identifier.

Datatype of field: `byte[]`.

Allowed TYPE parameter values: none.

CLASS

`public static final java.lang.String CLASS`

An access classification.

Datatype of field: `String`.

Allowed TYPE parameter values: none.

DOM

`public static final java.lang.String DOM`

ADR and LABEL field TYPE parameter value. Indicates domestic delivery label.

INTL

`public static final java.lang.String INTL`

ADR and LABEL field TYPE parameter value. International delivery address.

POSTAL

`public static final java.lang.String POSTAL`

ADR and LABEL field TYPE parameter value. Postal delivery address.

PARCEL

`public static final java.lang.String PARCEL`

ADR and LABEL field TYPE parameter value. Parcel delivery address.

HOME

`public static final java.lang.String HOME`

TEL, ADR and LABEL field TYPE parameter value. Indicates that field refers to a residence.

WORK

`public static final java.lang.String WORK`

TEL, ADR and LABEL field TYPE parameter value. Indicates that field refers to a place of work.

PREF

`public static final java.lang.String PREF`

TEL, EMAIL, ADR and LABEL field TYPE parameter value. Indicates that this field is preferred to be used when more than one field of this type is present in the contact item.

VOICE

`public static final java.lang.String VOICE`

TEL field TYPE parameter value. Indicates a voice telephone number.

FAX

`public static final java.lang.String FAX`

TEL field TYPE parameter value. Indicates a facsimile telephone number.

MSG

`public static final java.lang.String MSG`

TEL field TYPE parameter value. Indicates a messaging service on the telephone number.

CELL

`public static final java.lang.String CELL`

TEL field TYPE parameter value. Indicates a cellular telephone number.

PAGER

`public static final java.lang.String PAGER`

TEL field TYPE parameter value. Indicates a paging device telephone number.

BBS

`public static final java.lang.String BBS`

TEL field TYPE parameter value. Indicates a bulletin board system telephone number.

MODEM

`public static final java.lang.String MODEM`

TEL field TYPE parameter value. Indicates a MODEM connected telephone number.

CAR

`public static final java.lang.String CAR`

TEL field TYPE parameter value. Indicates a car-phone telephone number.

ISDN

public static final java.lang.String **ISDN**

TEL field TYPE parameter value. Indicates an ISDN telephone number.

VIDEO

public static final java.lang.String **VIDEO**

TEL field TYPE parameter value. Indicates a video conferencing telephone number.

PCS

public static final java.lang.String **PCS**

TEL field TYPE parameter value. Indicates a personal communication services number (Digital cellular or GSM).

INTERNET

public static final java.lang.String **INTERNET**

EMAIL field TYPE parameter value. Indicates an Internet addressing type.

X400

public static final java.lang.String **X400**

EMAIL field TYPE parameter value. Indicates a X.400 addressing type.

Method Detail

openDatabase

public static final [ContactDatabase](#) **openDatabase**(java.lang.String name)
throws [DatabaseException](#)

Opens the specified ContactDatabase. The predefined databases are named "DEFAULT_ADDRESSBOOK" and "DEFAULT_SIM" to provide easy access to the default contact information on the device or system.

Parameters:

name - of the ContactDatabase to open.

Throws:

DatabaseException - thrown on error for example if database doesn't exist or factory has not been set.

java.lang.SecurityException - if opening the database isn't allowed.

openDatabase

```
public static final ContactDatabase openDatabase()  
                                throws DatabaseException
```

Opens the default ContactDatabase.

Throws:

DatabaseException - thrown on error for example if database doesn't exist or factory has not been set.

java.lang.SecurityException - if opening the database isn't allowed

deleteDatabase

```
public static final void deleteDatabase(java.lang.String name)  
                                throws DatabaseException
```

Delete the specified ContactDatabase.

Parameters:

name - of the database to delete. It may not be null.

Throws:

DatabaseException - thrown on error for example if database doesn't exist or factory has not been set.

java.lang.SecurityException - if deleting the database isn't allowed.

createDatabase

```
public static final ContactDatabase createDatabase(java.lang.String name)  
                                throws DatabaseException
```

Create the specified ContactDatabase.

Parameters:

name - of the database to create. It may not be null.

Throws:

DatabaseException - is thrown on error for example if database already exists or the factory has not been set.

java.lang.SecurityException - is thrown if creating the database isn't allowed.

getDatabaseNames

```
public static Iterator getDatabaseNames()  
                                throws DatabaseException
```

Returns the names of available contact databases. Application may or may not have access to all these databases. If no contact databases are stored this method returns an empty Iterator.

Returns:

name of this database

Throws:

DatabaseException - is thrown if there was an error getting the names of databases.

java.lang.SecurityException - is thrown if fetching the names isn't allowed.

cardItems

```
public abstract Iterator cardItems()  
                                throws DatabaseException
```

Returns iterator of ContactCard items. Application may encounter exception in this method call or during traverse of items of iterator. This method does not return ContactTemplate or ContactGroup objects. If no items are stored this method returns an empty Iterator.

Returns:

Items

Throws:

DatabaseException - is thrown if there was an error retrieving results

java.lang.SecurityException - is thrown if retrieving the items isn't allowed

groupItems

```
public abstract Iterator groupItems()  
                                throws DatabaseException
```

Returns iterator of ContactGroup items. Application may encounter exception in this method call or during traverse of items of iterator. If no items are stored this method returns an empty Iterator.

Returns:

Items

Throws:

DatabaseException - is thrown if there was an error retrieving results

SecurityException - is thrown if retrieving the items isn't allowed

templateItems

```
public abstract Iterator templateItems()  
                                throws DatabaseException
```

Returns iterator of ContactTemplate items. Application may encounter exception in this method call or during traverse of items of iteration. If templates are not supported this method throws UnsupportedOperationException. If no templates are stored this method returns an empty Iterator.

Returns:

Items

Throws:

DatabaseException - is thrown if there was an error retrieving results

UnsupportedOperationException - if templates are not supported

java.lang.SecurityException - is thrown if retrieving the items isn't allowed

addItem

```
public abstract Item addItem(Item item)  
                                throws DatabaseException
```

Adds new item to the database. This method adds an item to the database. The item stored in the database may be a copy of specified item object. If the specified item already exists then item is duplicated. This method returns the added item, further changes must be done to this item. The returned item must also be used when updating changes to the database using update(Item item) method. Returned item and the item specified as parameter may be different objects.

When databases addItem or updateItem is invoked to update the ContactGroup item not already in the database the methods will also add contact cards not added to the database yet.

The addItem method can throw a IllegalArgumentException if an attempt is made to insert a subclass of an item when there is no support for extended addressbook items.

Specified by:

addItem in interface Database

Parameters:

item - item to add to database

Returns:

the added item, make further changes to this object

Throws:

java.lang.SecurityException - is thrown if adding the item isn't allowed

See Also:

Database.addItem(javax.pim.database.Item)

updateItem

```
public abstract void updateItem(Item item)  
                                throws DatabaseException
```

Updates item in this database. If this item has not yet been added to the database `DatabaseUpdateException` is thrown. Because the item stored in the database may be a copy of item object the items fetched from the database and then changed must be updated to the database using this method.

Use `addItem(Item item)` to add newly created item to the database.

When databases `addItem` or `updateItem` is invoked to update the `ContactGroup` item not already in the database the methods will also add contact cards not added to the database yet.

Update does update all the fields including aggregate fields and field parameters to the database.

Specified by:

updateItem in interface Database

Parameters:

`item` - item to be updated

Throws:

`java.lang.SecurityException` - is thrown if updating the items isn't allowed

See Also:

`Database.updateItem(javax.pim.database.Item)`

javax.pim.addressbook

Class ContactGroup

```

java.lang.Object
|
+-- javax.pim.database.Item
    |
    +-- javax.pim.addressbook.ContactCard
        |
        +-- javax.pim.addressbook.ContactGroup

```

```

public class ContactGroup
extends ContactCard

```

The ContactGroup class implements a group of ContactCards which themselves can also be groups. A single ContactCard can be a member of multiple groups. The implementation must retain the integrity of this many to many relationship. For example, when a ContactCard is removed from the database, it must be removed from the ContactGroups to which it belongs.

Constructor Summary

ContactGroup()

Creates a new empty ContactGroup.

Method Summary

void	<u>addContact</u> (<u>ContactCard</u> card) Adds a single ContactCard to group.
int	<u>contactCount</u> () Returns the number of ContactCards in this ContactGroup.
<u>ContactCard</u> []	<u>getContacts</u> () Returns the ContactCards of this ContactGroup in an array.
void	<u>removeContact</u> (<u>ContactCard</u> card) Removes a single ContactCard from the group.

Methods inherited from class javax.pim.addressbook.ContactCard

getGroups, getName, getOrganization, getPhoneNumbers,
getPreferredPhoneNumbers, setFormattedName

Methods inherited from class javax.pim.database.Item

addField, getField, getFields, getFields, isSameItem, removeField, size

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail*ContactGroup*

public **ContactGroup**()

Creates a new empty ContactGroup.

Method Detail*addContact*

public void **addContact**(ContactCard card)

Adds a single ContactCard to group.

When the addItem or updateItem method is invoked on a database with this group the database will be updated to reflect this group and its members. Members of this group that were not present in the database will be added. Changes made to individual members already in the database will not be updated in the database. They will need to be updated separately.

Duplicate items in the group are not permitted and are ignored.

Parameters:

card - a ContactCard to add to the group.

removeContact

```
public void removeContact(ContactCard card)
```

Removes a single ContactCard from the group. If the card is not a member of the group the attempt to remove it is silently ignored.

Parameters:

card - a ContactCard to remove from the group.

contactCount

```
public int contactCount()
```

Returns the number of ContactCards in this ContactGroup.

Returns:

the number of ContactCards this group contains.

getContacts

```
public ContactCard[] getContacts()
```

Returns the ContactCards of this ContactGroup in an array. If the ContactGroup does not contain any ContactCards, `null` is returned.

Returns:

an array of the ContactCards of this ContactGroup.

javax.pim.addressbook

Class ContactTemplate

```

java.lang.Object
|
+-- javax.pim.database.Item
    |
    +-- javax.pim.addressbook.ContactCard
        |
        +-- javax.pim.addressbook.ContactTemplate

```

```

public class ContactTemplate
extends ContactCard

```

The ContactTemplate class implements a template to be used when creating new contact cards. A template contact item defines a set of default fields (possibly with default values), and is used to seed a new contact card on creation. After a new contact card has been created, it retains no link to its template. Future changes to the template will not be reflected in any contact cards created from it prior to the change.

ContactTemplates provide a way for applications to know what fields exist in normal ContactCards stored in a specific contact database.

Constructor Summary

<u>ContactTemplate</u> ()	
Creates a new empty contact template.	

Method Summary

<u>ContactCard</u>	<u>newContactCard</u> ()
Creates a new ContactCard from the ContactTemplate.	

Methods inherited from class javax.pim.addressbook.ContactCard

getGroups, getName, getOrganization, getPhoneNumbers,
getPreferredPhoneNumbers, setFormattedName

Methods inherited from class javax.pim.database.Item

addField, getField, getFields, getFields, isSameItem, removeField, size

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail*ContactTemplate*

public **ContactTemplate**()

Creates a new empty contact template.

Method Detail*newContactCard*

public ContactCard **newContactCard**()

Creates a new ContactCard from the ContactTemplate. All fields of specified template are copied to this new ContactCard.

Package javax.pim.calendar

The Calendar package defines objects to access, add, delete and update the calendar event information in a Calendar database.

See:

Description

Class Summary	
<u>CalendarDatabase</u>	The CalendarDatabase models a database of calendar information.
<u>CalendarEntry</u>	The CalendarEntry class implements a calendar item.
<u>CalendarToDo</u>	The CalendarToDo class models "To do" items which are used to store tasks to be done.
<u>Repeat</u>	Repeat defines the sequence of dates on which a repeating event occurs.
<u>RepeatRule</u>	Describes a recurring sequence based on the iCalendar recurrence rules.

Package javax.pim.calendar Description

The Calendar package defines objects to access, add, delete and update the calendar event information in a Calendar database.

Applications have access to schedule information using the Calendar API, such as dates and task entries using the CalendarToDo and CalendarEntry classes. CalendarEntries provide access to descriptions, dates, and repeat information. ToDo entries provide access to notes and items that represent tasks that need to be accomplished by certain due dates. Scheduling data stored in a device implemented with the Calendar API is accessible to a user who can easily locate dates and times or make modifications to this information.

Design Goals

The primary design goal of the Calendar API is to be simple and thin so that it can fit in resource-limited devices. The API specifies a base storage class for all types of items including calendar entries and todo entries. Schema strings used in querying and storing contact information are those specified in iCalendar (RFC2445). (iCalendar interchange format has been accepted as a standard way to transfer scheduling information.)

Calendar information such as CalendarEntry and CalendarToDo can be retrieved from the CalendarDatabase using the `entryItems()` and `todoItems()` methods. The CalendarDatabase implements the Database interface. CalendarEntry implements calendar items that contain appointment and diary information. Using the `getStartDate()` method, this information can be retrieved for a

specific date so that an application can display all the calendar entries for today's date, for example, as illustrated in the following sample code.

Calendar Code Sample

```

/*
 * Copyright 1999 Sun Microsystems Inc, All rights reserved.
 */

import java.util.Calendar;
import java.util.Date;

import javax.pim.calendar.*;
import javax.pim.database.*;

/**
 * This example program shows how to lookup calendar entries for
 * the current day.
 */
public class CalendarSample1 {

    CalendarDatabase calendar; // The open calendar

    /**
     * Print the calendar entries for today.
     */
    public static void main(String[] args) {
        CalendarSample1 sample = new CalendarSample1();

        if (sample.open()) {

            sample.list(new Date());

            sample.close();
        }
    }

    /**
     * Look up the entries for the date.
     */
    void list(Date date) {
        Calendar when = Calendar.getInstance();

        /* Determine the desired year and day of year
         * to compare with year and day of each event.
         * The Calendar class will correctly handle timezones, etc.
         */
        when.setTime(date);
        int year = when.get(Calendar.YEAR);
        int dayofyear = when.get(Calendar.DAY_OF_YEAR);

        try {
            Iterator it = calendar.entryItems();
            while(it.hasNext()) {
                /**

```

```

        * Check each entry to see if it starts on the same day.
        */
        CalendarEntry entry = (CalendarEntry)it.next();
        Date startdate = entry.getStartDate();

        /* From the event get the day of year and compare.
        * If equal then get year and compare.
        */
        when.setTime(startdate);
        if ((dayofyear == when.get(Calendar.DAY_OF_YEAR)) &&
            year == when.get(Calendar.YEAR)) {

            String desc = entry.getDescription();
            System.out.println("Date: " + startdate +
                               ", " + desc);
        }
    }
} catch (DatabaseException ex) {
    System.out.println("Error getting calendar information: " +
                       ex.getMessage());
}

}

/*
 * Open the calendar database.
 *
 * @return true if open was successful
 */
boolean open() {
    try {
        calendar = CalendarDatabase.openDatabase();
    } catch (DatabaseException ex) {
        System.out.println("Unable to open CalendarDatabase: " +
                           ex.getMessage());
        return false;
    }
    return true; // Calendar opened successfully
}

/*
 * Close the calendar database.
 *
 * @return true if close was successful
 */
boolean close() {
    try {
        if (calendar != null )
            calendar.close();
    } catch (DatabaseException ex) {
        System.out.println("Unable to close the CalendarDatabase: " +
                           ex.getMessage());
        return false;
    }
}

```

```
        return true;
    }
}
```

Hierarchy For Package javax.pim.calendar

Package Hierarchies:

All Packages

Class Hierarchy

- class java.lang.Object
 - class javax.pim.calendar.**CalendarDatabase** (implements javax.pim.database.**Database**)
 - class javax.pim.database.**Item**
 - class javax.pim.calendar.**CalendarEntry**
 - class javax.pim.calendar.**CalendarToDo**
 - class javax.pim.calendar.**Repeat**
 - class javax.pim.calendar.**RepeatRule**
-
-

javax.pim.calendar **Class CalendarDatabase**

```
java.lang.Object
|
+-- javax.pim.calendar.CalendarDatabase
```

public abstract class **CalendarDatabase**
extends java.lang.Object
implements [Database](#)

The CalendarDatabase models a database of calendar information. CalendarDatabase items are subclasses of CalendarEntry or CalendarToDo classes. CalendarDatabase contains methods to find, add, delete and update the items. A CalendarDatabase can notify of changes made in database by sending events to listeners.

Static methods of CalendarDatabase are used create, open, delete, and get the names of CalendarDatabases. The default Calendar database can be opened with the `openDatabase()` method. The static methods delegate to the corresponding methods of the registered ContactDatabase factory. If a factory has not set, the implementation may try to instantiate a class specified in system property `javax.pim.calendar.factory` and set it as the factory.

Field names and field parameter values strings of Calendar API are conceptually the same than those defined in iCalendar specification ([RFC 2445](#)). The iCalendar property names are used as naming convention of Calendar API's field name and parameter name and value strings. CalendarDatabase has equivalent static String fields for most of the Strings used in the iCalendar specification. So for example iCalendar property name "ATTENDEE" has the equivalent static String field `CalendarDatabase.ATTENDEE`.

Below are the schema strings for access ing Calendar databases conforming to this API. The CalendarDatabase class has static final fields for these strings and part numbers. Fields are named similarly to iCalendar ("DTEND" is equivalent to `CalendarDatabase.DTEND`) except that if string contains a dash charcter ('-') it is converted to a underscore ('_') in the field name. Attribute name and parameter strings are treated case sensitively to improve performance. It's preferred to write both field name and parameter strings in uppercase as in the iCalendar specification

Both field name and parameter strings can be extended with private names using the non-standard, private values mechanism defined in [RFC 2045](#). Non-standard, private types with a name starting "X-" may be defined bilaterally between two cooperating agents.

In the table below are field name strings:

Name	Field datatype	Allowed parameters	Description
------	----------------	--------------------	-------------

ATTACH	String	ENCODING, FMTYPE, VALUE	Associates an object to a calendar item. By default contains URI to an attachment
CATEGORIES	String	LANGUAGE	Categories for a calendar component. For example APPOINTMENT or MEETING.
CLASS	String	-	Access classification for a calendar item. Possible values can be for example PUBLIC, PRIVATE, CONFIDENTIAL
COMMENT	String	ALTREP, LANGUAGE	Non-processing information intended to provide a comment to the calendar user.
DESCRIPTION	String	ALTREP, LANGUAGE	More complete description of the calendar item, than provided by the SUMMARY field.
GEO	Aggregate (contains 2 double fields)	-	Global position of the activity specified by the item.
LOCATION	String	ALTREP, LANGUAGE	Intended venue for the activity defined by the calendar item.
PERCENT-COMPLETE	int	-	Only allowed in: CalendarToDo. Used by an assignee or delegatee of a to-do to convey the percent completion of a to-do to the Organizer.
PRIORITY	int	-	Relative priority for a calendar item. Value is an integer in the range zero to nine. Zero defines undefined priority. Value one is highest priority.
RESOURCES	String	ALTREP, LANGUAGE	Equipment or resources anticipated for an activity specified by a calendar item.

STATUS	String	-	Overall status or confirmation for the calendar item.
SUMMARY	String	ALTREP, LANGUAGE	Short summary of the subject for the calendar item.
COMPLETED	Date	-	Only allowed in: CalendarToDo. Data and time that a to-do was actually completed. Note: This String can be used also as a STATUS field value and PARTSTAT parameter value. Used only in CalendarToDo.
DTEND	Date	TZID	Only allowed in: CalendarEntry. Date and time calendar entry ends.
DUE	Date	TZID	Allowed in: CalendarToDo. Date and time calendar to-do is expected to be completed.
DTSTART	Date	TZID	Specifies when a calendar item begins.
DURATION	Date	-	A positive duration of time.
TRANSP	String	-	Allowed in: CalendarEvent. Specifies if an entry is transparent or not to busy time searches.
ATTENDEE	String	LANGUAGE, CUTYPE, MEMBER, ROLE, PARTSTAT, RSVP, DELEGATED-FROM, DELEGATED-TO, SENT-BY, CN, DIR	Defines an attendee within a calendar item.
CONTACT	String	ALTREP, LANGUAGE	Represents contact information or alternately a reference to contact information associated with the calendar item.
ORGANIZER	String	LANGUAGE, CN, DIR, SENT-BY	Organizer for a calendar item.

RELATED-TO	String	-	Represents a relationship or reference between one calendar item and another.
URL	String	-	Specifies a URL associated with the calendar item.
UID	String	-	Defines persistent, globally unique identifier for the calendar item.
CREATED	Date	-	Date and time calendar item was created.
LAST-MODIFIED	Date	-	The date and time calendar item was last revised.
SEQUENCE	int	-	Defines the revision sequence number of the calendar item within a sequence of revisions.
REQUEST-STATUS	String	-	Defines the status code returned for a scheduling request.

Below are field parameter names and their values conforming to this API.

Parameter name	Parameter value	Description
ALTREP	Any String	To specify an alternate text representation for the field value.
CN	Any String	Common name to be associated with the calendar user specified by the field.
CUTYPE	INDIVIDUAL, GROUP, RESOURCE, ROOM, UNKNOWN	Type of calendar user specified by the field.
DELEGATED-FROM	MAILTO URI	Calendar users that have delegated their participation to the calendar user specified by the field.
DELEGATED-TO	MAILTO URI	Calendar users to whom the calendar user specified by the field has delegated participation.

DIR	URI	Reference to a directory entry associated with the calendar user specified by the field.
ENCODING	To specify an alternate inline encoding for the property value. One of 8Bit or BASE64 or IANA registered iCalendar encoding type or a non-standard content encoding type.	
FMTYPE	IANA registered content type or a non-standard content type String	Content type of a referenced object.
LANGUAGE	String identifying a language defined in RFC 1766	Language for text values in a field or field parameter.
MEMBER	MAILTO URI	Group or list membership of the calendar user specified by the field.
PARTSTAT	CalendarEvent: NEEDS-ACTION, ACCEPTED, DECLINED, TENTATIVE, DELEGATED. CalendarToDo: NEEDS-ACTION, ACCEPTED, DECLINED, TENTATIVE, DELEGATED, COMPLETED, IN-PROCESS	The participation status for the calendar user specified by the field.
RELTYPE	PARENT, CHILD, SIBLING	The type of hierarchical relationship associated with the calendar item specified by the field.
ROLE	CHAIR, REQ-PARTICIPANT, OPT-PARTICIPANT, NON-PARTICIPANT	The participation role for the calendar user specified by the field.
RSVP	TRUE, FALSE	To specify whether there is an expectation of a favor of a reply from the calendar user specified by the field value.
TZID	A string	To specify the identifier for the time zone definition for a time component in the property value.

SENT-BY	MAILTO URI	The calendar user that is acting on behalf of the calendar user specified by the property.
VALUE	BINARY, TEXT, INTEGER, FLOAT, URI	Explicitly specify the data type format for a field value. Some fields allow the change of data type.

Field Summary	
static java.lang.String	<u>ACCEPTED</u> PARTSTAT parameter value.
static java.lang.String	<u>ALTREP</u> To specify an alternate text representation for the field value.
static java.lang.String	<u>ATTACH</u> Associates an object with a calendar item.
static java.lang.String	<u>ATTENDEE</u> Defines an attendee within a calendar item.
static java.lang.String	<u>BINARY</u> VALUE parameter value.
static java.lang.String	<u>CANCELLED</u> STATUS field value.
static java.lang.String	<u>CATEGORIES</u> Categories for a calendar component.
static java.lang.String	<u>CHAIR</u> ROLE parameter value.
static java.lang.String	<u>CHILD</u> RELTYPE parameter value.
static java.lang.String	<u>CLASS</u> Access classification for a calendar item.
static java.lang.String	<u>CN</u> Common name to be associated with the calendar user specified by the field.
static java.lang.String	<u>COMMENT</u> Non-processing information intended to provide a comment to the calendar user.

static java.lang.String	<u>COMPLETED</u> Only allowed in: CalendarToDo.
static java.lang.String	<u>CONFIDENTIAL</u> CLASS field value.
static java.lang.String	<u>CONFIRMED</u> STATUS field value.
static java.lang.String	<u>CONTACT</u> Represents contact information or alternately a reference to contact information associated with the calendar item.
static java.lang.String	<u>CREATED</u> Date and time calendar item was created.
static java.lang.String	<u>CUTYPE</u> Type of calendar user specified by the field.
static java.lang.String	<u>DECLINED</u> PARTSTAT parameter value.
static java.lang.String	<u>DEFAULT CALENDAR</u> The name of the default calendar.
static java.lang.String	<u>DELEGATED</u> PARTSTAT parameter value.
static java.lang.String	<u>DELEGATED FROM</u> Calendar users that have delegated their participation to the calendar user specified by the field.
static java.lang.String	<u>DELEGATED TO</u> Calendar users to whom the calendar user specified by the field has delegated participation.
static java.lang.String	<u>DESCRIPTION</u> More complete description of the calendar item than provided by the SUMMARY field.
static java.lang.String	<u>DIR</u> Reference to a directory entry associated with the calendar user specified by the field.
static java.lang.String	<u>DTEND</u> Only allowed in: CalendarEntry.
static java.lang.String	<u>DTSTART</u> Specifies when a calendar item begins.
static java.lang.String	<u>DUE</u> Allowed in: CalendarToDo.

static java.lang.String	<u>DURATION</u> A positive duration of time.
static java.lang.String	<u>ENCODING</u> To specify an alternate inline encoding for the property value.
static java.lang.String	<u>FALSE</u> RSVP parameter value.
static java.lang.String	<u>FLOAT</u> VALUE parameter value.
static java.lang.String	<u>FMTYPE</u> Content type of a referenced object.
static java.lang.String	<u>GEO</u> Global position of the activity specified by the item.
static java.lang.String	<u>GROUP</u> CUTYPE parameter value.
static java.lang.String	<u>IN PROCESS</u> STATUS field value and PARTSTAT parameter value.
static java.lang.String	<u>INDIVIDUAL</u> CUTYPE parameter value.
static java.lang.String	<u>INTEGER</u> VALUE parameter value.
static java.lang.String	<u>LANGUAGE</u> Language for text values in a field or field parameter.
static java.lang.String	<u>LAST_MODIFIED</u> The date and time calendar item was last revised.
static java.lang.String	<u>LOCATION</u> Intended venue for the activity defined by the calendar item.
static java.lang.String	<u>MEMBER</u> Group or list membership of the calendar user specified by the field.
static java.lang.String	<u>NEEDS ACTION</u> STATUS field value and PARTSTAT parameter value.
static java.lang.String	<u>NON PARTICIPANT</u> ROLE parameter value.
static java.lang.String	<u>OPAQUE</u> TRANSP field value.
static java.lang.String	<u>OPT PARTICIPANT</u> ROLE parameter value.

static java.lang.String	<u>ORGANIZER</u> Organizer for a calendar item.
static java.lang.String	<u>PARENT</u> RELTYPE parameter value.
static java.lang.String	<u>PARTSTAT</u> The participation status for the calendar user specified by the field.
static java.lang.String	<u>PERCENT_COMPLETE</u> Only allowed in: CalendarToDo.
static java.lang.String	<u>PRIORITY</u> Relative priority for a calendar item.
static java.lang.String	<u>PRIVATE</u> CLASS field value.
static java.lang.String	<u>PUBLIC</u> CLASS field value.
static java.lang.String	<u>RELATED_TO</u> Represents a relationship or reference between one calendar item and another.
static java.lang.String	<u>RELTYPE</u> The type of hierarchical relationship associated with the calendar item specified by the field.
static java.lang.String	<u>REQ PARTICIPANT</u> ROLE parameter value.
static java.lang.String	<u>REQUEST STATUS</u> Defines the status code returned for a scheduling request.
static java.lang.String	<u>RESOURCE</u> CUTYPE parameter value.
static java.lang.String	<u>RESOURCES</u> Equipment or resources anticipated for an activity specified by a calendar item.
static java.lang.String	<u>ROLE</u> The participation role for the calendar user specified by the field.
static java.lang.String	<u>ROOM</u> CUTYPE parameter value.
static java.lang.String	<u>RSVP</u> To specify whether there is an expectation of a favor of a reply from the calendar user specified by the field value.

static java.lang.String	<u>SENT_BY</u> The calendar user that is acting on behalf of the calendar user specified by the property.
static java.lang.String	<u>SEQUENCE</u> Defines the revision sequence number of the calendar item within a sequence of revisions.
static java.lang.String	<u>SIBLING</u> RELTYPE parameter value.
static java.lang.String	<u>STATUS</u> Overall status or confirmation for the calendar item.
static java.lang.String	<u>SUMMARY</u> Short summary of the subject for the calendar item.
static java.lang.String	<u>TENTATIVE</u> STATUS field value and PARTSTAT parameter value.
static java.lang.String	<u>TEXT</u> VALUE parameter value.
static java.lang.String	<u>TRANSP</u> Allowed in: CalendarEvent.
static java.lang.String	<u>TRANSPARENT</u> TRANSP field value.
static java.lang.String	<u>TRUE</u> RSVP parameter value.
static java.lang.String	<u>TZID</u> To specify the identifier for the time zone definition for a time component in the property value.
static java.lang.String	<u>UID</u> Defines persistent, globally unique identifier for the calendar item.
static java.lang.String	<u>UNKNOWN</u> CUTYPE parameter value.
static java.lang.String	<u>URI</u> VALUE parameter value.
static java.lang.String	<u>URL</u> Specifies a URL associated with the calendar item.
static java.lang.String	<u>VALUE</u> Explicitly specify the data type format for a field value.

Method Summary

abstract <u>Item</u>	<u>addItem</u> (<u>Item</u> item) Adds new item to the database.
static <u>CalendarDatabase</u>	<u>createDatabase</u> (java.lang.String name) Create the specified CalendarDatabase.
static void	<u>deleteDatabase</u> (java.lang.String name) Delete the specified CalendarDatabase.
abstract <u>Iterator</u>	<u>entryItems</u> () Returns an iterator of CalendarEntry items.
static <u>Iterator</u>	<u>getDatabaseNames</u> () Returns the names of available calendar databases.
abstract <u>Iterator</u>	<u>items</u> (java.util.Date startDate, java.util.Date endDate) Returns an iterator of CalendarEntry items within the date range specified.
static <u>CalendarDatabase</u>	<u>openDatabase</u> () Opens the default CalendarDatabase.
static <u>CalendarDatabase</u>	<u>openDatabase</u> (java.lang.String name) Opens the CalendarDatabase with the specified name.
abstract <u>Iterator</u>	<u>todoItems</u> () Returns an iterator of CalendarToDo items.
abstract void	<u>updateItem</u> (<u>Item</u> item) Updates item in this database.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

DEFAULT_CALENDAR

public static final java.lang.String **DEFAULT_CALENDAR**

The name of the default calendar.

ATTACH

public static final java.lang.String **ATTACH**

Associates an object with a calendar item. By default contains URI to an attachment

CATEGORIES

public static final java.lang.String **CATEGORIES**

Categories for a calendar component. For example APPOINTMENT or MEETING.

CLASS

public static final java.lang.String **CLASS**

Access classification for a calendar item. Possible values can be for example PUBLIC, PRIVATE, CONFIDENTIAL

COMMENT

public static final java.lang.String **COMMENT**

Non-processing information intended to provide a comment to the calendar user.

DESCRIPTION

public static final java.lang.String **DESCRIPTION**

More complete description of the calendar item than provided by the SUMMARY field.

GEO

public static final java.lang.String **GEO**

Global position of the activity specified by the item.

LOCATION

```
public static final java.lang.String LOCATION
```

Intended venue for the activity defined by the calendar item.

PERCENT_COMPLETE

```
public static final java.lang.String PERCENT_COMPLETE
```

Only allowed in: CalendarToDo. Used by an assignee or delegatee of a to-do to convey the percent completion of a to-do to the Organizer.

PRIORITY

```
public static final java.lang.String PRIORITY
```

Relative priority for a calendar item. Value is an integer in the range zero to nine. Zero defines undefined priority. Value one is highest priority.

RESOURCES

```
public static final java.lang.String RESOURCES
```

Equipment or resources anticipated for an activity specified by a calendar item.

STATUS

```
public static final java.lang.String STATUS
```

Overall status or confirmation for the calendar item.

SUMMARY

```
public static final java.lang.String SUMMARY
```

Short summary of the subject for the calendar item.

COMPLETED

`public static final java.lang.String COMPLETED`

Only allowed in: CalendarToDo. Data and time that a to-do was actually completed. Note: This String can be used also as a STATUS field value and PARTSTAT parameter value. Used only in CalendarToDo.

DTEND

`public static final java.lang.String DTEND`

Only allowed in: CalendarEntry. Date and time calendar entry ends.

DUE

`public static final java.lang.String DUE`

Allowed in: CalendarToDo. Date and time calendar to-do is expected to be completed.

DTSTART

`public static final java.lang.String DTSTART`

Specifies when a calendar item begins.

DURATION

`public static final java.lang.String DURATION`

A positive duration of time.

TRANSP

`public static final java.lang.String TRANSP`

Allowed in: CalendarEvent. Specifies if an entry is transparent or not to busy time searches.

ATTENDEE

public static final java.lang.String **ATTENDEE**

Defines an attendee within a calendar item.

CONTACT

public static final java.lang.String **CONTACT**

Represents contact information or alternately a reference to contact information associated with the calendar item.

ORGANIZER

public static final java.lang.String **ORGANIZER**

Organizer for a calendar item.

RELATED_TO

public static final java.lang.String **RELATED_TO**

Represents a relationship or reference between one calendar item and another.

URL

public static final java.lang.String **URL**

Specifies a URL associated with the calendar item.

UID

public static final java.lang.String **UID**

Defines persistent, globally unique identifier for the calendar item.

CREATED

public static final java.lang.String **CREATED**

Date and time calendar item was created.

LAST_MODIFIED

public static final java.lang.String **LAST_MODIFIED**

The date and time calendar item was last revised.

SEQUENCE

public static final java.lang.String **SEQUENCE**

Defines the revision sequence number of the calendar item within a sequence of revisions.

REQUEST_STATUS

public static final java.lang.String **REQUEST_STATUS**

Defines the status code returned for a scheduling request.

ALTREP

public static final java.lang.String **ALTREP**

To specify an alternate text representation for the field value.

Value: Any String.

CN

public static final java.lang.String **CN**

Common name to be associated with the calendar user specified by the field.

Value: Any String.

CUTYPE

public static final java.lang.String **CUTYPE**

Type of calendar user specified by the field.

Value: INDIVIDUAL, GROUP, RESOURCE, ROOM, UNKNOWN.

DELEGATED_FROM

public static final java.lang.String **DELEGATED_FROM**

Calendar users that have delegated their participation to the calendar user specified by the field.

Value: MAILTO URI.

DELEGATED_TO

public static final java.lang.String **DELEGATED_TO**

Calendar users to whom the calendar user specified by the field has delegated participation.

Value: MAILTO URI.

DIR

public static final java.lang.String **DIR**

Reference to a directory entry associated with the calendar user specified by the field.

Value: URI.

ENCODING

public static final java.lang.String **ENCODING**

To specify an alternate inline encoding for the property value.

Value: "8BIT" or "BASE64" or IANA registered iCalendar encoding type or a non-standard content encoding type.

FMTYPE

public static final java.lang.String **FMTYPE**

Content type of a referenced object.

Value: IANA registered content type or a non-standard content type String.

LANGUAGE

`public static final java.lang.String LANGUAGE`

Language for text values in a field or field parameter.

Value: String identifying a language defined in RFC 1766.

MEMBER

`public static final java.lang.String MEMBER`

Group or list membership of the calendar user specified by the field.

Value: MAILTO URI.

PARTSTAT

`public static final java.lang.String PARTSTAT`

The participation status for the calendar user specified by the field.

Value: CalendarEvent: NEEDS-ACTION, ACCEPTED, DECLINED, TENTATIVE, DELEGATED.
CalendarToDo: NEEDS-ACTION, ACCEPTED, DECLINED, TENTATIVE, DELEGATED,
COMPLETED, IN-PROCESS.

RELTYPE

`public static final java.lang.String RELTYPE`

The type of hierarchical relationship associated with the calendar item specified by the field.

Value: PARENT, CHILD, SIBLING.

ROLE

`public static final java.lang.String ROLE`

The participation role for the calendar user specified by the field.

Value: CHAIR, REQ-PARTICIPANT, OPT-PARTICIPANT, NON-PARTICIPANT.

RSVP

`public static final java.lang.String RSVP`

To specify whether there is an expectation of a favor of a reply from the calendar user specified by the field value.

Value: TRUE, FALSE.

SENT_BY

`public static final java.lang.String SENT_BY`

The calendar user that is acting on behalf of the calendar user specified by the property.

Value: MAILTO URI.

TZID

`public static final java.lang.String TZID`

To specify the identifier for the time zone definition for a time component in the property value.

Value: a string.

VALUE

`public static final java.lang.String VALUE`

Explicitly specify the data type format for a field value. Some fields allow the change of data type.

Value: BINARY, TEXT, INTEGER, FLOAT, URI.

INDIVIDUAL

`public static final java.lang.String INDIVIDUAL`

CUTYPE parameter value. An individual.

GROUP

public static final java.lang.String **GROUP**

CUTYPE parameter value. A group of individuals.

RESOURCE

public static final java.lang.String **RESOURCE**

CUTYPE parameter value. A physical resource.

ROOM

public static final java.lang.String **ROOM**

CUTYPE parameter value. A room resource.

UNKNOWN

public static final java.lang.String **UNKNOWN**

CUTYPE parameter value. Otherwise not known.

NEEDS_ACTION

public static final java.lang.String **NEEDS_ACTION**

STATUS field value and PARTSTAT parameter value. Event needs action.

ACCEPTED

public static final java.lang.String **ACCEPTED**

PARTSTAT parameter value. Event accepted.

DECLINED

public static final java.lang.String **DECLINED**

PARTSTAT parameter value. Event declined.

TENTATIVE

public static final java.lang.String **TENTATIVE**

STATUS field value and PARTSTAT parameter value. Event tentatively.

DELEGATED

public static final java.lang.String **DELEGATED**

PARTSTAT parameter value. Event delegated.

IN_PROCESS

public static final java.lang.String **IN_PROCESS**

STATUS field value and PARTSTAT parameter value. Event in process. Used only in CalendarToDo.

PARENT

public static final java.lang.String **PARENT**

RELTYPE parameter value. Parent relationship. Default.

CHILD

public static final java.lang.String **CHILD**

RELTYPE parameter value. Child relationship.

SIBLING

public static final java.lang.String **SIBLING**

RELTYPE parameter value. Sibling relationship.

CHAIR

public static final java.lang.String **CHAIR**

ROLE parameter value. Indicates chair of the calendar entity.

REQ_PARTICIPANT

public static final java.lang.String **REQ_PARTICIPANT**

ROLE parameter value. Indicates a participant whose participation is required.

OPT_PARTICIPANT

public static final java.lang.String **OPT_PARTICIPANT**

ROLE parameter value. Indicates a participant whose participation is optional.

NON_PARTICIPANT

public static final java.lang.String **NON_PARTICIPANT**

ROLE parameter value. Indicates a participant who is copied for information purposes only.

TRUE

public static final java.lang.String **TRUE**

RSVP parameter value.

FALSE

public static final java.lang.String **FALSE**

RSVP parameter value.

BINARY

public static final java.lang.String **BINARY**

VALUE parameter value.

TEXT

public static final java.lang.String **TEXT**

VALUE parameter value.

INTEGER

public static final java.lang.String **INTEGER**

VALUE parameter value.

FLOAT

public static final java.lang.String **FLOAT**

VALUE parameter value.

URI

public static final java.lang.String **URI**

VALUE parameter value.

PUBLIC

public static final java.lang.String **PUBLIC**

CLASS field value.

PRIVATE

public static final java.lang.String **PRIVATE**

CLASS field value.

CONFIDENTIAL

public static final java.lang.String **CONFIDENTIAL**

CLASS field value.

CONFIRMED

public static final java.lang.String **CONFIRMED**

STATUS field value. Event is confirmed.

CANCELLED

public static final java.lang.String **CANCELLED**

STATUS field value. Event is cancelled.

OPAQUE

public static final java.lang.String **OPAQUE**

TRANSP field value. Blocks or opaque on busy time searches.

TRANSPARENT

public static final java.lang.String **TRANSPARENT**

TRANSP field value. Transparent on busy time searches.

Method Detail

openDatabase

public static final CalendarDatabase **openDatabase**(java.lang.String name)
throws DatabaseException

Opens the CalendarDatabase with the specified name.

Parameters:

name - of the database to open. It may not be null.

Returns:

the open CalendarDatabase.

Throws:

DatabaseException - is thrown if the database doesn't exist or factory has not been set.

java.lang.SecurityException - if opening the database isn't allowed.

openDatabase

```
public static final CalendarDatabase openDatabase()  
                                throws DatabaseException
```

Opens the default CalendarDatabase.

Returns:

the open default CalendarDatabase.

Throws:

DatabaseException - is thrown if the database doesn't exist or factory has not been set.

java.lang.SecurityException - if opening the database isn't allowed.

deleteDatabase

```
public static final void deleteDatabase(java.lang.String name)  
                                throws DatabaseException
```

Delete the specified CalendarDatabase.

Parameters:

name - of the database to delete. It may not be null.

Throws:

DatabaseException - is thrown if the database doesn't exist or factory has not been set.

java.lang.SecurityException - if deleting the database isn't allowed.

createDatabase

```
public static final CalendarDatabase createDatabase(java.lang.String name)  
                                throws DatabaseException
```

Create the specified CalendarDatabase.

Parameters:

name - of the database to create. It may not be null.

Throws:

DatabaseException - is thrown if the database doesn't exist or factory has not been set.

java.lang.SecurityException - if creating the database isn't allowed.

addItem

```
public abstract Item addItem(Item item)  
                                throws DatabaseException
```

Adds new item to the database. This method adds an item to the database. The item stored in the database may be a copy of specified item object. If the specified item already exists then item is duplicated. This method returns the added item, further changes must be done to this item. The returned item must also be used when updating changes to the database using update(Item item)

method. Returned item and the item specified as parameter may be different objects.

The addItem method can throw a IllegalArgumentException if an attempt is made to insert a subclass of an item when there is no support for extended calendar items.

Specified by:

addItem in interface Database

Parameters:

`item` - item to add to database

Returns:

the added item, make further changes to this object

Throws:

java.lang.SecurityException - is thrown if adding the item isn't allowed

See Also:

Database.addItem(javax.pim.database.Item)

updateItem

```
public abstract void updateItem(Item item)
                        throws DatabaseException
```

Updates item in this database. If this item has not yet been added to the database DatabaseUpdateException is thrown. Because the item stored in the database may be a copy of item object the items fetched from the database and then changed must be updated to the database using this method.

Use addItem(Item item) to add newly created item to the database.

Update does update all the fields including aggregate fields and field parameters to the database.

Specified by:

updateItem in interface Database

Parameters:

`item` - item to be updated

Throws:

java.lang.SecurityException - is thrown if updating the item isn't allowed

See Also:

Database.updateItem(javax.pim.database.Item)

getDatabaseNames

```
public static Iterator getDatabaseNames()
                        throws DatabaseException,
                        java.lang.SecurityException
```

Returns the names of available calendar databases. The application may or may not have access to all these databases. If no databases are stored this method returns an empty Iterator.

Returns:

iterator of database names; each call to its `next` method will return the name of a calendar database as a `String`.

Throws:

[DatabaseException](#) - is thrown if there was an error getting the names of databases.
`java.lang.SecurityException` - is thrown if fetching the names isn't allowed.

entryItems

```
public abstract Iterator entryItems()  
                        throws DatabaseException
```

Returns an iterator of `CalendarEntry` items. The application may encounter exceptions in this method call or during the `iterator.next` method. If there are no entries stored an empty `Iterator` is returned.

Returns:

an iterator that returns `CalendarEntry`s.

Throws:

[DatabaseException](#) - is thrown if there was an error retrieving results.

items

```
public abstract Iterator items(java.util.Date startDate,  
                                java.util.Date endDate)  
                        throws DatabaseException
```

Returns an iterator of `CalendarEntry` items within the date range specified. The `Iterator` returns `CalendarEntry`s and `CalendarToDo` objects for which any part of the time span overlaps the specified start and end dates inclusively. The items are not returned in any particular order.

For repeating events the `CalendarEvent` has at least one repetition that spans the requested date range. The application can iterate over the event using the normal `getDates` iterator with the same start and end date to retrieve the individual times.

Parameters:

`startDate` - starting date
`endDate` - ending date

Returns:

an iterator that returns the `CalendarEntry` or `CalendarToDo` items within the date range specified.

Throws:

[DatabaseException](#) - is thrown if there was an error retrieving results.

toDoItems

```
public abstract Iterator toDoItems()  
                        throws DatabaseException
```

Returns an iterator of CalendarToDo items. Application may encounter exceptions in this method call or during the `next` of the iterator. If todos are not supported this method returns a `DatabaseException`. If no todos are stored this method returns an empty `Iterator`.

Returns:

an iterator that returns `CalendarToDos`.

Throws:

DatabaseException - is thrown if there was an error retrieving results.

javax.pim.calendar **Class CalendarEntry**

```
java.lang.Object
|
+-- javax.pim.database.Item
    |
    +-- javax.pim.calendar.CalendarEntry
```

```
public class CalendarEntry
    extends Item
```

The CalendarEntry class implements a calendar item. Instances of this class model different kinds of appointments and diary information including the time the entry takes place, recurrency information of the entry, etc. Entry information is stored in a CalendarEntry's fields. Recurrency information is added with the `setRepeat` method.

New CalendarEntries must be added to the database using `CalendarDatabase.addItem`. Calendarentries are updated in the database using the `CalendarDatabase.updateItem` method.

Constructor Summary

Constructor Summary	
<u>CalendarEntry()</u>	
Creates a new empty CalendarEntry.	

Method Summary

<u>ItemField</u> []	<u>getAttendees</u> () This is a utility method for retrieving this entry's "ATTENDEE" fields.
<u>Iterator</u>	<u>getDates</u> (java.util.Date beginDate, java.util.Date endDate) Returns all the Dates this CalendarEntry is going to take place in the range defined by parameters startDate and endDate.
java.lang.String	<u>getDescription</u> () This is a utility method for retrieving the value of this entry's "DESCRIPTION" field.
java.util.Date	<u>getEndDate</u> () This is a utility method for retrieving the value of this entry's "DTEND" field.
<u>Repeat</u>	<u>getRepeat</u> () Gets repeating information of this calendar entry.
java.util.Date	<u>getStartDate</u> () This is a utility method for retrieving the value of this entry's "DTSTART" field.
void	<u>setDescription</u> (java.lang.String description) This is a utility method for setting the value of this entry's "DESCRIPTION" field.
void	<u>setEndDate</u> (java.util.Date date) This is a utility method for setting the value of this entry's "DTEND" field.
void	<u>setRepeat</u> (<u>Repeat</u> repeat) Sets the repeating information of this CalendarEntry.
void	<u>setStartDate</u> (java.util.Date date) This is a utility method for setting the value of this entry's "DTSTART" field.

Methods inherited from class javax.pim.database.Item

addField, getField, getFields, getFields, isSameItem, removeField, size

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail*CalendarEntry*

```
public CalendarEntry()
```

Creates a new empty CalendarEntry.

Method Detail*setRepeat*

```
public void setRepeat(Repeat repeat)
```

Sets the repeating information of this CalendarEntry. Repeating information can be removed by calling this method with a `null` parameter.

Parameters:

repeat - Repeat recurrency information; `null` can be used to remove repeat information.

getRepeat

```
public Repeat getRepeat()
```

Gets repeating information of this calendar entry. Method returns `null` if this entry has no repeating information.

Returns:

the Repeat object for this entry; `null` is returned if there is no repeat information.

getDates

```
public Iterator getDates(java.util.Date beginDate,  
                           java.util.Date endDate)
```

Returns all the Dates this CalendarEntry is going to take place in the range defined by parameters startDate and endDate. This method takes into account all the exemptions this CalendarEntry might have. For non-recurring events the Iterator contains only one Date. If parameters startDate and endDate are `null`, the iterator contains the entire recurring range. Note that the iterator may produce

an unlimited number of dates if both the entry's end date is `null` and `endDate` given as parameter is `null`. If there are no dates in the specified range an empty `Iterator` is returned.

Parameters:

`startDate` - starting date

`endDate` - ending date

Returns:

an iterator of `Date` objects in the defined range of this entry.

getStartDate

```
public java.util.Date getStartDate()
```

This is a utility method for retrieving the value of this entry's "DTSTART" field.

Returns:

starting date (and time) of this entry; `null` is returned if the field is not present.

getEndDate

```
public java.util.Date getEndDate()
```

This is a utility method for retrieving the value of this entry's "DTEND" field.

Returns:

starting date (and time) of this entry; `null` is returned if the field is not present.

setStartDate

```
public void setStartDate(java.util.Date date)
```

This is a utility method for setting the value of this entry's "DTSTART" field. If the field is not present, a new field is added.

Parameters:

`date` - new starting date of this entry

Throws:

`NullPointerException` - is thrown if `date` is `null`.

setEndDate

```
public void setEndDate(java.util.Date date)
```

This is a utility method for setting the value of this entry's "DTEND" field. If the field is not present, a new field is added.

Parameters:

`date` - new ending date of this entry

Throws:

NullPointerException - is thrown if date is `null`.

getDescription

```
public java.lang.String getDescription()
```

This is a utility method for retrieving the value of this entry's "DESCRIPTION" field.

Returns:

the string describing the event; `null` is returned if the field is not present.

setDescription

```
public void setDescription(java.lang.String description)
```

This is a utility method for setting the value of this entry's "DESCRIPTION" field. If the field is not present, a new field is added. You have to update this item to the database to make changes take effect.

Parameters:

`description` - new description for this entry

Throws:

NullPointerException - is thrown if date is `null`.

getAttendees

```
public ItemField[] getAttendees()
```

This is a utility method for retrieving this entry's "ATTENDEE" fields.

Returns:

an array of ItemFields one for each ATTENDEE ItemField in the CalentryEntry; `null` is returned if there no attendees.

javax.pim.calendar **Class CalendarToDo**

```

java.lang.Object
|
+-- javax.pim.database.Item
    |
    +-- javax.pim.calendar.CalendarToDo

```

public class **CalendarToDo**
extends Item

The CalendarToDo class models "To do" items which are used to store tasks to be done. To do item's information is stored in fields.

The addItem method of the CalendarDatabase can throw an UnsupportedOperationException if an attempt is made to insert a CalendarToDo when there is no support.

Constructor Summary

CalendarToDo()

Creates a new empty CalendarToDo.

Method Summary

java.lang.String	<u>getDescription()</u> This is a utility method for retrieving the value of this todo's "DESCRIPTION" field.
java.util.Date	<u>getDueDate()</u> This is a utility method for retrieving the value of this todo's "DUE" field.
int	<u>getPriority()</u> This is a utility method for retrieving the value of this todo's "PRIORITY" field.
void	<u>setDescription(java.lang.String description)</u> This is a utility method for setting the value of this todo's "DESCRIPTION" field.
void	<u>setDueDate(java.util.Date date)</u> This is a utility method for setting the value of this todos's "DUE" field.
void	<u>setPriority(int priority)</u> This is a utility method for setting the value of this todo's "PRIORITY" field.

Methods inherited from class javax.pim.database.Item

addField, getField, getFields, getFields, isSameItem, removeField, size

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

CalendarToDo

```
public CalendarToDo()
```

Creates a new empty CalendarToDo.

Method Detail

getDueDate

```
public java.util.Date getDueDate()
```

This is a utility method for retrieving the value of this todo's "DUE" field.

Returns:

due date (and time) of this todo; `null` is returned if the field is not present.

setDueDate

```
public void setDueDate(java.util.Date date)
```

This is a utility method for setting the value of this todos's "DUE" field. If the field is not present, a new field is added.

Parameters:

`date` - new due date of this entry

Throws:

NullPointerException - is thrown if date is `null`.

getDescription

```
public java.lang.String getDescription()
```

This is a utility method for retrieving the value of this todo's "DESCRIPTION" field.

Returns:

the description of the ToDo; `null` is returned if the field is not present.

setDescription

```
public void setDescription(java.lang.String description)
```

This is a utility method for setting the value of this todo's "DESCRIPTION" field. If the field is not present, a new field is added.

Parameters:

`description` - new description for this entry.

Throws:

NullPointerException - is thrown if description is `null`.

getPriority

```
public int getPriority()
```

This is a utility method for retrieving the value of this todo's "PRIORITY" field. Zero is returned if the field is not present or the PRIORITY field has a value of zero (undefined priority).

Returns:

the priority of this TODO item. The value is an int in the range from zero to nine. Zero indicates an undefined priority; 1 is the highest priority, 9 is the lowest.

setPriority

```
public void setPriority(int priority)
```

This is a utility method for setting the value of this todo's "PRIORITY" field. If the field is not present, a new field is added.

Value is a relative priority for a calendar item. Value is an integer in the range from zero to nine. Zero defines undefined priority. Value one is the highest priority.

Parameters:

`priority` - new priority for this todo

Throws:

`java.lang.IllegalArgumentException` - if specified priority is not in the range from 0 to 9 inclusive.

javax.pim.calendar

Class Repeat

```
java.lang.Object
|
+-- javax.pim.calendar.Repeat
```

```
public class Repeat
    extends java.lang.Object
```

Repeat defines the sequence of dates on which a repeating event occurs. The CalendarEntry refers to a Repeat object which defines the rules for when the event occurs. A Repeat contains one or more rules and dates which describe when the event is to recur. It also can contain exception rules and dates on which the event is *not* to occur. Exception rules and dates take precedence over rules and dates that describe when the event should occur.

The Internet Calendaring and Scheduling Core Object Specification (iCalendar) is [RFC2445](#) and provides the descriptions of the computations used to determine the specific recurring dates. The encodings of day, weekday, month, and other numberings have been modified to follow the values assigned in the JDK(tm) java.util.Calendar class.

The iterator of Repeat returns the events that should occur taking into account the date sequences produced by the rules and dates for event dates and eliminating dates produced by the exception rules and dates.

Constructor Summary

Repeat()

Create an empty Repeat.

Method Summary	
void	<u>addExceptDate</u> (java.util.Date date) Add a date on which this repeating event should NOT occur.
void	<u>addExceptRule</u> (RepeatRule rule) Add a rule representing a sequence of dates when events should NOT occur.
void	<u>addRepeatDate</u> (java.util.Date date) Add a date on which this repeating event should occur.
void	<u>addRepeatRule</u> (RepeatRule rule) Add a rule representing a sequence of dates when events should occur.
<u>Iterator</u>	<u>dates</u> (java.util.Date startDate, java.util.Date from, java.util.Date to) An iterator returning the Dates on which this event occurs.
boolean	<u>equals</u> (java.lang.Object obj) Compares this Repeat to the specified object.
java.util.Date[]	<u>getExceptDates</u> () Get the explicitly set exception dates for this repeating event.
<u>RepeatRule</u> []	<u>getExceptRules</u> () Get the list of exception rules for this repeating event.
java.util.Date[]	<u>getRepeatDates</u> () Get the explicitly set dates for this repeating event.
<u>RepeatRule</u> []	<u>getRepeatRules</u> () Get the RepeatRules for this repeating event.
void	<u>removeExceptDate</u> (java.util.Date date) Remove this date from the execption list of this Repeat.
void	<u>removeExceptRule</u> (RepeatRule rule) Remove an exception rule present in this Repeat.
void	<u>removeRepeatDate</u> (java.util.Date date) Remove a date from this Repeat.
void	<u>removeRepeatRule</u> (RepeatRule rule) Remove a rule present in this Repeat.

Methods inherited from class java.lang.Object
getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

Repeat

public **Repeat**()

Create an empty Repeat.

Method Detail

addRepeatRule

public void **addRepeatRule**([RepeatRule](#) rule)

Add a rule representing a sequence of dates when events should occur.

Parameters:

rule - a RepeatRule to add to the Repeat

Throws:

NullPointerException - is thrown if the rule is null.

removeRepeatRule

public void **removeRepeatRule**([RepeatRule](#) rule)

Remove a rule present in this Repeat.

Parameters:

rule - to remove.

Throws:

NullPointerException - is thrown if the rule is null.

getRepeatRules

public [RepeatRule](#)[] **getRepeatRules**()

Get the RepeatRules for this repeating event.

Returns:

an array of RepeatRules; null is returned if there are no repeat rules.

addRepeatDate

```
public void addRepeatDate(java.util.Date date)
```

Add a date on which this repeating event should occur.

Parameters:

`date` - a Date to add to the Repeat.

Throws:

NullPointerException - is thrown if the rule is `null`.

removeRepeatDate

```
public void removeRepeatDate(java.util.Date date)
```

Remove a date from this Repeat.

Parameters:

`date` - to remove.

Throws:

NullPointerException - is thrown if the rule is `null`.

getRepeatDates

```
public java.util.Date[] getRepeatDates()
```

Get the explicitly set dates for this repeating event.

Returns:

an array of Dates; `null` is returned if there are no repeat dates.

addExceptRule

```
public void addExceptRule(RepeatRule rule)
```

Add a rule representing a sequence of dates when events should NOT occur.

Parameters:

`rule` - a RepeatRule to add to the the exclude sequences.

Throws:

NullPointerException - is thrown if the rule is `null`.

removeExceptRule

```
public void removeExceptRule(RepeatRule rule)
```

Remove an exception rule present in this Repeat.

Parameters:

`rule` - to remove from the exception list.

Throws:

`NullPointerException` - is thrown if the rule is `null`.

getExceptRules

```
public RepeatRule[] getExceptRules()
```

Get the list of exception rules for this repeating event.

Returns:

an array of `RepeatRules`, one for each sequence of exceptions to when this repeat should occur; `null` is returned if there are no exception rules.

addExceptDate

```
public void addExceptDate(java.util.Date date)
```

Add a date on which this repeating event should NOT occur.

Parameters:

`date` - a `Date` to add to the Repeat as a date on which the event should NOT occur.

Throws:

`NullPointerException` - is thrown if the date is `null`.

removeExceptDate

```
public void removeExceptDate(java.util.Date date)
```

Remove this date from the exception list of this Repeat.

Parameters:

`date` - to remove from the list of dates on which the event should not occur.

Throws:

`NullPointerException` - is thrown if the date is `null`.

getExceptDates

```
public java.util.Date[] getExceptDates()
```

Get the explicitly set exception dates for this repeating event.

Returns:

an array of `Dates` on which events should not occur; `null` is return if there are no exception dates.

dates

```
public Iterator dates(java.util.Date startDate,  
                        java.util.Date from,  
                        java.util.Date to)
```

An iterator returning the Dates on which this event occurs. `startDate` is the reference date of the event. The iterator will generate the dates between the `from` and `to` dates. If there are no dates in the specified range an empty `Iterator` is returned.

The iterator returns in sequence the dates described by each of the repeat rules or dates and eliminating any date described by the except rules and dates.

Parameters:

`startDate` - start Date of this iterator

`from` - the beginning of the period for which events should be produced.

`to` - the end of the period for which events should be produced. `null` indicates repeat until there are no more occurrences.

Returns:

an `Iterator` whose `next` method returns `Date` objects for the next occurrence.

equals

```
public boolean equals(java.lang.Object obj)
```

Compares this `Repeat` to the specified object. The result is `true` if and only if the argument is not `null` and is a `Repeat` object that has the same values and repeat rules as this object.

Overrides:

`equals` in class `java.lang.Object`

Parameters:

`obj` - the reference object with which to compare.

Returns:

`true` if this object is non-null, is a `Repeat` and has the same values and repeat rules, otherwise `false`.

javax.pim.calendar ***Class RepeatRule***

```
java.lang.Object
|
+-- javax.pim.calendar.RepeatRule
```

```
public class RepeatRule
extends java.lang.Object
```

Describes a recurring sequence based on the iCalendar recurrence rules. These sequences are used both for repeating occurrences and for the exceptions to those recurrences.

How often events repeat is controlled by a base frequency and interval. The frequency base can be each of minute, hour, day, week, month, or year.

How long the events repeat is controlled by either an end date or a count of events. The base frequency can be modified to repeat on specified minutes, hours, days of the week, months of the year, months, day of month, day or year, week of year and position in the sequence. All modifications can be specified concurrently; for example, "Monthly on the third to the last day of the month between 8 and 9 every 15 minutes"

The Internet Calendaring and Scheduling Core Object Specification (iCalendar) is [RFC2445](#) and provides the descriptions of the computations used to determine the specific recurring dates. The encodings of day, weekday, month, and other numberings have been modified to follow the values assigned in the JDK(tm) `java.util.Calendar` class.

See Also:
[Repeat](#)

Field Summary

static int	<u>DAILY</u> Frequency of events is every day.
static int	<u>FRIDAY</u> Weekday Friday, defined as in java.util.Calendar
static int	<u>HOURLY</u> Frequency of events is every hour.
static int	<u>MINUTELY</u> Frequency of events is every minute.
static int	<u>MONDAY</u> Weekday Monday, defined as in java.util.Calendar
static int	<u>MONTHLY</u> Frequency of events is every month.
static int	<u>SATURDAY</u> Weekday Saturday, defined as in java.util.Calendar
static int	<u>SUNDAY</u> Weekday Sunday, defined as in java.util.Calendar
static int	<u>THURSDAY</u> Weekday Thursday, defined as in java.util.Calendar
static int	<u>TUESDAY</u> Weekday Tuesday, defined as in java.util.Calendar
static int	<u>WEDNESDAY</u> Weekday Wednesday, defined as in java.util.Calendar
static int	<u>WEEKLY</u> Frequency of events is every week.
static int	<u>YEARLY</u> Frequency of events is every year.

Constructor Summary

RepeatRule()

Construct a new recurrence with default parameters.

Method Summary

void	<u>addHour</u> (int hour) Add the value to the list of hours on which to recur.
void	<u>addMinute</u> (int minute) Add the value to the list of minutes on which to recur.
void	<u>addMonth</u> (int month) Add the value to the list of months on which to recur.
void	<u>addMonthDay</u> (int monthday) Add the value to the list of monthdays on which to recur.
void	<u>addSetPos</u> (int setpos) Add the value to the list of positions within the sequence on which to recur.
void	<u>addWeekday</u> (int weekday) Add the value to the list of weekdays on which to recur.
void	<u>addWeeknum</u> (int weeknum) Add the value to the list of week numbers on which to recur.
void	<u>addYearDay</u> (int yearday) Add the value to the list of days of the year on which to recur.
<u>Iterator</u>	<u>dates</u> (java.util.Date startDate, java.util.Date from, java.util.Date to) An iterator returning the Dates on which this event occurs.
boolean	<u>equals</u> (java.lang.Object obj) Compares this RepeatRule to the specified object.
int	<u>getCount</u> () Returns repeat count.
java.util.Date	<u>getEndDate</u> () Gets ending date for recurring event.
int	<u>getFrequency</u> () Get the base frequency of this recurrence rule.
int[]	<u>getHour</u> () Return a copy of the array of hours on which the event is to recur.
int	<u>getInterval</u> () Returns the interval value of this recurring rule..
int[]	<u>getMinute</u> () Return a copy of the array of minutes on which the event is to recur.
int[]	<u>getMonth</u> () Return a copy of the array of months on which the event is to recur.
int[]	<u>getMonthDay</u> () Return a copy of the array of monthdays on which the event is to recur.

int[]	<u>getSetPos()</u> Return a copy of the array of sequence positions on which the event is to recur.
int[]	<u>getWeekday()</u> Return a copy of the array of weekdays on which the event is to recur.
int[]	<u>getWeeknum()</u> Return a copy of the array of weeknums on which the event is to recur.
int	<u>getWeekStart()</u> Returns the weekday of the first day of the week.
int[]	<u>getYearDay()</u> Return a copy of the array of yeardays on which the event is to recur.
void	<u>removeHour(int hour)</u> Remove the value from the list of hours on which to recur.
void	<u>removeMinute(int minute)</u> Remove the value from the list of minutes on which to recur.
void	<u>removeMonth(int month)</u> Remove the value from the list of months on which to recur.
void	<u>removeMonthDay(int monthday)</u> Remove the value from the list of monthdays on which to recur.
void	<u>removeSetPos(int setpos)</u> Remove the value from the list of sequence positions on which to recur.
void	<u>removeWeekday(int weekday)</u> Remove the value from the list of weekdays on which to recur.
void	<u>removeWeeknum(int weeknum)</u> Remove the value from the list of week numbers on which to recur.
void	<u>removeYearDay(int yearday)</u> Remove the value from the list of yeardays on which to recur.
void	<u>setCount(int count)</u> Set the repeat count (count >= 0).
void	<u>setEndDate(java.util.Date end)</u> Sets ending date for recurring event.
void	<u>setFrequency(int frequency)</u> Set the base frequency of this recurrence rule.
void	<u>setInterval(int interval)</u> Sets the interval value of this object (interval > 0).
void	<u>setWeekStart(int weekday)</u> Set the starting day of the week.

Methods inherited from class java.lang.Object

getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail*SUNDAY*

public static final int **SUNDAY**

Weekday Sunday, defined as in java.util.Calendar

MONDAY

public static final int **MONDAY**

Weekday Monday, defined as in java.util.Calendar

TUESDAY

public static final int **TUESDAY**

Weekday Tuesday, defined as in java.util.Calendar

WEDNESDAY

public static final int **WEDNESDAY**

Weekday Wednesday, defined as in java.util.Calendar

THURSDAY

public static final int **THURSDAY**

Weekday Thursday, defined as in java.util.Calendar

FRIDAY

public static final int **FRIDAY**

Weekday Friday, defined as in java.util.Calendar

SATURDAY

public static final int **SATURDAY**

Weekday Saturday, defined as in java.util.Calendar

MINUTELY

public static final int **MINUTELY**

Frequency of events is every minute.

HOURLY

public static final int **HOURLY**

Frequency of events is every hour.

DAILY

public static final int **DAILY**

Frequency of events is every day.

WEEKLY

public static final int **WEEKLY**

Frequency of events is every week.

MONTHLY

public static final int **MONTHLY**

Frequency of events is every month.

YEARLY

public static final int **YEARLY**

Frequency of events is every year.

Constructor Detail

RepeatRule

public **RepeatRule**()

Construct a new recurrence with default parameters. The default frequency is daily. The default interval is 1. The default count is zero and end date is `null` so it repeats forever. The default start of the week is `MONDAY`.

Method Detail

setFrequency

public void **setFrequency**(int frequency)

Set the base frequency of this recurrence rule.

Parameters:

frequency - the base period of this recurrence rule. Should be one of the values `MINUTELY`, `HOURLY`, `DAILY`, `WEEKLY`, `MONTHLY`, `YEARLY`

Throws:

`java.lang.IllegalArgumentException` - is thrown if frequency is not a valid period.

getFrequency

public int **getFrequency**()

Get the base frequency of this recurrence rule.

Returns:

how often this rule applies; `MINUTELY`, `HOURLY`, `DAILY`, `WEEKLY`, `MONTHLY`, `YEARLY`.

getInterval

```
public int getInterval()
```

Returns the interval value of this recurring rule.. The interval is combined with the frequency to select the base period controlling how often the event is to occur. For example, every other week would be: frequency = WEEKLY, interval = 2;

Returns:

the interval of the base frequency.

setInterval

```
public void setInterval(int interval)
```

Sets the interval value of this object (interval > 0).

Parameters:

`interval` - new interval of this repeat

Throws:

`java.lang.IllegalArgumentException` - is thrown if interval is not greater than zero.

getEndDate

```
public java.util.Date getEndDate()
```

Gets ending date for recurring event. The specified date is inclusive; that means that an event occurring at the end date/time is part of the event sequence. If there is no end date then the count controls the number of if events that should be generated.

Returns:

the end date for the events. `null` is returned if an end date is not defined.

See Also:

[getCount\(\)](#)

setEndDate

```
public void setEndDate(java.util.Date end)
```

Sets ending date for recurring event. If set to `null` then the count controls the number of events to be generated.

Parameters:

`end` - the end date for the repeat. May be `null` if there is no end date.

See Also:

[setCount\(int\)](#)

setCount

```
public void setCount(int count)
```

Set the repeat count (count >= 0). Only this number of events should be generated for this recurrence rule.

Parameters:

count - the maximum number of events that should occur

Throws:

java.lang.IllegalArgumentException - is thrown if count is less than zero.

See Also:

[setEndDate\(java.util.Date\)](#)

getCount

```
public int getCount()
```

Returns repeat count. Only this number of events should be generated for this recurrence rule.

Returns:

the maximum number of recurrences that should occur.

See Also:

[getEndDate\(\)](#)

setWeekStart

```
public void setWeekStart(int weekday)
```

Set the starting day of the week.

Parameters:

weekday - the day on which a week begins.

Throws:

java.lang.IllegalArgumentException - is thrown if weekday is not valid.

See Also:

[SUNDAY](#)

getWeekStart

```
public int getWeekStart()
```

Returns the weekday of the first day of the week. For example, Sunday is the first day in the US and Monday is the first day in France.

Returns:

the weekday of the first day of the week.

See Also:
[SUNDAY](#)

addMinute

```
public void addMinute(int minute)
```

Add the value to the list of minutes on which to recur.

Parameters:

minute - value (0-59) to add to the sequence of minutes on which to recur.

Throws:

java.lang.IllegalArgumentException - is thrown if minute is less than zero or greater than 59.

removeMinute

```
public void removeMinute(int minute)
```

Remove the value from the list of minutes on which to recur.

Parameters:

minute - value (0-59) to remove from the sequence of minutes on which to recur.

getMinute

```
public int[] getMinute()
```

Return a copy of the array of minutes on which the event is to recur.

Returns:

array of int, one element for each minute on which the event should recur. `null` is returned if there are none.

addHour

```
public void addHour(int hour)
```

Add the value to the list of hours on which to recur.

Parameters:

hour - value (0-23) to add to the sequence of hours on which to recur.

Throws:

java.lang.IllegalArgumentException - is thrown if hour is less than zero or greater than 23.

removeHour

```
public void removeHour(int hour)
```

Remove the value from the list of hours on which to recur.

Parameters:

hour - value (0-23) to remove to the sequence of hours on which to recur.

getHour

```
public int[] getHour()
```

Return a copy of the array of hours on which the event is to recur.

Returns:

array of int, one element for each hour on which the event should recur. null is returned if there are none.

addWeekday

```
public void addWeekday(int weekday)
```

Add the value to the list of weekdays on which to recur. Weekdays begin at 1. Sunday = 1, Monday = 2,... Saturday = 7; (The enumerations in this class can be used for convenience. e.g. See SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY) Negative values indicate selecting the last weekday in the period. For example, -6 is the last Friday in the period. Weekdays specified in the range of 1-7 will repeat each week. To repeat a particular day each month, the weekday is augmented by adding 7 * "week number", where week numbers start at one. For example, the 3rd Thursday would be represented by : RepeatRule . THURSDAY + (3 * 7).

Parameters:

weekday - value to add to the sequence of weekdays on which to recur. A positive value indicates the first weekday in period. A negative value indicates the last weekday in period.

removeWeekday

```
public void removeWeekday(int weekday)
```

Remove the value from the list of weekdays on which to recur. See addWeekday for the encoding of days of the week.

Parameters:

weekday - value to remove from the sequence of weekdays on which to recur.

getWeekday

```
public int[] getWeekday()
```

Return a copy of the array of weekdays on which the event is to recur.

Returns:

array of int, one element for each weekday on which the event should recur. `null` is returned if there are none.

addMonthDay

```
public void addMonthDay(int monthday)
```

Add the value to the list of monthdays on which to recur. Negative values indicate offsets from the end of the month.

Parameters:

monthday - value (-31 to -1 and 1 to 31) to add to the sequence of day of the month on which to recur.

Throws:

`java.lang.IllegalArgumentException` - is thrown if monthday is zero or less than -31 or greater than 31.

removeMonthDay

```
public void removeMonthDay(int monthday)
```

Remove the value from the list of monthdays on which to recur.

Parameters:

monthday - value (-31 to -1 and 1 to 31) to remove to the sequence of monthdays on which to recur.

getMonthDay

```
public int[] getMonthDay()
```

Return a copy of the array of monthdays on which the event is to recur.

Returns:

array of int, one element for each monthday on which the event should recur. `null` is returned if there are none.

addYearDay

```
public void addYearDay(int yearday)
```

Add the value to the list of days of the year on which to recur. Negative values indicate offsets from the end of the Year.

Parameters:

yearday - value (-366.. -1 and 1..366) to add to the sequence of yeardays on which to recur.

Throws:

java.lang.IllegalArgumentException - is thrown if yearday is zero or is less than -366 or greater than 366.

removeYearDay

```
public void removeYearDay(int yearday)
```

Remove the value from the list of yeardays on which to recur.

Parameters:

yearday - value (-366.. -1 and 1..366) to remove to the sequence of yeardays on which to recur.

getYearDay

```
public int[] getYearDay()
```

Return a copy of the array of yeardays on which the event is to recur.

Returns:

array of int, one element for each yearday on which the event should recur. `null` is returned if there are none.

addWeeknum

```
public void addWeeknum(int weeknum)
```

Add the value to the list of week numbers on which to recur. Negative values indicate offsets from the end of the year.

Parameters:

weeknum - value (-53 to -1 and 1 to 53) to add to the sequence of week numbers on which to recur.

Throws:

java.lang.IllegalArgumentException - is thrown if weeknum is zero or is less than -53 or greater than 53.

removeWeeknum

```
public void removeWeeknum(int weeknum)
```

Remove the value from the list of week numbers on which to recur.

Parameters:

weeknum - value (-53 to -1 and 1 to 53) to remove from the sequence of week numbers on which to recur.

getWeeknum

```
public int[] getWeeknum()
```

Return a copy of the array of weeknums on which the event is to recur.

Returns:

array of int, one element for each weeknum on which the event should recur. `null` is returned if there are none.

addMonth

```
public void addMonth(int month)
```

Add the value to the list of months on which to recur.

Parameters:

month - value (0-11) to add to the sequence of months on which to recur.

Throws:

`java.lang.IllegalArgumentException` - is thrown if month is less than zero or greater than 11.

removeMonth

```
public void removeMonth(int month)
```

Remove the value from the list of months on which to recur.

Parameters:

month - 0-11 value to remove from the sequence of months on which to recur.

getMonth

```
public int[] getMonth()
```

Return a copy of the array of months on which the event is to recur.

Returns:

array of int, one element for each month on which the event should recur. `null` is returned if there are none.

addSetPos

```
public void addSetPos(int setpos)
```

Add the value to the list of positions within the sequence on which to recur. The base sequence is established by the frequency parameter and the minute, hour, weekday, week number, and month qualifiers. Negative values indicate offsets from the end of the sequence.

Parameters:

`setpos` - value (-366 to -1 and 1 - 366) to add to the sequence of positions on which to recur.

removeSetPos

```
public void removeSetPos(int setpos)
```

Remove the value from the list of sequence positions on which to recur.

Parameters:

`setpos` - 0-59 value to remove from the sequence of positions on which to recur.

getSetPos

```
public int[] getSetPos()
```

Return a copy of the array of sequence positions on which the event is to recur.

Returns:

array of int, one element for each sequence position on which the event should recur. `null` is returned if there are none.

dates

```
public Iterator dates(java.util.Date startDate,  
                      java.util.Date from,  
                      java.util.Date to)
```

An iterator returning the Dates on which this event occurs. `startDate` is the reference date for the sequence. It defines the initial hour, minute, second, week, weekday, month, and year from which the pattern is computed. The iterator will generate the dates between the `from` and `to` dates.

The iterator returns in sequence the dates described by this repeat rule.

Parameters:

`startDate` - start Date of this iterator

`from` - the beginning of the period for which events should be produced.

`to` - the end of the period for which events should be produced. `null` indicates repeat until there are no more occurrences.

Returns:

an Iterator whose `next` method returns `Date` objects for the next occurrence.

equals

```
public boolean equals(java.lang.Object obj)
```

Compares this `RepeatRule` to the specified object. The result is `true` if and only if the argument is not `null` and is a `RepeatRule` object that has the same values as this object.

Overrides:

`equals` in class `java.lang.Object`

Parameters:

`obj` - the reference object with which to compare.

Returns:

`true` if this object is non-null, is a `RepeatRule` and has the same values, otherwise `false`.

Package *javax.pim.database*

The Database package defines the objects used to access Addressbook and Calendar information.

See:

Description

Interface Summary	
<u>Database</u>	The Database interface models a single database.
<u>DatabaseFactory</u>	DatabaseFactory is the interface to factories that manage databases.
<u>DatabaseListener</u>	The Listener interface for notifications of events occurring in the Database.
<u>Iterator</u>	Interface provided for JDK 1.1 compatibility.

Class Summary	
<u>AggregateField</u>	AggregateField is a subtype of ItemField that can contain a sequence of ItemFields.
<u>DatabaseEvent</u>	DatabaseEvents are used to notify DatabaseListeners about changes to a Database.
<u>Item</u>	Item are stored in a Database.
<u>ItemField</u>	ItemFields hold the values stored in Database Items.
<u>Parameter</u>	Parameter objects hold a name and optional value.

Exception Summary	
<u>DatabaseException</u>	DatabaseExceptions are thrown when some kind of exception occurs in database operations.
<u>DatabaseUpdateException</u>	This exception is thrown if the application tries to update an Item which has not been added to the database yet or has been updated in the database by another application.

Package javax.pim.database Description

The Database package defines the objects used to access Addressbook and Calendar information. When changes are made in the database, the Database package sends events to notify listeners. Subclasses of databases such as AddressBook or Calendar can notify listeners of changes to the database. For example, if a date entry is modified, registered listeners are notified.

Hierarchy For Package javax.pim.database

Package Hierarchies:

All Packages

Class Hierarchy

- class java.lang.Object
 - class java.util.EventObject (implements java.io.Serializable)
 - class javax.pim.database.**DatabaseEvent**
 - class javax.pim.database.**Item**
 - class javax.pim.database.**ItemField**
 - class javax.pim.database.**AggregateField**
 - class javax.pim.database.**Parameter**
 - class java.lang.Throwable (implements java.io.Serializable)
 - class java.lang.Exception
 - class javax.pim.database.**DatabaseException**
 - class javax.pim.database.**DatabaseUpdateException**

Interface Hierarchy

- interface javax.pim.database.**Database**
 - interface javax.pim.database.**DatabaseFactory**
 - interface java.util.EventListener
 - interface javax.pim.database.**DatabaseListener**
 - interface javax.pim.database.**Iterator**
-
-

javax.pim.database

Class AggregateField

```

java.lang.Object
|
+-- javax.pim.database.ItemField
    |
    +-- javax.pim.database.AggregateField

```

```

public class AggregateField
extends ItemField

```

AggregateField is a subtype of ItemField that can contain a sequence of ItemFields. The Itemfields in the sequence can be added, inserted, removed, set, and retrieved.

The value of an ItemField is undefined for AggregateFields. Inherited methods to get and set the value will throw IllegalStateException.

Fields inherited from class javax.pim.database. <u>ItemField</u>
--

AGGREGATE, <u>B</u> OOLEAN, <u>B</u> YTE_ARRAY, <u>D</u> ATE, <u>D</u> OUBLE, <u>I</u> NT, <u>I</u> TEM, <u>S</u> TRING

Constructor Summary

<u>AggregateField</u> (java.lang.String name) Constructs a new AggregateField with the specified name.	
--	--

Method Summary

void	<u>addField</u> (ItemField field) Appends the specified ItemField to the end of this aggregate.
boolean	<u>equals</u> (java.lang.Object obj) Compares this AggregateField to the specified object.
ItemField	<u>getField</u> (int index) Get a specific field of this aggregate.
int	<u>hashCode</u> () Get the hashCode for this ItemField.
void	<u>insertField</u> (ItemField field, int index) Inserts the specified field at the specified position in this aggregate.
void	<u>removeField</u> (int index) Removes the field at the specified index in this aggregate.
void	<u>setField</u> (ItemField field, int index) Sets the field at the specified index of this aggregate to the ItemField.
int	<u>size</u> () Returns the number of fields in this aggregate.
java.lang.String	<u>toString</u> () Return the value of the aggregate field by concatenating the field values.
java.lang.String	<u>toString</u> (java.lang.String separator) Return the value of the aggregate field by concatenating the field values with a separator character.

Methods inherited from class javax.pim.database.ItemField

addParameter, compareTo, getBoolean, getByteArray, getDate, getDouble, getFieldName, getInt, getItem, getLabel, getParameters, getParameterValues, getString, getValueType, hasParameter, removeParameter, setBoolean, setByteArray, setDate, setDouble, setInt, setItem, setLabel, setString

Methods inherited from class java.lang.Object

getClass, notify, notifyAll, wait, wait, wait

Constructor Detail

AggregateField

public **AggregateField**(java.lang.String name)

Constructs a new AggregateField with the specified name.

Parameters:

name - the name of the aggregate field.

Method Detail

size

public int **size**()

Returns the number of fields in this aggregate.

Returns:

the number of fields.

getField

public ItemField **getField**(int index)

Get a specific field of this aggregate.

Parameters:

index - field number starting from 0 to size() - 1

Returns:

corresponding ItemField.

Throws:

java.lang.IndexOutOfBoundsException - thrown if index is less than 0 or greater than or equals to size().

removeField

public void **removeField**(int index)
throws java.lang.ArrayIndexOutOfBoundsException

Removes the field at the specified index in this aggregate. Shifts any subsequent fields to the left (subtracts one from their indices). The size of this item is decreased by 1.

The index must be a value greater than or equal to 0 and less than the current size of the aggregate field.

Parameters:

`index` - of the field in this aggregate to be removed.

Throws:

`java.lang.ArrayIndexOutOfBoundsException` - if the index was invalid.

addField

```
public void addField(ItemField field)
```

Appends the specified ItemField to the end of this aggregate.

Parameters:

`field` - ItemField to append.

insertField

```
public void insertField(ItemField field,  
                        int index)  
    throws java.lang.ArrayIndexOutOfBoundsException
```

Inserts the specified field at the specified position in this aggregate. Shifts the field currently at that position (if any) and any subsequent fields to the right (adds one to their indices).

The index must be a value greater than or equal to 0 and less than or equal to the current size of the aggregate field.

Parameters:

`field` - ItemField to insert at the index.

`index` - at which the specified field is to be inserted.

Throws:

`java.lang.ArrayIndexOutOfBoundsException` - index is out of range (`index < 0 || index > size()`).

setField

```
public void setField(ItemField field,  
                    int index)  
    throws java.lang.ArrayIndexOutOfBoundsException
```

Sets the field at the specified index of this aggregate to the ItemField.

The index must be a value greater than or equal to 0 and less than the current size of the aggregate field.

Parameters:

`index` - index of element to replace.

`field` - ItemField to be stored at the specified index.

Throws:

java.lang.ArrayIndexOutOfBoundsException - index is out of range (index < 0 || index >= size()).

equals

```
public boolean equals(java.lang.Object obj)
```

Compares this AggregateField to the specified object. The result is true if and only if the argument is not null and is an AggregateField with the same name, value type, parameters, and values as this AggregateField.

Overrides:

equals in class ItemField

Parameters:

obj - the reference object with which to compare.

Returns:

true if this object is non-null, is a AggregateField and has the same name, value type, parameters and values, otherwise false.

hashCode

```
public int hashCode()
```

Get the hashCode for this ItemField.

Overrides:

hashCode in class ItemField

Returns:

the hashCode for this ItemField.

toString

```
public java.lang.String toString()
```

Return the value of the aggregate field by concatenating the field values.

Overrides:

toString in class ItemField

Returns:

a string consisting of the individual fields concatenated with the separator of ", ".

toString

```
public java.lang.String toString(java.lang.String separator)
```

Return the value of the aggregate field by concatenating the field values with a separator character.

Parameters:

`string` - to separate the fields.

Returns:

a string describing this `AggregateField` as a concatenation of the individual fields.

javax.pim.database **Interface Database**

All Known Implementing Classes:

[CalendarDatabase](#), [ContactDatabase](#)

public interface **Database**

The Database interface models a single database. Each Database consists of a set of Items. Database defines methods to traverse, add, delete and update the Items. Clients of the Database can register listeners to be notified when changes are made to the database.

Each Item is a set of zero or more ItemFields. ItemFields can be added, removed, and retrieved from the Item.

Each ItemField contains a named and typed value. Each ItemField may also have a set of parameters used to qualify the value.

See Also:

[Item](#), [ItemField](#), [DatabaseListener](#)

Method Summary	
void	<u>addDatabaseListener</u> (<u>DatabaseListener</u> listener) Adds the specified database listener to receive events from this database.
<u>Item</u>	<u>addItem</u> (<u>Item</u> item) Adds a new item to the database.
void	<u>close</u> () Close this database.
void	<u>deleteItem</u> (<u>Item</u> item) Delete an Item from this database.
boolean	<u>isSupported</u> (java.lang.String name) Check if this Database supports the named field or parameter.
<u>Iterator</u>	<u>items</u> () Returns an iterator that returns every Item in the database.
<u>Iterator</u>	<u>items</u> (java.lang.String name) Returns an iterator that returns every Item in the database that has an ItemField with the same field name as specified.
<u>Iterator</u>	<u>items</u> (java.lang.String name, java.lang.Object value) Returns an iterator that returns every Item in the database that has an ItemField with the same field name and field value as specified.
void	<u>removeDatabaseListener</u> (<u>DatabaseListener</u> listener) Removes the specified database listener so that it no longer receives events from this database.
void	<u>updateItem</u> (<u>Item</u> item) Updates the Item in this database.

Method Detail

items

```
public Iterator items()
    throws DatabaseException
```

Returns an iterator that returns every Item in the database. Errors that occur related to establishing the iteration generate DatabaseExceptions. Errors that occur related to accessing the Items in the database may occur in the methods of the returned Iterator. Those errors are reported with RuntimeExceptions. If no items are stored this method returns an empty Iterator.

Returns:

an Iterator that returns the `Items` in the database in an arbitrary order.

Throws:

DatabaseException - if there was an error accessing the database.

items

```
public Iterator items(java.lang.String name)
    throws DatabaseException
```

Returns an iterator that returns every `Item` in the database that has an `ItemField` with the same field name as specified. Errors that occur related to establishing the iteration generate `DatabaseExceptions`. Errors that occur related to accessing the `Items` in the database may occur in the methods of the returned `Iterator`. Those errors are reported with `RuntimeExceptions`. If no items are stored this method returns an empty `Iterator`.

Parameters:

`name` - the name of an `ItemField`. Every `Item` in the database that contains an `ItemField` with this name will be returned in the `Iterator`.

Returns:

an Iterator that returns the `Items` in the database that have an `ItemField` with the specified name. The `Items` are returned in an arbitrary order.

Throws:

DatabaseException - if there was an error accessing the database.

items

```
public Iterator items(java.lang.String name,
    java.lang.Object value)
    throws DatabaseException
```

Returns an iterator that returns every `Item` in the database that has an `ItemField` with the same field name and field value as specified. Errors that occur related to establishing the iteration generate `DatabaseExceptions`. Errors that occur related to accessing the `Items` in the database may occur in the methods of the returned `Iterator`. Those errors are reported with `RuntimeExceptions`. If no items are stored this method returns an empty `Iterator`.

Matching is done with following rules and can be locale dependent:

For fields that store `Strings` as values, matching is free from beginning of words, case insensitive and searches through aggregate fields also. Word separation is defined as any character other than alphanumeric.

- Example 0: `items("NOTE","pho")` matches, for example, in "Mobile phone" but not "JavaPhone".
- Example 1: `items("N","Luc")`; matches, for example, in `AggregateField` "Picard"; "Jean"; "Luc"; "captain"; "" and also in `String` "Jean-Luc".

All the other data types match 'exactly'. If value parameter is `null`, then this method return all fields with specified name that have not set the value (have value `null`).

Parameters:

name - the name of an ItemField. Every Item in the database that contains an ItemField with this name will be returned in the Iterator.

value - to match the value of the selected ItemFields.

Returns:

an Iterator that returns the **Items** in the database that have an ItemField with the specified name. The Items are returned in an arbitrary order.

Throws:

DatabaseException - if there was an error accessing the database. or if the value is incompatible with specified field name.

close

```
public void close()
    throws DatabaseException
```

Close this database. Subsequent invocation of methods on this database throw a DatabaseException.

All of the registered DatabaseListeners are notified before the database is closed.

Throws:

DatabaseException - if there was an error closing the database

addItem

```
public Item addItem(Item item)
    throws DatabaseException
```

Adds a new item to the database. This method adds an item to the database. Items stored in the database are not required to be unique. Adding an Item with ItemFields identical to a previously added Item will result in both Items being in the database. To modify an existing Item, retrieve it with one of the **Items**() iterators, modify it and invoke updateItem. The returned Item and the Item specified as a parameter may be different objects.

All of the registered DatabaseListeners are notified of the change after the Item has been added to the database.

Parameters:

item - to add to the database

Returns:

the added Item, further changes should be made to this object.

Throws:

DatabaseException - if there was an error adding to the database.

java.lang.IllegalArgumentException - for an illegal item subclass or a field with an illegal value that can't be stored in the database. The argument to the exception should indicate the illegal value.

UnsupportedOperationException - if a field or parameter type or value is not supported by the database implementation. An application can check which field or parameter types are

supported using the `isSupported` method. The argument to the exception should indicate the unsupported field or value.

updateItem

```
public void updateItem(Item item)
    throws DatabaseException,
           DatabaseUpdateException
```

Updates the Item in this database. The `Item` must have previously been retrieved from the database or returned from `addItem`. If this item has not yet been added to the database `DatabaseUpdateException` is thrown. Use `addItem` to add a newly created Item to the database.

All of the fields including aggregate fields and field parameters are updated in the database. Item and ItemField objects previously retrieved from the database may not reflect the changes and should be retrieved from the database again.

All of the registered DatabaseListeners are notified of the change after the Item has been updated in the database.

Parameters:

`item` - to be updated in the database

Throws:

DatabaseException - if there was an error updating the database.

DatabaseUpdateException - if the item cannot be updated in the database because it was not retrieved from the database or was modified by another application before this update.

`java.lang.IllegalArgumentException` - for an illegal item subclass or a field with an illegal value that can't be stored in the database. The argument to the exception should indicate the illegal value.

`UnsupportedOperationException` - if a field or parameter type or value is not supported by the database implementation. An application can check which field or parameter types are supported using the `isSupported` method. The argument to the exception should indicate the unsupported field or value.

deleteItem

```
public void deleteItem(Item item)
    throws DatabaseException
```

Delete an Item from this database. The specified Item must have been retrieved from the database using one of the query methods `items` Iterators or returned from `addItem`.

All of the registered DatabaseListeners are notified of the change after the Item has been deleted from the database.

Parameters:

`item` - to be deleted from the database.

Throws:

DatabaseException - if there was an error deleting the Item from the database.

isSupported

```
public boolean isSupported(java.lang.String name)
```

Check if this Database supports the named field or parameter. Returns true if the name is supported. If it returns true then **addItem** and **updateItem** must not throw an exception because an ItemField with the specified name is added or updated, or the ItemField has a parameter of the specified name is added or updated.

Parameters:

name - of a field or parameter.

Returns:

true if the named field or parameter is supported by this database, false otherwise.

addDatabaseListener

```
public void addDatabaseListener(DatabaseListener listener)
```

Adds the specified database listener to receive events from this database.

Parameters:

listener - to receive notifications of changes to this database.

removeDatabaseListener

```
public void removeDatabaseListener(DatabaseListener listener)
```

Removes the specified database listener so that it no longer receives events from this database.

Parameters:

listener - to be removed.

javax.pim.database ***Class DatabaseEvent***

```

java.lang.Object
|
+-- java.util.EventObject
|
+-- javax.pim.database.DatabaseEvent

```

```

public class DatabaseEvent
extends java.util.EventObject

```

DatabaseEvents are used to notify DatabaseListeners about changes to a Database. The `EventObject.getSource` method returns the Database in which the event occurred.

See Also:

[Database](#), [DatabaseListener](#), [Serialized Form](#)

Field Summary

static int	<u>DATABASE_CLOSED</u> This state-change-value indicates that the database was closed.
static int	<u>DATABASE_ITEM_ADDED</u> This state-change-value indicates that an item was added to the database.
static int	<u>DATABASE_ITEM_DELETED</u> This state-change-value indicates that an item was deleted from the database.
static int	<u>DATABASE_ITEM_UPDATED</u> This state-change-value indicates that an item was updated in the database.

Constructor Summary

<u>DatabaseEvent</u> (Database database, int type)	
Construct a new DatabaseEvent with database and event type.	
<u>DatabaseEvent</u> (Database database, int type, Item item)	
Construct a new DatabaseEvent with database, event type and Item.	

Method Summary

void	<u>dispatch</u> (DatabaseListener listener) Dispatch the event to the specified listener.
<u>Item</u>	<u>getItem</u> () Returns the Item which caused this event.
int	<u>getType</u> () Returns the type of this event.

Methods inherited from class java.util.EventObject

getSource, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Detail

DATABASE_ITEM_DELETED

```
public static final int DATABASE_ITEM_DELETED
```

This state-change-value indicates that an item was deleted from the database.

DATABASE_ITEM_ADDED

```
public static final int DATABASE_ITEM_ADDED
```

This state-change-value indicates that an item was added to the database.

DATABASE_ITEM_UPDATED

```
public static final int DATABASE_ITEM_UPDATED
```

This state-change-value indicates that an item was updated in the database.

DATABASE_CLOSED

```
public static final int DATABASE_CLOSED
```

This state-change-value indicates that the database was closed.

Constructor Detail

DatabaseEvent

```
public DatabaseEvent(Database database,  
                     int type)
```

Construct a new DatabaseEvent with database and event type. The database will returned from getSource when requested.

Parameters:

database - in which the event occurred.

type - the type of event.

DatabaseEvent

```
public DatabaseEvent(Database database,  
                     int type,  
                     Item item)
```

Construct a new DatabaseEvent with database, event type and Item. The database will returned from getSource when requested.

Parameters:

database - in which the event occurred.

type - the type of event.

item - which caused the event.

Method Detail

getType

```
public int getType()
```

Returns the type of this event.

Returns:

type of event; one of DATABASE_CLOSED, DATABASE_ITEM_UPDATED, DATABASE_ITEM_ADDED, DATABASE_ITEM_DELETED.

getItem

public Item **getItem()**

Returns the Item which caused this event. It may return `null` if there is no Item related to the event.

Returns:

the Item modified or deleted in the database; is `null` if the event does not pertain to an Item.

dispatch

public void **dispatch**(DatabaseListener listener)

Dispatch the event to the specified listener. This method calls the method of the listener corresponding to the event type on this event object.

Parameters:

listener - used for the dispatch

javax.pim.database ***Class DatabaseException***

```

java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--javax.pim.database.DatabaseException
  
```

Direct Known Subclasses:

[DatabaseUpdateException](#)

```

public class DatabaseException
extends java.lang.Exception
  
```

DatabaseExceptions are thrown when some kind of exception occurs in database operations.

See Also:

[Database](#), [Serialized Form](#)

Constructor Summary

DatabaseException()

Constructs a DatabaseException with no detail message.

DatabaseException(java.lang.String msg)

Constructs a DatabaseException with the specified detail message.

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

DatabaseException

`public DatabaseException()`

Constructs a DatabaseException with no detail message.

DatabaseException

`public DatabaseException(java.lang.String msg)`

Constructs a DatabaseException with the specified detail message.

Parameters:

msg - the detail message

javax.pim.database

Interface DatabaseFactory

public interface **DatabaseFactory**

DatabaseFactory is the interface to factories that manage databases. Application programmers should not call these factory methods directly, they should use the static methods in database implementations, for example in class ContactDatabase.

Character "/" is reserved in database names for future extension for hierarchical names.

Method Summary

<u>Database</u>	createDatabase (java.lang.String name) The database with the requested name should be created by this factory if possible.
void	deleteDatabase (java.lang.String name) This factory method deletes the named database.
<u>Iterator</u>	getDatabaseNames () Returns an Iterator that returns the names of the databases this factory makes available.
<u>Database</u>	openDatabase (java.lang.String name) The database with the requested name should be opened by this factory if possible.

Method Detail

openDatabase

```
public Database openDatabase(java.lang.String name)
                        throws DatabaseException,
                        java.lang.SecurityException
```

The database with the requested name should be opened by this factory if possible. If successful the factory will return an open Database. For example, when opening the database, the implementation may open a native contact db.

Parameters:

name - of the database to open.

Returns:

an open Database of the specified name.

Throws:

DatabaseException - if there was an error opening the database or the specified database doesn't exist.

java.lang.SecurityException - if opening the database isn't allowed.

createDatabase

```
public Database createDatabase(java.lang.String name)
    throws DatabaseException,
    java.lang.SecurityException
```

The database with the requested name should be created by this factory if possible. If successful the factory will return an open Database. DatabaseException exception is thrown if the implementation doesn't allow this or the specified database already exists.

Parameters:

name - of the database to create.

Returns:

an new Database of the specified name.

Throws:

DatabaseException - if there was an error creating the database.

java.lang.SecurityException - if creating a database isn't allowed.

deleteDatabase

```
public void deleteDatabase(java.lang.String name)
    throws DatabaseException,
    java.lang.SecurityException
```

This factory method deletes the named database. DatabaseException exception is thrown if the implementation doesn't allow this or the specified database doesn't exist.

Parameters:

name - of the database to delete.

Throws:

DatabaseException - if there was an error deleting the database

java.lang.SecurityException - if deleting the database isn't allowed.

getDatabaseNames

```
public Iterator getDatabaseNames()
    throws DatabaseException,
    java.lang.SecurityException
```

Returns an Iterator that returns the names of the databases this factory makes available. If no databases are stored this method returns an empty Iterator. If the database supports DEFAULT_XXX databases, they are included in the list of returned database names.

Returns:

iterator of database names returned as Strings.

Throws:

DatabaseException - if there was an error getting the names of databases.

java.lang.SecurityException - if fetching the names isn't allowed.

javax.pim.database **Interface DatabaseListener**

public interface **DatabaseListener**
extends java.util.EventListener

The Listener interface for notifications of events occurring in the Database. When registered with an open database the listener methods are invoked when the database is closed, and items are added, deleted, or updated.

See Also:

[Database](#), [DatabaseEvent](#)

Method Summary

void	databaseClosed (DatabaseEvent event) Invoked when the database is closed.
void	databaseItemAdded (DatabaseEvent event) Invoked when an item is added to database.
void	databaseItemDeleted (DatabaseEvent event) Invoked when an item is deleted from database.
void	databaseItemUpdated (DatabaseEvent event) Invoked when an item is updated in database.

Method Detail

databaseClosed

public void **databaseClosed**([DatabaseEvent](#) event)

Invoked when the database is closed.

Parameters:

event - describing the DATABASE_CLOSED event.

databaseItemDeleted

```
public void databaseItemDeleted(DatabaseEvent event)
```

Invoked when an item is deleted from database.

Parameters:

event - describing the DATABASE_ITEM_DELETED event.

databaseItemAdded

```
public void databaseItemAdded(DatabaseEvent event)
```

Invoked when an item is added to database.

Parameters:

event - describing the DATABASE_ITEM_ADDED event.

databaseItemUpdated

```
public void databaseItemUpdated(DatabaseEvent event)
```

Invoked when an item is updated in database.

Parameters:

event - describing the DATABASE_ITEM_UPDATED event.

javax.pim.database

Class DatabaseUpdateException

```

java.lang.Object
|
+-- java.lang.Throwable
    |
    +-- java.lang.Exception
        |
        +-- javax.pim.database.DatabaseException
            |
            +-- javax.pim.database.DatabaseUpdateException

```

public class **DatabaseUpdateException**
 extends DatabaseException

This exception is thrown if the application tries to update an Item which has not been added to the database yet or has been updated in the database by another application.

See Also:

Serialized Form

Constructor Summary

DatabaseUpdateException()

Constructs a DatabaseUpdateException with no detail message.

DatabaseUpdateException(java.lang.String msg)

Constructs a DatabaseUpdateException with the specified detail message.

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

DatabaseUpdateException

`public DatabaseUpdateException()`

Constructs a DatabaseUpdateException with no detail message.

DatabaseUpdateException

`public DatabaseUpdateException(java.lang.String msg)`

Constructs a DatabaseUpdateException with the specified detail message.

Parameters:

`msg` - the detail message

javax.pim.database***Class Item***

java.lang.Object

|

+-- **javax.pim.database.Item**

Direct Known Subclasses:

[CalendarEntry](#), [CalendarToDo](#), [ContactCard](#)

public class **Item**
extends java.lang.Object

Item are stored in a Database. Each Item is a set of zero or more ItemFields. Items are not required to be unique within a database. ItemFields can be added, removed, and retrieved from the Item. Item has methods to add, remove and retrieve ItemFields. ItemFields in an Item are not ordered and are not required to be unique.

See Also:

[Database](#), [ItemField](#), [AggregateField](#)

Constructor Summary	
<u>Item</u> ()	
Constructs a new Item object that has no fields.	

Method Summary

void	<u>addField</u> (<u>ItemField</u> field) Adds the specified field to this Item.
<u>ItemField</u>	<u>getField</u> (java.lang.String name) Return an arbitrary field that has the specified name.
<u>ItemField</u> []	<u>getFields</u> () Return an array of all ItemFields of this item.
<u>ItemField</u> []	<u>getFields</u> (java.lang.String name) Return an array of fields from this Item with the specified field name string.
boolean	<u>isSameItem</u> (<u>Item</u> item) Determines if this Item and the other item either refers to the same Item object (using <code>this == item</code>) or they were both retrieved from the same database and from the same Item within that database.
void	<u>removeField</u> (<u>ItemField</u> field) Removes the field from this Item.
int	<u>size</u> () Returns the number of fields in this Item.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

Item

public **Item**()

Constructs a new Item object that has no fields.

Method Detail

getField

```
public ItemField getField(java.lang.String name)
```

Return an arbitrary field that has the specified name. `null` is returned if a field is not found. Because the Item can have several fields with the same name the Item returned is arbitrary. Use `getFields(String name)` instead if you want to get all the fields with single name.

Parameters:

name - of the field to return.

Returns:

an ItemField with the specified name.

getFields

```
public ItemField[] getFields(java.lang.String name)
```

Return an array of fields from this Item with the specified field name string. This method will return all fields with the specified name in an array or `null` if no fields were found. Modifying the array will not modify the fields of this Item.

Parameters:

name - of the fields to select.

Returns:

an array of ItemFields in this Item with the specified name; `null` is returned if there are no matching fields.

getFields

```
public ItemField[] getFields()
```

Return an array of all ItemFields of this item.

Returns:

an array of ItemFields in this Item; `null` is returned if the item has no fields.

addField

```
public void addField(ItemField field)
```

Adds the specified field to this Item.

Parameters:

field - to add to the Item.

removeField

```
public void removeField(ItemField field)
```

Removes the field from this Item. If the Item contains a reference to the field (using `==`) it is removed, otherwise no action is taken. If field is an Aggregate field, its subfields are also removed.

Parameters:

`field` - in this Item to be removed.

size

```
public int size()
```

Returns the number of fields in this Item. Aggregate fields' subfields are not counted.

Returns:

number of fields in this Item.

isSameItem

```
public boolean isSameItem(Item item)
```

Determines if this Item and the other item either refers to the same Item object (using `this == item`) or they were both retrieved from the same database and from the same Item within that database. This is useful when a `DATABASE_ITEM_UPDATED` Event is delivered to determine whether the changed item is an update to one or more items already retrieved from the Database.

Parameters:

`item` - an Item to compare with this item.

Returns:

`true` if this Item and the other item refer to the same Item object or were retrieved from the same database and same stored Item. `false` is returned in all other cases.

javax.pim.database ***Class ItemField***

```
java.lang.Object
|
+-- javax.pim.database.ItemField
```

Direct Known Subclasses:

[AggregateField](#)

```
public class ItemField
extends java.lang.Object
```

ItemFields hold the values stored in Database Items. Each ItemField instance stores a value. ItemFields are added to Items which in turn are added to a Database. The database stores them persistently. The data type of the field is defined when the field is created and cannot be changed afterwards. An ItemField has a set of named Parameters that are used to qualify the value.

Particular types of Databases require that some fields with specific names have specific value types. For example, in the vCard specification a field with the name "FN" has a value that must be of datatype String.

If an ItemField stores an object such as Date or Item it does not copy the object.

This class is used only for single valued fields. The subclass `AggregateField` is used for multivalued fields, for example for AddressBook fields "N" or "ADR".

See Also:

[AggregateField](#), [Parameter](#), [Item](#), [Database](#)

Field Summary	
static byte	<u>AGGREGATE</u> The valuetype for Aggregate valued ItemFields.
static byte	<u>BOOLEAN</u> The valuetype for boolean valued ItemFields.
static byte	<u>BYTE_ARRAY</u> The valuetype for byte array valued ItemFields.
static byte	<u>DATE</u> The valuetype for Date valued ItemFields.
static byte	<u>DOUBLE</u> The valuetype for double valued ItemFields.
static byte	<u>INT</u> The valuetype for int valued ItemFields.
static byte	<u>ITEM</u> The valuetype for Item valued ItemFields.
static byte	<u>STRING</u> The valuetype for String valued ItemFields.

Constructor Summary	
<u>ItemField</u> (java.lang.String name, boolean value)	Constructs new ItemField with the specified boolean value.
<u>ItemField</u> (java.lang.String name, byte[] value)	Constructs new ItemField with the specified byte array value.
<u>ItemField</u> (java.lang.String name, java.util.Date value)	Constructs new ItemField with the specified Date value.
<u>ItemField</u> (java.lang.String name, double value)	Constructs new ItemField with the specified double value.
<u>ItemField</u> (java.lang.String name, int value)	Constructs new ItemField with the specified int value.
<u>ItemField</u> (java.lang.String name, <u>Item</u> value)	Constructs new ItemField with the specified Item value.
<u>ItemField</u> (java.lang.String name, java.lang.String value)	Constructs new ItemField with the specified String value.

Method Summary	
boolean	<u>addParameter</u> (<u>Parameter</u> parameter) Adds the Parameter object to this field's set of parameters.
int	<u>compareTo</u> (java.lang.Object obj) Compares the values of this ItemField and another ItemField of the same type.
boolean	<u>equals</u> (java.lang.Object obj) Compares this ItemField to the specified object.
boolean	<u>getBoolean</u> () Returns the boolean value of this field.
byte[]	<u>getByteArray</u> () Returns the byte array value of this field.
java.util.Date	<u>getDate</u> () Returns the Date value of this field.
double	<u>getDouble</u> () Returns the double value of this field.
java.lang.String	<u>getFieldName</u> () Returns the name string of this field.
int	<u>getInt</u> () Returns the integer value of this field.
<u>Item</u>	<u>getItem</u> () Returns the Item value of this field.
java.lang.String	<u>getLabel</u> () Return the label of this field.
<u>Parameter</u> []	<u>getParameters</u> () Gets an array of the Parameters of this field or null if no parameters are present.
java.lang.String[]	<u>getParameterValues</u> (java.lang.String name) Get the values of all Parameters with the specified name.
java.lang.String	<u>getString</u> () Returns the String value of this field.
byte	<u>getValueType</u> () Returns the type of the value.
int	<u>hashCode</u> () Get the hashcode for this ItemField.

boolean	<u>hasParameter</u> (java.lang.String name) Checks if this field has at least one parameter with the name.
boolean	<u>removeParameter</u> (Parameter parameter) Removes the specified Parameter object from this ItemField.
void	<u>setBoolean</u> (boolean value) Sets the boolean value of this field.
void	<u>setByteArray</u> (byte[] value) Sets the byte array value of this field.
void	<u>setDate</u> (java.util.Date value) Sets the Date value of this field.
void	<u>setDouble</u> (double value) Sets the double value of this field.
void	<u>setInt</u> (int value) Sets the int value of this field.
void	<u>setItem</u> (Item value) Sets the Item value of this field.
void	<u>setLabel</u> (java.lang.String label) Sets user defined label associated with this field.
void	<u>setString</u> (java.lang.String value) Sets the String value of this field.
java.lang.String	<u>toString</u> () Returns a String representation of the value of this ItemField.

Methods inherited from class java.lang.Object

getClass, notify, notifyAll, wait, wait, wait

Field Detail

STRING

public static final byte **STRING**

The valuetype for String valued ItemFields.

INT

public static final byte **INT**

The valuetype for `int` valued ItemFields.

DOUBLE

public static final byte **DOUBLE**

The valuetype for `double` valued ItemFields.

BYTE_ARRAY

public static final byte **BYTE_ARRAY**

The valuetype for `byte array` valued ItemFields.

BOOLEAN

public static final byte **BOOLEAN**

The valuetype for `boolean` valued ItemFields.

DATE

public static final byte **DATE**

The valuetype for `Date` valued ItemFields.

ITEM

public static final byte **ITEM**

The valuetype for `Item` valued ItemFields.

AGGREGATE

public static final byte **AGGREGATE**

The valuetype for `Aggregate` valued `ItemFields`.

Constructor Detail

ItemField

```
public ItemField(java.lang.String name,  
                 java.lang.String value)
```

Constructs new `ItemField` with the specified `String` value.

Parameters:

`name` - of the field.

`value` - `String` with which to initialize the field.

Throws:

`NullPointerException` - if the name is `null`.

ItemField

```
public ItemField(java.lang.String name,  
                 int value)
```

Constructs new `ItemField` with the specified `int` value.

Parameters:

`name` - of the field.

`value` - `int` with which to initialize the field.

Throws:

`NullPointerException` - if the name is `null`.

ItemField

```
public ItemField(java.lang.String name,  
                 double value)
```

Constructs new `ItemField` with the specified `double` value.

Parameters:

`value` - `double` with which to initialize the field.

Throws:

`NullPointerException` - if the name is `null`.

ItemField

```
public ItemField(java.lang.String name,  
                 byte[] value)
```

Constructs new ItemField with the specified byte array value.

Parameters:

name - of the field.

value - byte array with which to initialize the field.

Throws:

NullPointerException - if the name is null.

ItemField

```
public ItemField(java.lang.String name,  
                 boolean value)
```

Constructs new ItemField with the specified boolean value.

Parameters:

name - of the field.

value - boolean with which to initialize the field.

Throws:

NullPointerException - if the name is null.

ItemField

```
public ItemField(java.lang.String name,  
                 java.util.Date value)
```

Constructs new ItemField with the specified Date value.

Parameters:

name - of the field.

value - Date with which to initialize the field.

Throws:

NullPointerException - if the name is null.

ItemField

```
public ItemField(java.lang.String name,  
                 Item value)
```

Constructs new ItemField with the specified Item value.

Parameters:

name - of the field.

value - Item with which to initialize the field.

Throws:

NullPointerException - if the name is null.

Method Detail

getFieldName

```
public java.lang.String getFieldName()
```

Returns the name string of this field.

Returns:

the name of the field.

getValueType

```
public byte getValueType()
```

Returns the type of the value. For example, BYTE_ARRAY is the data type for byte[] values.

Returns:

the valuetype.

getInt

```
public int getInt()  
    throws java.lang.IllegalStateException
```

Returns the integer value of this field.

Returns:

the integer value of this field.

Throws:

java.lang.IllegalStateException - thrown if the field stores a type other than integer.

getDouble

```
public double getDouble()  
    throws java.lang.IllegalStateException
```

Returns the double value of this field.

Returns:

the double value of this field.

Throws:

java.lang.IllegalStateException - thrown if the field stores a type other than double.

getString

```
public java.lang.String getString()  
    throws java.lang.IllegalStateException
```

Returns the String value of this field.

Returns:

the String value of this field.

Throws:

java.lang.IllegalStateException - thrown if the field stores a type other than String.

getBoolean

```
public boolean getBoolean()  
    throws java.lang.IllegalStateException
```

Returns the boolean value of this field.

Returns:

the boolean value of this field.

Throws:

java.lang.IllegalStateException - thrown if the field stores a type other than boolean.

getDate

```
public java.util.Date getDate()  
    throws java.lang.IllegalStateException
```

Returns the Date value of this field.

Returns:

the Date value of this field.

Throws:

java.lang.IllegalStateException - thrown if the field stores a type other than Date.

getByteArray

```
public byte[] getByteArray()  
    throws java.lang.IllegalStateException
```

Returns the byte array value of this field.

Returns:

the byte array value of this field.

Throws:

java.lang.IllegalStateException - thrown if the field stores a type other than byte array.

getItem

```
public Item getItem()  
    throws java.lang.IllegalStateException
```

Returns the `Item` value of this field.

Returns:

the `Item` value of this field.

Throws:

`java.lang.IllegalStateException` - is thrown if this `ItemField` does not contain an `Item`.

setInt

```
public void setInt(int value)  
    throws java.lang.IllegalStateException
```

Sets the `int` value of this field.

Parameters:

`value` - the `int` value to store.

Throws:

`java.lang.IllegalStateException` - if the field stores a datatype other than an integer.

setDouble

```
public void setDouble(double value)  
    throws java.lang.IllegalStateException
```

Sets the `double` value of this field.

Parameters:

`value` - the `double` value to store.

Throws:

`java.lang.IllegalStateException` - if the field stores a datatype other than double.

setString

```
public void setString(java.lang.String value)  
    throws java.lang.IllegalStateException
```

Sets the `String` value of this field.

Parameters:

`value` - the `String` to store. It may be set to `null`.

Throws:

`java.lang.IllegalStateException` - if the field stores a datatype other than `String`.

setBoolean

```
public void setBoolean(boolean value)
    throws java.lang.IllegalStateException
```

Sets the `boolean` value of this field.

Parameters:

`value` - the `boolean` value to store.

Throws:

`java.lang.IllegalStateException` - if the field stores a datatype other than `boolean`.

setDate

```
public void setDate(java.util.Date value)
    throws java.lang.IllegalStateException
```

Sets the `Date` value of this field.

Parameters:

`value` - the `Date` to store. It may be set to `null`.

Throws:

`java.lang.IllegalStateException` - if the field stores a datatype other than `Date`.

setItem

```
public void setItem(Item value)
    throws java.lang.IllegalStateException
```

Sets the `Item` value of this field.

Parameters:

`value` - the `Item` to store. It may be set to `null`.

Throws:

`java.lang.IllegalStateException` - if the field stores a datatype other than `Item`.

setByteArray

```
public void setByteArray(byte[] value)
    throws java.lang.IllegalStateException
```

Sets the `byte` array value of this field.

Parameters:

`value` - the `byte` array to store. It may be set to `null`.

Throws:

`java.lang.IllegalStateException` - if the field stores other datatype than this method sets.

getParameters

```
public Parameter[] getParameters()
```

Gets an array of the Parameters of this field or `null` if no parameters are present.

Returns:

an array of Parameter objects or `null` if no parameters are present.

addParameter

```
public boolean addParameter(Parameter parameter)
```

Adds the Parameter object to this field's set of parameters. If a Parameter object with same name and value is already in the set this parameter is not added and this method returns false. Multiple Parameters may be added with different values. Parameter names and values are compared using `String.equals`.

Returns:

true if the parameter was added, false if a parameter of the same name and value is already present.

Throws:

`NullPointerException` - if parameter is `null`.

removeParameter

```
public boolean removeParameter(Parameter parameter)
```

Removes the specified Parameter object from this ItemField. If the parameter is the same (using `==`) as a Parameter in this ItemField then the parameter will be removed.

Parameters:

`parameter` - the Parameter to be removed.

Returns:

true if a identical Parameter was found and was removed.

Throws:

`NullPointerException` - if parameter is `null`.

hasParameter

```
public boolean hasParameter(java.lang.String name)
```

Checks if this field has at least one parameter with the name. If the field has a parameter object with the same name this method returns true.

Parameters:

`name` - of a Parameter to find in this ItemField.

Returns:

true if a Parameter is found with the same name.

Throws:

NullPointerException - if name is `null`.

getParameterValues

```
public java.lang.String[] getParameterValues(java.lang.String name)
```

Get the values of all Parameters with the specified name. Any array of the values is returned. The array will contain `null` for any parameter that does not have a value.

Parameters:

name - of the Parameter(s) for which to get the values.

Returns:

an array of the values of Parameters with the specified name. `null` is returned if there are no Parameters with the same name. The array will contain `null` for any Parameter with no value.

Throws:

NullPointerException - is thrown if the name is `null`.

getLabel

```
public java.lang.String getLabel()
```

Return the label of this field.

Returns:

the label for this ItemField. `null` is returned if it has not been set.

setLabel

```
public void setLabel(java.lang.String label)
```

Sets user defined label associated with this field.

Parameters:

label - new value of label

toString

```
public java.lang.String toString()
```

Returns a String representation of the value of this ItemField.

Overrides:

toString in class java.lang.Object

compareTo

```
public int compareTo(java.lang.Object obj)
```

Compares the values of this ItemField and another ItemField of the same type. The other object must be an ItemField whose value type is equal to the value type of this ItemField. The value of this ItemField is compared to the value of the other ItemField and the method returns 0 if the values are equal, less than zero if this value is "less than" the other value and greater than zero if this value is "greater than" the other value.

Parameters:

obj - the ItemField to be compared.

Returns:

a negative integer, zero, or a positive integer as this object is less than, equal to, or greater than the specified object.

Throws:

ClassCastException - if the specified object's type prevents it from being compared to this ItemField.

equals

```
public boolean equals(java.lang.Object obj)
```

Compares this ItemField to the specified object. The result is true if and only if the argument is not null and is an ItemField with the same name, value type, parameters, and value as this ItemField.

Overrides:

equals in class java.lang.Object

Parameters:

obj - the reference object with which to compare.

Returns:

true if this object is non-null, is a ItemField and has the same name, value type, parameters and value, otherwise false.

hashCode

```
public int hashCode()
```

Get the hashcode for this ItemField.

Overrides:

hashCode in class java.lang.Object

Returns:

the hashcode for this ItemField.

javax.pim.database **Interface Iterator**

public interface **Iterator**

Interface provided for JDK 1.1 compatibility. This interface will extend java.util.Iterator in Java Platform 2 compatible machines.

Method Summary

boolean	<u>hasNext</u> () Returns <code>true</code> if the iteration has more elements.
java.lang.Object	<u>next</u> () Returns the next element in the iteration.
void	<u>remove</u> () Removes from the underlying collection the last element returned by the iterator (optional operation).

Method Detail

hasNext

public boolean **hasNext**()

Returns `true` if the iteration has more elements. (In other words, returns `true` if `next` would return an element rather than throwing an exception.)

Returns:

`true` if the iterator has more elements.

next

public java.lang.Object **next**()

Returns the next element in the iteration.

Throws:

NoSuchElementException - iteration has no more elements.

remove

```
public void remove()
```

Removes from the underlying collection the last element returned by the iterator (optional operation). This method can be called only once per call to `next`. The behavior of an iterator is unspecified if the underlying collection is modified while the iteration is in progress in any way other than by calling this method.

Throws:

`UnsupportedOperationException` - if the `remove` operation is not supported by this `Iterator`.
`java.lang.IllegalStateException` - if the `next` method has not yet been called, or the `remove` method has already been called after the last call to the `next` method.

javax.pim.database ***Class Parameter***

```
java.lang.Object
|
+-- javax.pim.database.Parameter
```

```
public final class Parameter
extends java.lang.Object
```

Parameter objects hold a name and optional value. Each ItemField can have set of Parameter objects that contain information about the field or the field value. Parameter's are provided to specify such information as the location of an alternate text representation for a field value, the language of a text field value and other attributes. Parameters are immutable and cannot changed.

The Parameter name cannot be `null` but its value can.

Constructor Summary

<u>Parameter</u> (java.lang.String name)	
Constructs a new Parameter object with the specified name.	
<u>Parameter</u> (java.lang.String name, java.lang.String value)	
Constructs new Parameter object with specified name and value.	

Method Summary	
boolean	<u>equals</u> (java.lang.Object obj) Compares this parameter to the specified object.
java.lang.String	<u>getName</u> () This method returns the name of this Parameter object.
java.lang.String	<u>getValue</u> () Return the value of this Parameter object.
int	<u>hashCode</u> () Get the hashCode for this Parameter.
boolean	<u>match</u> (Parameter parameter) Return true if this parameter is equal to another or has no value and the same name as another parameter.
java.lang.String	<u>toString</u> () Return the value of this Parameter as a string.

Methods inherited from class java.lang.Object
getClass, notify, notifyAll, wait, wait, wait

Constructor Detail

Parameter

```
public Parameter(java.lang.String name,
                  java.lang.String value)
```

Constructs new Parameter object with specified name and value.

Parameters:

name - the name of the Parameter; may not be null.
value - the value of the Parameter.

Parameter

```
public Parameter(java.lang.String name)
```

Constructs a new Parameter object with the specified name. The value attribute of this Parameter is `null`.

Parameters:

name - the name of the Parameter; may not be `null`.

Method Detail

getValue

```
public java.lang.String getValue()
```

Return the value of this Parameter object.

Returns:

the value of the Parameter. Returns `null` if this Parameter has no value.

getName

```
public java.lang.String getName()
```

This method returns the name of this Parameter object.

Returns:

the name of the Parameter.

equals

```
public boolean equals(java.lang.Object obj)
```

Compares this parameter to the specified object. The result is true if and only if the argument is not `null` and is a Parameter object that represents the same parameter as this object.

Overrides:

equals in class java.lang.Object

Parameters:

obj - the reference object with which to compare.

Returns:

true if this object is non-null, is a Parameter and has the same name and value, otherwise false.

hashCode

```
public int hashCode()
```

Get the hashCode for this Parameter.

Overrides:

hashCode in class java.lang.Object

Returns:

the hashCode for this Parameter.

match

public boolean **match**(Parameter parameter)

Return true if this parameter is equal to another or has no value and the same name as another parameter.

Parameters:

`parameter` - the other Parameter with which to compare.

Returns:

true if the parameters are equal or if the value of this parameter is `null` and the names are the same.

toString

public java.lang.String **toString**()

Return the value of this Parameter as a string.

Overrides:

toString in class java.lang.Object

Returns:

the value of the Parameter; may be `null`.

Package javax.pim.userprofile

The UserProfile package defines objects to get and set the current user contact information, which is returned in the same format as the AddressBook package.

See:

Description

Interface Summary	
<u>UserProfileImpl</u>	The implementation of the UserProfile.

Class Summary	
<u>UserProfile</u>	Provide information about the current user.

Package javax.pim.userprofile Description

The UserProfile package defines objects to get and set the current user contact information, which is returned in the same format as the AddressBook package. For example, in a GSM phone, the User Profile API provides access to the SIM card information for the owner which might include the user's name and other attributes that identify the user.

Package Features

The UserProfile API provides the following features:

- Provides an application with information about the current user or owner of the device
- Can set the current user information (subject to security restrictions)

Design Goals

The primary design goal for the UserProfile API is to provide applications with the ability to set authorized user profiles for the device.

UserProfile provides the `getCurrentUser()` method to get current user information. Each device uses the `UserProfileImpl`, which is the interface that implements the `UserProfile` class, to implement the `getCurrentUser` functions. That implementation then can provide as needed the user information such as preferred phone numbers to be accessed by an application. An application uses the

`getPreferredPhoneNumbers()` method from the `ContactDatabase` class to obtain preferred phone numbers for a specific user profile as illustrated in the following sample code.

UserProfile Sample Code

```
/*
 * Copyright 1999 Sun Microsystems Inc, All rights reserved.
 */

import javax.pim.userprofile.*;
import javax.pim.addressbook.*;
import javax.pim.database.*;

/**
 * This example program shows how to lookup the preferred phone numbers for
 * the current user.
 */
public class ProfileSample1 {

    /**
     * Get the current user and print their phone numbers.
     */
    public static void main(String[] args) {

        try {
            ContactCard user = UserProfile.getCurrentUser();

            // Get the array of preferred phone numbers and print them.
            String[] tel =
                user.getPreferredPhoneNumbers(ContactDatabase.VOICE);
            for (int i = 0; i < tel.length; i++) {
                System.out.println("Name: " + user.getName() +
                                   ": " + tel[i]);
            }
        } catch (DatabaseException ex) {
            System.out.println("Error getting user information: " +
                               ex.getMessage());
        }
    }
}
```

Hierarchy For Package javax.pim.userprofile

Package Hierarchies:

All Packages

Class Hierarchy

- class java.lang.Object
 - class javax.pim.userprofile.**UserProfile**

Interface Hierarchy

- interface javax.pim.userprofile.**UserProfileImpl**
-
-

javax.pim.userprofile **Class UserProfile**

```
java.lang.Object
|
+-- javax.pim.userprofile.UserProfile
```

```
public class UserProfile
    extends java.lang.Object
```

Provide information about the current user. The information is provided in a `ContactCard` from the `AddressBook` package.

The implementation of the `UserProfile` must be set prior to its first use. The implementation may be initialized from the system property `"javax.pim.userprofile.implementation"`.

See Also:

[ContactCard](#)

Method Summary

static <u>ContactCard</u>	<u>getCurrentUser()</u> Use this method to get the current user profile information.
static void	<u>setCurrentUser(ContactCard userinfo)</u> Set the current user's profile information.
static void	<u>setImplementation(UserProfileImpl implementation)</u> Set the implementation object to which current user calls will be delegated.

Methods inherited from class java.lang.Object

`equals`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Method Detail

setImplementation

```
public static void setImplementation(UserProfileImpl implementation)
```

Set the implementation object to which current user calls will be delegated.

Parameters:

implementation - an instance of the UserProfileImpl api.

Throws:

java.lang.IllegalStateException - is thrown if the implementation has already been set.

getCurrentUser

```
public static ContactCard getCurrentUser()  
                                throws java.lang.SecurityException
```

Use this method to get the current user profile information. It delegates to the implementation object set with setImplementation.

Returns:

the current user's profile object; null may be returned if there is no current user.

Throws:

java.lang.SecurityException - is thrown if the user does not have permission to do this

java.lang.IllegalStateException - is thrown if the implementation has not been set.

setCurrentUser

```
public static void setCurrentUser(ContactCard userinfo)  
                                throws java.lang.SecurityException
```

Set the current user's profile information. It delegates to the object set with setImplementation. The information from the ContactCard is copied from the userinfo ContactCard and stored internally.

Parameters:

userinfo - the current user's contact information; null may be used to indicate there is no current user.

Throws:

java.lang.SecurityException - thrown if the user does not have permission to do this.

java.lang.IllegalStateException - is thrown if the implementation has not been set.

javax.pim.userprofile **Interface UserProfileImpl**

public interface **UserProfileImpl**

The implementation of the UserProfile.

Method Summary

<u>ContactCard</u>	<u>getCurrentUser</u> () Use this method to get the current user profile information.
void	<u>setCurrentUser</u> (<u>ContactCard</u> userinfo) Set the current user's profile information.

Method Detail

getCurrentUser

public ContactCard **getCurrentUser**()
throws java.lang.SecurityException

Use this method to get the current user profile information.

Returns:

the current user's profile object

Throws:

java.lang.SecurityException - thrown if the caller does not have permission to do this

setCurrentUser

public void **setCurrentUser**(ContactCard userinfo)
throws java.lang.SecurityException

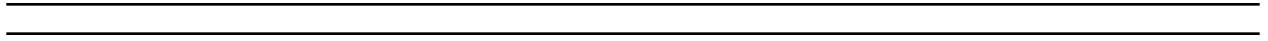
Set the current user's profile information.

Parameters:

userinfo - the current user's profile object

Throws:

java.lang.SecurityException - thrown if the caller does not have permission to do this



Package javax.ce.install

Table of Contents

Package javax.ce.install	1
Package javax.ce.install	1
Package javax.ce.install Description	1
Package Features	1
Design Goals	1
Open Issues	2
Install Code Sample	2
javax.ce.install Class Hierarchy	4
Hierarchy For Package javax.ce.install	4
Class Hierarchy	4
Class Installer	5
javax.ce.install Class Installer	5
Installer	7
getInstaller	7
setInstaller	7
install	7
remove	8
list	8
getAttributes	9
Class InstallInfo	10
javax.ce.install Class InstallInfo	10
MAIN_CLASS	11
MAIN_ICON	11
IMPLEMENTATION_TITLE	11
IMPLEMENTATION_VERSION	12
IMPLEMENTATION_VENDOR	12
SPECIFICATION_TITLE	12
SPECIFICATION_VERSION	12
SPECIFICATION_VENDOR	12
getName	12
getAttribute	13
Package javax.net.datagram	14
Package javax.net.datagram	14
Package javax.net.datagram Description	15
Design Goals	15
Network Datagram API Sample Code	15
Abstract Addressing	17
Concrete Addressing	17
SMS Address Type	17
UDP Datagram Address Type	18
WDP over SMS Datagram Address Type	18

Notes for implementors of DatagramService	19
Classes implementing Address	19
Using the local Addressbook for a DatagramNameService	19
javax.net.datagram Class Hierarchy	21
Hierarchy For Package javax.net.datagram	21
Class Hierarchy	21
Interface Hierarchy	21
Interface Address	22
javax.net.datagram Interface Address	22
toString	22
Class MalformedAddressException	24
javax.net.datagram Class MalformedAddressException	24
MalformedAddressException	25
Class AddressNotSupportedException	26
javax.net.datagram Class AddressNotSupportedException	26
AddressNotSupportedException	27
Class IntermittentNetworkException	28
javax.net.datagram Class IntermittentNetworkException	28
IntermittentNetworkException	29
IntermittentNetworkException	29
getCause	29
Class Datagram	30
javax.net.datagram Class Datagram	30
Datagram	31
Datagram	32
Datagram	32
getData	32
setData	33
getLength	33
setLength	33
getOffset	33
setOffset	34
getAddress	34
setAddress	34
Class DatagramNameService	35
javax.net.datagram Class DatagramNameService	35
lookup	36
lookupAll	37
addNameService	38
removeNameService	38
Class DatagramService	39
javax.net.datagram Class DatagramService	39
getService	42
getServerService	42
parseAddress	43
send	43

receive	44
receive	44
getNominalLength	45
getMaximumLength	45
close	45
getAddress	46
Class DatagramServiceFactory	47
javax.net.datagram Class DatagramServiceFactory	47
addService	48
removeService	48
Package javax.pim.addressbook	49
Package javax.pim.addressbook	49
Package javax.pim.addressbook Description	49
Package Features	49
Design Goals	50
AddressBook Code Sample	50
javax.pim.addressbook Class Hierarchy	53
Hierarchy For Package javax.pim.addressbook	53
Class Hierarchy	53
Class ContactCard	54
javax.pim.addressbook Class ContactCard	54
ContactCard	55
getGroups	56
getName	56
setFormattedName	56
getPhoneNumbers	57
getPreferredPhoneNumbers	57
getOrganization	57
Class ContactDatabase	58
javax.pim.addressbook Class ContactDatabase	58
DEFAULT_ADDRESSBOOK	68
DEFAULT_SIM	69
SIMSLOT	69
FAMILY_NAME	69
GIVEN_NAME	69
ADDITIONAL_NAMES	69
PREFIXES	69
SUFFIXES	69
PO_BOX	70
EXT_ADR	70
STREET_ADR	70
LOCALITY	70
REGION	70
POSTAL_CODE	70
COUNTRY	71
TYPE	71

N	71
FN	71
NICKNAME	71
ORG	72
CATEGORIES	72
TITLE	72
ADR	72
LABEL	73
BDAY	73
TEL	73
EMAIL	73
MAILER	74
TZ	74
GEO	74
ROLE	74
NOTE	75
URL	75
KEY	75
LOGO	75
PHOTO	76
AGENT	76
REV	76
SORT_STRING	76
SOUND	77
UID	77
CLASS	77
DOM	77
INTL	77
POSTAL	78
PARCEL	78
HOME	78
WORK	78
PREF	78
VOICE	78
FAX	78
MSG	79
CELL	79
PAGER	79
BBS	79
MODEM	79
CAR	79
ISDN	80
VIDEO	80
PCS	80
INTERNET	80
X400	80

openDatabase	80
openDatabase	81
deleteDatabase	81
createDatabase	81
getDatabaseNames	82
cardItems	82
groupItems	82
templateItems	83
addItem	83
updateItem	84
Class ContactGroup	85
javax.pim.addressbook Class ContactGroup	85
ContactGroup	86
addContact	86
removeContact	87
contactCount	87
getContacts	87
Class ContactTemplate	88
javax.pim.addressbook Class ContactTemplate	88
ContactTemplate	89
newContactCard	89
Package javax.pim.calendar	90
Package javax.pim.calendar	90
Package javax.pim.calendar Description	90
Design Goals	90
Calendar Code Sample	91
javax.pim.calendar Class Hierarchy	94
Hierarchy For Package javax.pim.calendar	94
Class Hierarchy	94
Class CalendarDatabase	95
javax.pim.calendar Class CalendarDatabase	95
DEFAULT_CALEDAR	106
ATTACH	106
CATEGORIES	106
CLASS	106
COMMENT	106
DESCRIPTION	106
GEO	106
LOCATION	107
PERCENT_COMPLETE	107
PRIORITY	107
RESOURCES	107
STATUS	107
SUMMARY	107
COMPLETED	108
DTEND	108

DUE	108
DTSTART	108
DURATION	108
TRANSP	108
ATTENDEE	109
CONTACT	109
ORGANIZER	109
RELATED_TO	109
URL	109
UID	109
CREATED	109
LAST_MODIFIED	110
SEQUENCE	110
REQUEST_STATUS	110
ALTREP	110
CN	110
CUTYPE	110
DELEGATED_FROM	111
DELEGATED_TO	111
DIR	111
ENCODING	111
FMTYPE	111
LANGUAGE	112
MEMBER	112
PARTSTAT	112
RELTYPE	112
ROLE	112
RSVP	113
SENT_BY	113
TZID	113
VALUE	113
INDIVIDUAL	113
GROUP	114
RESOURCE	114
ROOM	114
UNKNOWN	114
NEEDS_ACTION	114
ACCEPTED	114
DECLINED	114
TENTATIVE	115
DELEGATED	115
IN_PROCESS	115
PARENT	115
CHILD	115
SIBLING	115
CHAIR	116

REQ_PARTICIPANT	116
OPT_PARTICIPANT	116
NON_PARTICIPANT	116
TRUE	116
FALSE	116
BINARY	116
TEXT	117
INTEGER	117
FLOAT	117
URI	117
PUBLIC	117
PRIVATE	117
CONFIDENTIAL	117
CONFIRMED	118
CANCELLED	118
OPAQUE	118
TRANSPARENT	118
openDatabase	118
openDatabase	119
deleteDatabase	119
createDatabase	119
addItem	119
updateItem	120
getDatabaseNames	120
entryItems	121
items	121
toDoItems	122
Class CalendarEntry	123
javax.pim.calendar Class CalendarEntry	123
CalendarEntry	125
setRepeat	125
getRepeat	125
getDates	125
getStartDate	126
getEndDate	126
setStartDate	126
setEndDate	126
getDescription	127
setDescription	127
getAttendees	127
Class CalendarToDo	128
javax.pim.calendar Class CalendarToDo	128
CalendarToDo	129
getDueDate	130
setDueDate	130
getDescription	130

setDescription	130
getPriority	131
setPriority	131
Class Repeat	132
javax.pim.calendar Class Repeat	132
Repeat	134
addRepeatRule	134
removeRepeatRule	134
getRepeatRules	134
addRepeatDate	135
removeRepeatDate	135
getRepeatDates	135
addExceptRule	135
removeExceptRule	135
getExceptRules	136
addExceptDate	136
removeExceptDate	136
getExceptDates	136
dates	137
equals	137
Class RepeatRule	138
javax.pim.calendar Class RepeatRule	138
SUNDAY	142
MONDAY	142
TUESDAY	142
WEDNESDAY	142
THURSDAY	142
FRIDAY	143
SATURDAY	143
MINUTELY	143
HOURLY	143
DAILY	143
WEEKLY	143
MONTHLY	143
YEARLY	144
RepeatRule	144
setFrequency	144
getFrequency	144
getInterval	145
setInterval	145
getEndDate	145
setEndDate	145
setCount	146
getCount	146
setWeekStart	146
getWeekStart	146

addMinute	147
removeMinute	147
getMinute	147
addHour	147
removeHour	148
getHour	148
addWeekday	148
removeWeekday	148
getWeekday	149
addMonthDay	149
removeMonthDay	149
getMonthDay	149
addYearDay	150
removeYearDay	150
getYearDay	150
addWeeknum	150
removeWeeknum	151
getWeeknum	151
addMonth	151
removeMonth	151
getMonth	151
addSetPos	152
removeSetPos	152
getSetPos	152
dates	152
equals	153
Package javax.pim.database	154
Package javax.pim.database	154
Package javax.pim.database Description	155
javax.pim.database Class Hierarchy	156
Hierarchy For Package javax.pim.database	156
Class Hierarchy	156
Interface Hierarchy	156
Class AggregateField	157
javax.pim.database Class AggregateField	157
AggregateField	159
size	159
getField	159
removeField	159
addField	160
insertField	160
setField	160
equals	161
hashCode	161
toString	161
toString	162

Interface Database	163
javax.pim.database Interface Database	163
items	164
items	165
items	165
close	166
addItem	166
updateItem	167
deleteItem	167
isSupported	168
addDatabaseListener	168
removeDatabaseListener	168
Class DatabaseEvent	169
javax.pim.database Class DatabaseEvent	169
DATABASE_ITEM_DELETED	170
DATABASE_ITEM_ADDED	170
DATABASE_ITEM_UPDATED	170
DATABASE_CLOSED	171
DatabaseEvent	171
DatabaseEvent	171
getType	171
getItem	172
dispatch	172
Class DatabaseException	173
javax.pim.database Class DatabaseException	173
DatabaseException	174
DatabaseException	174
Interface DatabaseFactory	175
javax.pim.database Interface DatabaseFactory	175
openDatabase	175
createDatabase	176
deleteDatabase	176
getDatabaseNames	176
Interface DatabaseListener	178
javax.pim.database Interface DatabaseListener	178
databaseClosed	178
databaseItemDeleted	179
databaseItemAdded	179
databaseItemUpdated	179
Class DatabaseUpdateException	180
javax.pim.database Class DatabaseUpdateException	180
DatabaseUpdateException	181
DatabaseUpdateException	181
Class Item	182
javax.pim.database Class Item	182
Item	183

getField	184
getFields	184
getFields	184
addField	184
removeField	185
size	185
isSameItem	185
Class ItemField	186
javax.pim.database Class ItemField	186
STRING	189
INT	190
DOUBLE	190
BYTE_ARRAY	190
BOOLEAN	190
DATE	190
ITEM	190
AGGREGATE	190
ItemField	191
ItemField	191
ItemField	191
ItemField	192
ItemField	192
ItemField	192
ItemField	192
getFieldName	193
getValueType	193
getInt	193
getDouble	193
getString	194
getBoolean	194
getDate	194
getByteArray	194
getItem	195
setInt	195
setDouble	195
setString	195
setBoolean	196
setDate	196
setItem	196
setByteArray	196
getParameters	197
addParameter	197
removeParameter	197
hasParameter	197
getParameterValues	198
getLabel	198

setLabel	198
toString	198
compareTo	199
equals	199
hashCode	199
Interface Iterator	201
javax.pim.database Interface Iterator	201
hasNext	201
next	201
remove	202
Class Parameter	203
javax.pim.database Class Parameter	203
Parameter	204
Parameter	204
getValue	205
getName	205
equals	205
hashCode	205
match	206
toString	206
Package javax.pim.userprofile	207
Package javax.pim.userprofile	207
Package javax.pim.userprofile Description	207
Package Features	207
Design Goals	207
UserProfile Sample Code	208
javax.pim.userprofile Class Hierarchy	209
Hierarchy For Package javax.pim.userprofile	209
Class Hierarchy	209
Interface Hierarchy	209
Class UserProfile	210
javax.pim.userprofile Class UserProfile	210
setImplementation	211
getCurrentUser	211
setCurrentUser	211
Interface UserProfileImpl	212
javax.pim.userprofile Interface UserProfileImpl	212
getCurrentUser	212
setCurrentUser	212